This document is for the sole use of Rajat G of Analog

SystemVerilog Assertions

Course Version 6.0

Lecture Manual **Revision 1.0**

cadence®

This document is for the sole use of Rajat G of Analog

© 1990-2026 Cadence Design Systems, Inc. All rights reserved.

Printed in the United States of America.

Cadence Design Systems, Inc. (Cadence), 2655 Seely Ave., San Jose, CA 95134, USA.

Trademarks: Trademarks and service marks of Cadence Design Systems, Inc. (Cadence) contained in this document are attributed to Cadence with the appropriate symbol. For queries regarding Cadence trademarks, contact the corporate legal department at the address shown above or call 1-800-862-4522.

All other trademarks are the property of their respective holders.

Restricted Print Permission: This publication is protected by copyright, and any unauthorized use of this publication may violate copyright, trademark, and other laws. Except as specified in this permission statement, this publication may not be copied, reproduced, modified, published, uploaded, posted, transmitted, or distributed in any way without prior written permission from Cadence. This statement grants you permission to print one (1) hard copy of this publication subject to the following conditions:

- The publication may be used solely for personal, informational, and noncommercial purposes;
- The publication may not be modified in any way;
- Any copy of the publication or portion thereof must include all original copyright, trademark, and other proprietary notices, and this permission statement and
- Cadence reserves the right to revoke this authorization at any time, and any such use shall be discontinued immediately upon written notice from Cadence.

Disclaimer: Information in this publication is subject to change without notice and does not represent a commitment on the part of Cadence. The information contained herein is the proprietary and confidential information of Cadence or its licensors and is supplied subject to and may be used only by Cadence customers in accordance with a written agreement between Cadence and the customer.

Except as may be explicitly set forth in such agreement, Cadence does not make and expressly disclaims any representations or warranties as to the completeness, accuracy, or usefulness of the information contained in this document. Cadence does not warrant that the use of such information will not infringe any third party rights, nor does Cadence assume any liability for damages or costs of any kind that may result from the use of such information.

Restricted Rights: Use, duplication, or disclosure by the Government is subject to restrictions as set forth in FAR52.227-14 and DFAR252.227-7013 et seq. or its successor.

This document is for the sole use of Rajat G of Analog

Table of Contents

SystemVerilog Assertions

Module 1 About This Course 2

Module 2 Assertion-Based Verification 11

Module 3 Simple Boolean Assertions 25

Module 4 Sequences 60

Module 5 Coverage 89

Module 6 Liveness Properties 106

Module 7 Property Reuse 116

Module 8 Auxiliary Code 131

Module 9 Sequence Operations 143

Module 10 Advanced SVA Features 168

Module 11 Constructs Which Form Properties 184

Submodule Constructs with Pragmatic Value 187

Submodule Constructs with Little Pragmatic Value 205

Module 12 The Verification Completeness Problem 224

Module 13 Course Conclusions 244

Module 14 Next Steps 250

This document is for the sole use of Rajat G of Analog



This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog



Module 1

About This Course

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog

Course Prerequisites

Before taking this course, you need to have already:

- One week's experience of writing Verilog or SystemVerilog
- Attended a [System]Verilog training course, for example, SystemVerilog for Design and Verification

All you need to know, for now, is how to:

1. Write a Boolean expression in [System]Verilog
2. Describe a module
3. Define an event control, for example, `@(posedge clk)`

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog

What's Coming

This course covers several versions of the SystemVerilog standard

- IEEE 1800 SystemVerilog Language Reference Manual 2005, 2009, 2017, 2023

This course is effectively in 2 parts:

- Part 1: SVA for Formal and Simulation users
- Part 2: SVA for Simulation users only

Why two parts?

- SVA describes design behavior using properties that are valid in both Formal and Simulation.
- Some of them show different behavior in Formal and Simulation.
- Formal requires a specific coding style, known as Formal-Friendly SVA.

Part 1 is SVA basic syntax and principles essential for both Formal and Simulation users

Part 2 is SVA constructs which, while supported by Formal tools, are not “Formal Friendly”

- These constructs might be useful in simulation
- It is also important to be aware of the limitations of those constructs

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog

Course Objectives
Part 1: SVA for Formal and Simulation Users

In this course, you

- Explain the motivation and role of Assertion-Based Verification (ABV)
- Differentiate between assertions, assumptions, and cover statements in SVA
- Describe how SVA captures design intent independent of simulation or formal tools
- Develop simple Boolean properties using SVA
- Construct sequences to express multi-cycle design behavior
- Apply SVA coverage to measure verification completeness
- Identify safety and liveness properties and their relevance to formal verification
- Reuse properties using parameterization and binding techniques
- Identify when auxiliary (helper) code is required to model complex behavior

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog

Course Objectives
Part 2: SVA for Simulation Users

In this course, you

- Apply advanced SVA constructs intended primarily for simulation.
- Construct complex temporal sequences using delays, repetition operators with ranges.
- Use sequence operators (`and, or, intersect, within, throughout`) to model protocol behavior.
- Analyze overlapping assertions and simulation-specific timing effects.
- Evaluate which SVA constructs are not formal-friendly and understand their limitations.

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog

Course Agenda

Part 1 – Formal and Simulation Users

- Assertion-Based Verification (ABV)
- Simple Boolean Assertions
  - Lab: Simple Assertions to Verify FSM
- Sequences
  - Lab: Sequence-Based Properties
- Coverage
- Liveness Properties
- Property Reuse
- Auxiliary Code
  - Lab: Auxiliary Code

Part 2 – Simulation Users Only

- Sequence Operation
  - Lab: Serial Peripheral Interface Verification
- Advanced SVA Features
- Constructs Which Form Properties
  - Submodule: Constructs with Pragmatic Value
  - Submodule: Constructs with Little Pragmatic Value
  - Lab: Property Forming Constructs Based Lab
- The Verification Completeness Problem

© Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog

Software and Licenses

For the software and licenses used in the labs for this course, go to:

https://www.cadence.com/en_US/home/training/all-courses/82165.html

The course was developed with Xcelium 2603 and Jasper 2603.

Product Name	Product ID	Purpose
Part 1		
Jasper® Formal Verification Platform	JGFVBASE	Formal Verification tools
Part 2		
Xcelium™ Single-Core	X300	Simulator
Verisium™ Debug	VERDBGS	Debug solution

© Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog

Become Cadence Certified by Earning a Digital Badge

Digital badges indicate mastery in a certain technology or skill and give managers and potential employers a way to validate your expertise.

- Cadence Training Services offers digital badges for our popular training courses.
- Your digital badge can be added to your email signature or social media platforms like LinkedIn or Facebook.

Benefits of Cadence Certified Digital Badges

- Validate expertise
  - Expand career opportunities
- Professional credibility
  - Stand apart from your peers
- For more information, go to www.cadence.com/training or email es_digitalbadge@cadence.com.



How do I register to take the exam?

- Log in to our Learning Management System, click on the course in your transcript, and go to the Content tab to locate the exam.

How long will it take to complete the exam?

- Most exams take 45 to 90 minutes to complete. You may retake the exam multiple times to pass the exam.

How do I access and use the digital badge?

- After you pass the exam, you get a digital badge and instructions on how to place it on social media sites.

How is the digital badge validated?

- Credly validates the digital badge as issued to you by Cadence and includes the details of the criteria you completed to earn the badge.

© Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog

Icons Used in This Class


Best Practice


Language/
Command Syntax


Concept/
Glossary


Frequently Asked
Questions/
Quiz


Error Message

Throughout this class, we use icons to draw your attention to certain kinds of information. Here are the icons we use, and what they mean.


Problem & Solution


Quick Reference


GUI and Command


How To


Lab List

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog



Module 2

Assertion-Based Verification

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog

Module Objectives

In this module, you will learn how to:

- Explain the concepts of Assertions and Covers
- Apply assertion directives appropriately in simulation and formal test environments
- Define concepts and practicalities of Assertion-Based Verification (ABV)

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog

What Is an Assertion?

An assertion is an embedded check-in code used during verification to automatically monitor whether specific conditions or sequences of events occur as expected. Assertions are primarily used to validate the behavior of a design.

Assertions generate warnings or errors when:

- A specified condition fails, or
- A specified sequence does not complete properly.

Assertions offer a way to significantly enhance productivity for designers

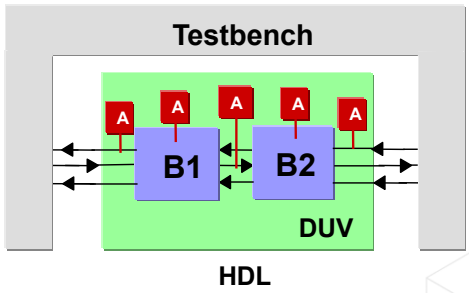
- Find bugs earlier and more easily

Checks without coverage are of little use

- We need to know what was exercised when the checks were performed
- This is fundamental to any verification methodology

Assertions are of two kinds

- Immediate Assertions
- Concurrent Assertions



13 © Cadence Design Systems, Inc. All rights reserved.

cadence

Definition: property – a collection of logical and temporal relationships between boolean expressions, sequential expressions, and other properties that represent a set of behaviors.

Definition: assertion – a statement that a given property is required to be true within a specific design – and a directive to verification tools to verify that it is true.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog

SystemVerilog Assertions (SVA) Overview

Defined as part of the SystemVerilog LRM (IEEE1800)

Very similar to Property Specification Language (PSL) IEEE1850 and the Temporal Language of Specman® **e** IEEE1647

Used to specify functional *properties* of a design

- A property is a general behavioral attribute used to characterize a design
  - What it should or should not do

SVA characteristics:

- Mathematically precise and computationally efficient
- Sufficiently expressive to specify “real world” design properties
- Uses Verilog syntax
- Embedded in functional code and ignored by synthesis

Supported by mainstream simulators and formal verification tools

 © Cadence Design Systems, Inc. All rights reserved.

cadence

A property is a general behavioral attribute used to characterize a design. It is similar to stating to a synthesis tool that the design must have the property that the clock can run at 250 MHz.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog

SVA Verification Directives

Property

Expression of design behavior

- A design behavior defined as logical and temporal relationships between Boolean and sequential expressions

Verification Directive Keywords

What the tool should do with the expressed design behavior

- Assert – An assertion
  - The tool must check that the property is always true
- assume – An assumption/constraint
  - In formal – The tool must only allow the DUT and inputs to behave in ways that never violate the property
  - In Simulation – Has no meaning in this context, so it is automatically converted into an assertion
- cover – A cover expression
  - In formal – Demonstrate that it is possible for the expression to be true
  - In Simulation – Count how many times the expression is true
- restrict – A readability intent statement
  - In formal – Exactly the same as assume
  - In Simulation – Has no meaning; hence, is ignored

The stated design behaviors in the SVA properties do not care if we are a simulation or formal user

© Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog

What Is a Cover?

A cover is a sequence of normal design behavior that one would expect to observe during verification.

The motivation for writing covers is to ensure that our verification is thorough.

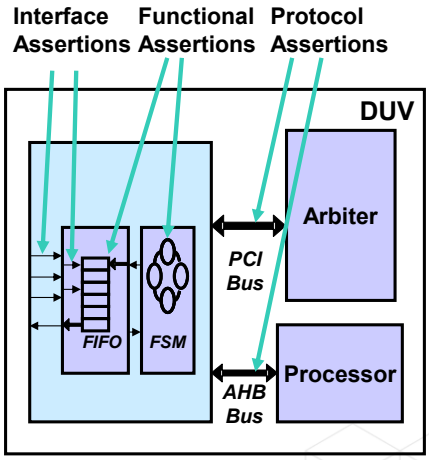
Unlike an assertion, a cover does not pass or fail; it simply tracks whether an interesting scenario happened or not, for example:

- My FIFO buffer can get full and then empty again
- I had a PCI bus transaction that was interrupted by a reset
- I observed all interesting variations of AHB transfer

In simulation, a cover is the count of how many times the sequence was observed

In formal, a cover is a demonstration of that sequence occurring

- Normally, the shortest trace



16 © Cadence Design Systems, Inc. All rights reserved.

cadence

Definition: property – a collection of logical and temporal relationships between boolean expressions, sequential expressions, and other properties that represent a set of behaviors.

Definition: assertion – a statement that a given property is required to be true within a specific design – and a directive to verification tools to verify that it is true.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog

SVA in Simulation Testbench



Testbench drives the DUV inputs, typically with randomized stimulus

SVA Assertions monitor and report:

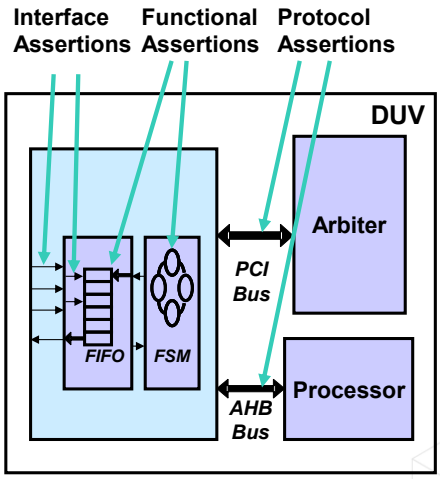
- Expected behavior
- Forbidden behavior
- Signal protocols

SVA Cover Statements monitor and report:

- A count of how many times specific sequences are observed

SVA Assumes are not relevant for simulation

- The simulator will automatically tu.rn assumes into asserts



17 © Cadence Design Systems, Inc. All rights reserved.

cadence

Static verification is verification that does not require a verification engineer to think of what stimulus to apply to verify a design.

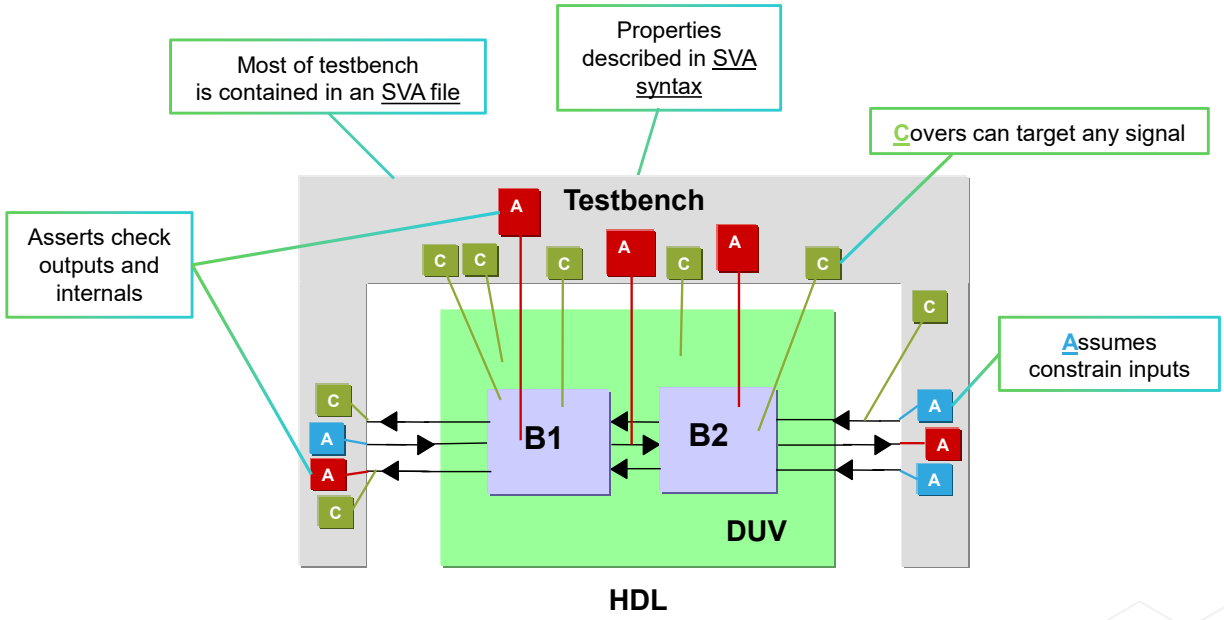
The design's "correctness" is verified using mathematical techniques. This is known as "formal verification." It is performed using symbolic proofs rather than applying simulation vectors. There are different kinds of "formal verification," Logic Equivalence Checking (LEC) and Functional Property Checking, for which there is no universally accepted name. In the present context, we are referring to Functional Property Checking.

Formal Verification (formal) is a fundamentally different paradigm to simulation. There are different considerations for effective ABV in formal vis-à-vis simulation. One of these is to ensure that the environment is set up correctly for the results of a formal proof to have any value. We will discuss how formal works and why assumptions are needed later in this course.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog

SVA in Formal Testbench



 © Cadence Design Systems, Inc. All rights reserved.

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog

What Is SVA Property?



A property is a design behavior defined by the logical and temporal relationships between Boolean and sequential expressions.

Improves observability

- Automatically and constantly checks behavior
- Can detect hard-to-find bugs embedded deep in the design
- Can non-intrusively apply properties, even to legacy code
- Isolates problem close to source
  - No need to propagate error to outputs and “track-back” to source

Improves verification efficiency

- Concisely and unambiguously document design intent
- Detect bugs earlier in the design cycle
- Trap and exit on errors, saving simulation cycles
- Writing assertions helps designers to better understand their design
- Writing assertions encourages designers to do more debugging

Assertions can be embedded in code to “travel with” design:

- From project to project – ideal for IP
- From tool to tool – ideal for design methodology integrity

19 © Cadence Design Systems, Inc. All rights reserved.

cadence

Assertions can be placed anywhere in your HDL hierarchy, enabling you to monitor design behavior locally and highlight problems as soon as they occur from the source of the problem. Conventional HDL testbenches need a problem to propagate to the design outputs before it can be detected, and then the problem has to be tracked back through the design hierarchy to discover the source.

Assertions can help detect problems as testbenches are developed, allowing earlier debugging. Assertions can also be used to terminate simulation on critical errors, thus saving simulation time.

Writing assertions encourages designers to think more clearly about their designs. This can help highlight problems without specific assertions being written. Verification efficiency is improved by designers having a better understanding of their designs and carrying out more debugging of their code.

Embedded assertions travel with a design as it is used in different projects, warning against unexpected or incorrect use of the IP block.

Embedded assertions also travel with a design from tool to tool (for example, simulation to formal verification), avoiding error-prone re-entry of design behavior definitions.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog

Who Writes Properties and Why?

Who	Type of Assertion	Reason	Example
System Architects	Abstract properties	Ensure implementation meets functional spec	An asynchronous FIFO does not indicate full and empty at the same time
RTL Designer	Functional Property	Implementation has properties the RTL designer intended it to have	When Read and Write pointers both 0, the FIFO indicates empty
Verification Engineer	Functional Coverage	Ensure test stimulus exercises the corner cases as required by the test plan	Did you read from full FIFO, write to the last empty FIFO location, etc.
IP Designer	Correct use of IP	Indicate to IP user that the IP is not being used correctly	FIFO was read when it was empty

 © Cadence Design Systems, Inc. All rights reserved.

cadence

Assertions are effective for many applications and at many levels.

Coverage is, strictly speaking, separate from assertions that properties are met, as you will see later in the course. However, SVA supports both assertions and coverage measurement, so both are mentioned here.

Note that for IP, assertions guard against misuse (and consequent complaints back to the IP provider).

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog

Practical Issues with Properties

Quality of test stimulus for simulation is still necessary

- Assertions are only useful if exercised by the test stimulus
  - My properties are not violated on this simulation run but ...

Extremely complex behaviors can be represented using SystemVerilog assertions.

However, it is important not to get carried away using complex SVA constructs

- A dedicated syntax for assertions is great.
- It is easy to shoot yourself in both feet using complex SVA constructs.
  - The syntax is “sugared”*
- Sometimes writing simple auxiliary Verilog code can help in assertion definition and specification**

 © Cadence Design Systems, Inc. All rights reserved.

cadence

*Sugaring of a syntax means that it looks very easy because there are very few characters used.

However, the reason for this is because difficult concepts have been hidden to allow complex behaviors to be described very concisely.

**This is known as Auxiliary code. It is just HDL code and is used purely for verification. We discuss this at length in a later chapter in the course.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog

Practical Issues with Properties (continued)

Assertions must be constructed with care

- Incorrect assertions can give misleading results
- Debugging an assertion can be difficult
- How do you know when enough assertions have been written?

Simulation overhead

Formal Verification capacity limitations (complexity)

Other issues, as you will see...

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

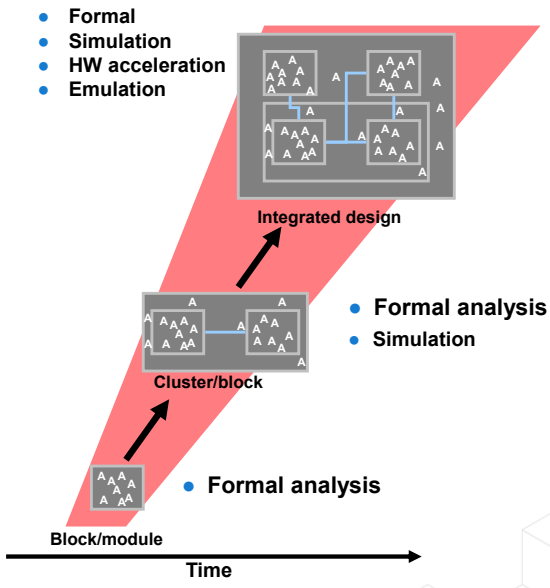
© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog

What Is Assertion-Based Verification (ABV)?

Assertion-based verification involves the systematic and formal application of assertions to describe and check design characteristics.

- Integration of clean blocks and modules significantly reduces effort and debug time
- Same assertions [re]used in:
  - Formal
  - Simulation
  - Acceleration/emulation
  - Cluster and full chip level
- Unified ABV flow results in:
  - Improved productivity and quality
  - Overall reduction in verification time/effort



23 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog

Summary

- Assertions can help reduce debugging time by identifying incorrect design behavior **when** and **where** it occurs
- Assertion-Based Verification:
  - Captures specifications
  - Captures design assumptions
  - Documents interfaces
  - Provides white-box visibility in simulation
  - Supports advanced verification technologies
    - Dynamic simulation
    - Functional coverage analysis
    - Static (formal) verification

 © Cadence Design Systems, Inc. All rights reserved.

cadence

Static (formal) verification is a symbolic proof, i.e., a mathematical proof. There is no need to think of what stimulus to apply hence there is no testbench.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog



Module 3

Simple Boolean Assertions

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog

Module Objectives

At the end of this module, you will be able to:

- Demonstrate simple assertions derived from Boolean expressions
- Explain the purpose and behavior of Immediate Assertions
- Demonstrate how concurrent assertions operates
- Construct Boolean expressions for SystemVerilog Assertions and describe the complete structure and components of an assertion
- Analyze counterintuitive clocking behaviors and apply correct clocking strategies
- Create conditional assertions using implication operators

© Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog

Terminology Introduction



Invariant

- A boolean condition that should always hold (namely, be true)
- For example, `A` and `B` are never both high



Temporal sequence

- A series of boolean conditions evaluated over time
  - Hence, a sequence of length 1 cycle is the same as a boolean
- For example, If `A` is high in one cycle, then `B` must be high in the next cycle, and `C` must be high cycle after that



Property

- A design behavior is defined as logical and temporal relationships between Boolean and sequential expressions

It's important to understand that a property and an assertion are not the same thing

An assertion is one example of a "Verification Directive"

- An instruction to the EDA tool to tell it what to do with the property

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog

What Is an Immediate Assertion?



An immediate assertion is a check that evaluates a Boolean expression at the exact moment the procedural code executes it. The expression is non-temporal and is interpreted the same way as the condition of a procedural *if* statement.

An assertion as a procedural statement

- Embedded within a procedural block
  - Usually! - deferred immediate assertions can also appear in concurrent code
- Only active within the block
- Similar to an if statement

```
always @(posedge clock)
  assert !(wr_en && rd_en);
```

Immediate Assertion

Instantaneous

- Checks a specific Boolean expression when executed in procedural code
- If expression evaluates to `Z`, `X` or `0`, assertion *fails*
- Error message reported when assertion fails

```
always @(posedge clock)
  if (wr_en && rd_en)
    $display("error");
```

Equivalent Verilog

28 © Cadence Design Systems, Inc. All rights reserved.

cadence

Immediate assertions are similar to if statements, but note that the assertion fails if the assertion expression evaluates to “untrue,” that is, Z, X, or 0.

When an assertion fails, a tool-specific message is written to the simulator transcript window containing the following minimum information:

File name and line number of the assertion

Assertion label (see next slide) or if unlabelled, the scope of the assertion

Severity level of the assertion (see second next slide) – default error

Simulation time at which the assertion failed

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog

Variants of Immediate Assertions

Simple immediate assertions

- Evaluation of the assertion uses the current (immediate) signal values when the line of code containing the assertion is evaluated

```
always @(posedge clock)
  assert !(wr_en && rd_en);
```

Deferred immediate assertions

- Evaluation of the assertion uses the signal values after signal updates in later “regions” of a timeslot
- Allows one to deal with glitches on combination signals in simulation
- Formal is cycle based so deferred immediate assertions are not required

```
always @(posedge clock)
  assert #0 !(full && empty);

always @(posedge clk2)
  assert final (K == L);
```



TB: What is a deferred Immediate Assertion

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog

Limitations of Immediate Assertions

- Instantaneous boolean check only
- Extra code is required around the assertion for complex or temporal checks
  - Substantial extra code is required to check even a simple protocol
- Complexity becomes comparable to that of design
- Requires as much debugging as design
- Poor readability as design intent is not obvious
- However, tools can discern that a check was intended
  - Allows verification management tools like Cadence® vManager to recognize a check was specified

```
@(posedge CE);
for (i = 0; i < 16; i++)
begin
  @(posedge SCLK or negedge CE);
  assert (CE)
  else begin // short pulse
    $error("short CE pulse");
    break;
  end
...
```

```
always @(posedge clk)
  case (state)
    `state1:  ...
    `state2:  ...
     default:
       adef: assert(1'b0)
         $warning( "case default");
  endcase
```

Assertion **just** used to report problem

30 © Cadence Design Systems, Inc. All rights reserved.

Immediate assertions are useful, are ignored by synthesis tools, and should be used liberally in RTL code.

However, as they are procedural statements checking instantaneous boolean conditions, they usually have to be surrounded by extra code to control when the assertion is actually evaluated. Commonly this extra code is substantial and requires debugging and verification, just like design code.

For complex checks, the immediate assertion is reduced to simply reporting the error rather than detecting it, leaving the assertion with little advantage over using a $display statement.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog

What Is a Concurrent Assertion?



Concurrent assertion is a check that evaluate behavior over time, based on a clock. They are only evaluated when a clock event occurs, not immediately when the code is executed.

- Used to specify functional *properties* of a design
  - A property is a general behavioral attribute used to characterize a design
    - What it should or should not do…
- Are standalone rules that describe design behavior
- They are not procedural code
- Evaluated as separate threads in simulation
- Formal typically evaluates each assertion in isolation
- These are what are used in practice and will be the focus for the remainder of this course

 © Cadence Design Systems, Inc. All rights reserved.

cadence

Concurrent assertions are statements inside of hierarchical blocks, for example modules.

They describe a standalone behavior which is evaluated in parallel with all other assertions and design behavior.

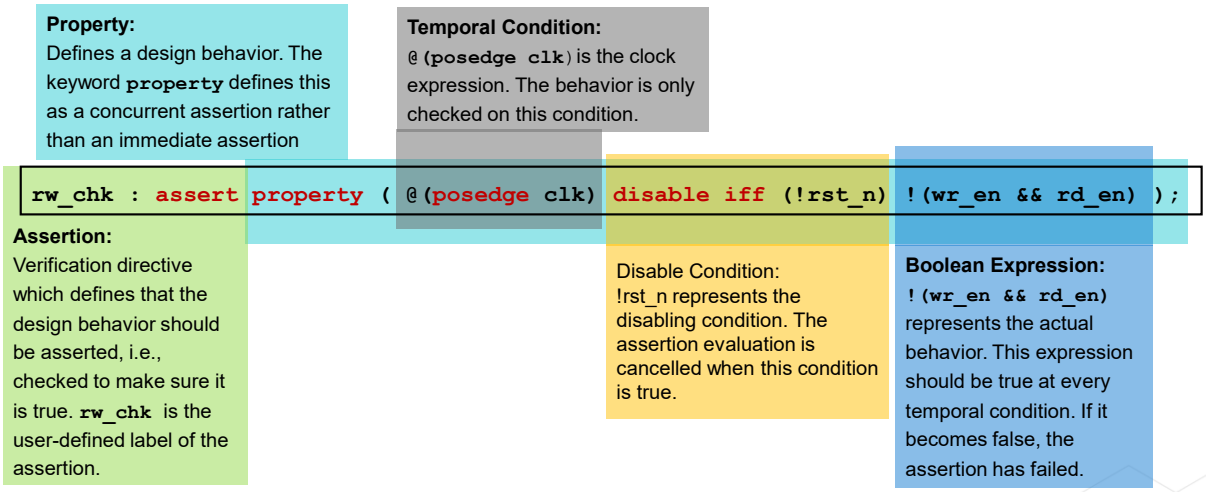
© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog

Concurrent Assertion Structure

Design Behavior

- Signals `wr_en` and `rd_en` are never both high at the positive edge of `clk`



 © Cadence Design Systems, Inc. All rights reserved.

cadence

The temporal condition defines when the behaviour is actually checked. In this example, it is not important what wr_en and rd_en are doing except at the negedge of some clock signal named clk.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog

Creating a Simple Property



Defining behavior specifies what the design must accomplish, while naming clarifies elements, enabling easier reuse and debugging. Asserting properties directs the verification process, and clocking determines when these checks occur. The 'disable iff' condition limits the checks to only valid scenarios.



 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog

Step 1: Defining Design Behavior



Define a design behavior as a Verilog boolean expression.

Any expression allowed in an `if` condition can be used, including:

- Functions
- Out of module references
- Logical and relational operators

```
(EN1 | EN2);
```

```
!(wr_en && rd_en)
```

```
((A + B) >= 42) && (C | D);
```

Subject to normal Verilog restrictions, particularly:

- Case sensitivity
- Naming rules

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog

Step 2: Naming and Asserting the Property



Two options:

1. Label and assert the property in one statement
   This single statement is both the declaration and instantiation of the property

```
ASSERT1 : assert property (EN1 | EN2);
```

2. Name the property and assert in a separate (labeled) statement
   - Property can be used as a building block in other properties
   - Property can have arguments
   - Allows a more flexible approach

Declaration / Instantiation

```
property PROP1;
  (EN1 | EN2);
endproperty

ASSERT1 : assert property (PROP1);
```



Brackets for named property in assertion

Caution



Always label assertions or the tool will assign a meaningless name

Tip

35 © Cadence Design Systems, Inc. All rights reserved.

cadence

Assertions can be defined in 1 of 2 ways:

Single statement assertion – define the boolean condition as an un-named property and assert the property directly. The assertion statement should be labeled so that the assertion can be referenced by verification tools.

Property declaration – declare a property to define the desired behavior and assert the property in a separate statement. The property must have a name and the assertion statement can be labeled. The assertion label or property name can be used by the verification tools to identify the assertion.

The property name or assertion label must be a legal Verilog name that does not conflict with other names in the Verilog namespace.

The second, 2-statement form is more flexible for complex assertions. Separately declaring and naming properties allows the properties to be used as building blocks in more complex assertions, or to be separately used for dynamic assertions, functional coverage or formal verification.

In this course, you use the 2-statement form throughout the course and assume the property is separately asserted if not explicitly shown.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog

Step 3: Clocking the Property



Properties must be clocked:

- Does not necessarily have to be in the property's declaration

Evaluation of clocked properties controlled by clock edge expression:

- Attached to property via clock operator `@`
- Same syntax as procedural sensitivity lists in RTL Verilog
- Any variable or combination of variables can be used for clock...

```
property PCLK1;
  @(posedge CLK) (EN1 | EN2);
endproperty
```

```
property PCLK2;
  @(EN1 or EN2) (EN1 | EN2);
endproperty
```

```
property PCLK3;
  @(negedge ADDR_EN) ADDR <= 7;
endproperty
```

36 © Cadence Design Systems, Inc. All rights reserved.

SystemVerilog allows any boolean expression to be used for a clock operator, allowing a property to be evaluated on a complex boolean condition.

Typically, RTL synthesis-style clock edge expressions are used to define clock conditions

A property must be clocked by something, but this can come from places other than the property declaration, as you will see later.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog

Step 4: Disabling the Property



To terminate an assertion when a condition is true, use `disable iff`

- If `expr` is true at any point, then the property evaluation is terminated (passes)

Evaluation of `disable iff` is asynchronous, i.e., the clock is irrelevant

Disable expression must be bracketed `()`

Useful for cancelling multicycle sequence properties

- For example, bus transactions which are interrupted by reset

```
property NAME;
  @(clocking) disable iff (expr)
   property;
endproperty
```

```
property ABORT;
  @(posedge CLK) disable iff (RST)
    EN1 | EN2 ;
endproperty
```



Disabled properties pass!

Caution

37 © Cadence Design Systems, Inc. All rights reserved.

cadence

Disable is essential for terminating long sequence properties if design conditions change, for example. In this case, if a reset occurs.

If the disable condition (RST) is true at any time, then the assertion is disabled (terminated) immediately, regardless of how the property is clocked.

You can view the disable as being asynchronous from the property sampling clock.

Note: Depending upon which tool you are using, a disabled property may be defined as a pass, just as if the property had successfully completed. They are not counted as a fail. This can possibly lead to misleading results when a simulator reports the number of design properties that have passed or failed.

This is because in Formal Verification (where assertion languages were first used), the absence of a failure is deemed as a pass.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog

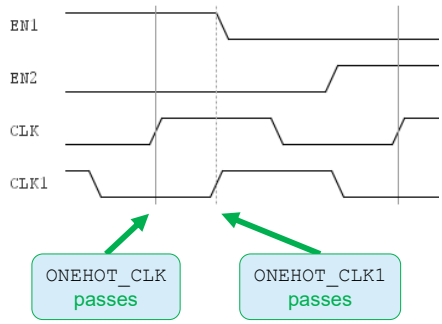
Default Clock Statement

Specifying a clock operator for every assertion is repetitive, making assertion writing and maintenance unnecessarily difficult.

Solution: Use a default clock block

- Applies to every assertion in the same scope without a specific clock
- Only 1 default clock per scope allowed
- SVA allows for definition of global clocks, but these are almost never used

TB: SVA clocking explained




```
default clocking DEFCLK @(posedge CLK);
endclocking

property ONEHOT_CLK;
  (EN1 | EN2);
endproperty

property ONEHOT_CLK1;
  @(posedge CLK1) (EN1 | EN2);
endproperty
```

38 © Cadence Design Systems, Inc. All rights reserved.

Clocking blocks are a new feature of SystemVerilog, aimed at defining timing and synchronization for testbenches. They are covered in more detail in the SystemVerilog Verification module.

A clock for a SystemVerilog assertion can be declared or inferred in many different ways. You will explore these in greater detail in the advanced chapters of this course.

© Cadence Design Systems, Inc. All rights reserved.

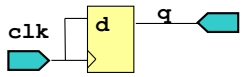
This document is for the sole use of Rajat G of Analog

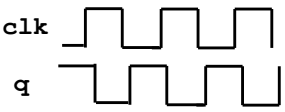
What Is Counterintuitive Clocking Behavior?

Counterintuitive clock behavior refers to situations where a property can exhibit unexpected behavior when the same signal is used both in the clocking event and within the property.

Despite the LRM clearly defines sampling and update regions, the resulting behavior can feel counterintuitive.

Caution This property fails, always!






```
P5: assert property (@(posedge clk) clk );
```

- At @(posedge clk) you might assume clk is 1, but signal values are sampled before the clock edge.
- Using the same signal for both clocking and the property expression can lead to incorrect behavior and should be avoided.

 © Cadence Design Systems, Inc. All rights reserved.

cadence

Yes, @(posedge clk) you may expect that clk has the value ‘1’.

However, the properties shown always fail in both formal tools and simulators.

LRM defines where values of signals are sampled and updated.

Think as if you were considering the clock and D inputs of a D-type flip-flop and then you will be OK.

If you connect the clock signal to both D input and the clock input of a FF, then the OP will be the inverse of the clock as the value at D is “sampled” before the clock changes.

If you use the same signal in both the property and clocking expression, then maybe you need to consider if you are doing the correct thing.

<<clocking, counter intuitive>>

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog

Why Counterintuitive Clock Behavior Occurs?

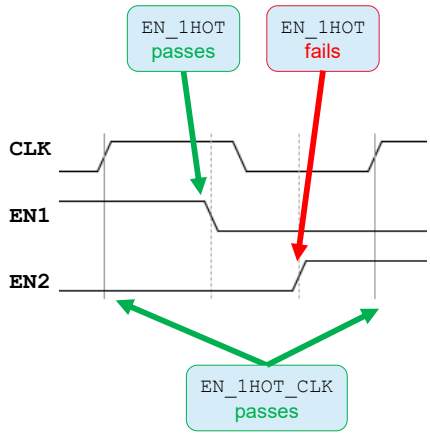
Objects sampled *before* clock edge

`EN_1HOT_CLK` evaluated on positive edge of `CLK`

- 2 evaluations, 2 passes

`EN_1HOT` evaluated when `EN1` or `EN2` change value

- 2 evaluations, 1 failure




```
property EN_1HOT;
  @(EN1 or EN2) (EN1 | EN2);
endproperty

property EN_1HOT_CLK;
  @(posedge CLK) (EN1 | EN2);
endproperty
```

40 © Cadence Design Systems, Inc. All rights reserved.

Objects in a property are sampled before the edge on the clocking variable. This is covered in more detail in the Advanced SVA Features module.

The most obvious effect of this is that the following is always true:

```
@(posedge CLK) !CLK;
```

However, in general, it is best to avoid using the same signal in the clocking and boolean parts of a property.

Asynchronous properties should be clocked on changes of any signals in the property expression.

Synchronous properties are clocked on the rising or falling edge of a design clock or other signal.

Notice that the failure of EN_1HOT occurred sometime after that “bad” behavior started. Indeed if you end the simulation before EN2 changes from 0 >1, then you would never know that there was a problem.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog

Consequences of Counterintuitive Clock Behavior

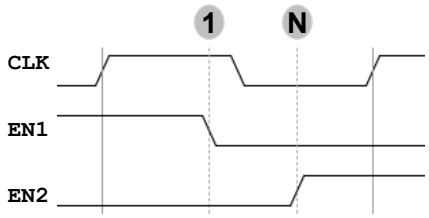
- At **cycle 1**, the property evaluates because **EN1** changed from 1→0.
  - The **preponed (sampled-before-edge)** value of EN1 is **1**, so the property **passes**.
- At **cycle N**, the property evaluates because **EN2** changed from 0→1.
  - The **preponed** value of EN2 is **0**, so the property **fails**.
- This creates multiple issues:
  - You only discover the failure at **cycle N**, long after the bad behavior began.
  - **Cycle N could be extremely far** from the original violation.
  - **Cycle N may never occur** (EN1 or EN2 might never toggle again).
  - As a result, the bug can be **completely missed**.
- **Rule of thumb:** Do **not** use the same signal as your property clock *and* inside the property expression. Otherwise, many subtle differences can hide bugs.

```
property EN_1HOT;
  @(EN1 or EN2) (EN1 | EN2);
endproperty
```



Don't write a property like this

Caution



© Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog

Solution: Using Asynchronous Property Clock

If we want to evaluate “Asynchronous” Properties:

- Use an appropriate property clock.
- You can use an existing design clock, or
- You can create your own clock
  - A new signal specifically for clocking the assertion

```
logic MY_PROP_CLK = 0;

always @(EN1 or EN2)
  MY_PROP_CLK <= !MY_PROP_CLK
```

```
property EN_1HOT_CLKD;
  @(MY_PROP_CLK) (EN1 | EN2);
endproperty
```

Now we see the assertion fail at the exact time/cycle the “bad behavior” is observed

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog

Choosing Clock Edges for Assertions



If your design uses `@(posedge clock)` then all of your properties should as well

- For simulation, this removes the possibility that a Gate-Level Simulation (GLS) assertion fails due to delay annotations
  - For example, if sampling on the `negedge clock` then any signal delayed by more than ½ cycle could cause a false negative
- For formal it means that the tool can ignore all activity on the `negedge` clock
  - This significantly reduces proof times

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog

Assertion Placement



- Placed in a hierarchical block
  - Normally, a module or interface
- Usually placed as a concurrent statement outside of a procedure
  - Can be placed inside procedural code
    - Possible issues – see link
- Will cause compilation errors with non-SystemVerilog tools
  - Disable with `ifdef` conditional compilation
  - Allows assertions to be added to existing Verilog models

```
module mod1
  (input logic CLK,
   ...
   output logic EN1, EN2);

`define SVA_ENABLE

`ifdef SVA_ENABLE
  property PROP1;
    @(posedge CLK) (EN1 | EN2);
  endproperty

  ASSERT1: assert property (PROP1);
`endif
...
```



TB: What is the Difference Between a Concurrent SVA Property in Procedural Code and an Immediate Assertion

© Cadence Design Systems, Inc. All rights reserved.

cadence

SystemVerilog allows concurrent assertions to be placed in any hierarchical block, i.e., Module, Interface, Program block, Package, Clocking block or Compilation unit scope.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog

Implication Construct



```
expr1 |-> expr2;
expr1 |=> expr2;
```

What does Implication mean?

- These are conditional operators.
- If expression `expr1` completes, then expression `expr2` must occur.
- If enabling condition starts but does not complete, property does not fail.

Two types of implication operators:

- Same cycle implication |->
  - If expr1 completes, assertion looks for expr2 to begin in the same cycle
  - Known as overlapping sequence implication
- Next cycle implication |=>
  - If expr1 completes, assertion looks for expr2 to begin in the next cycle
  - Known as non-overlapping sequence implication

 © Cadence Design Systems, Inc. All rights reserved.

cadence

The implication operators for simple boolean assertions can also be used for sequence assertions.

You can think of both operators as defining when the consequence must start relative to the completion of the enabling expression.

It's also important to understand the conditional nature of implication.

If expr1 never completes, the assertion does not fail, as the condition to trigger expr2 was never satisfied. However, if expr1 does complete, then expr2 becomes mandatory. Failing to observe expr2 after expr1 completes will cause the assertion to fail.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog

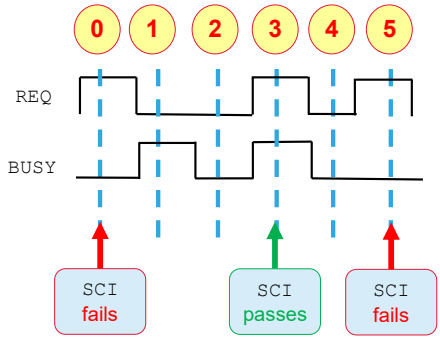
Same Cycle Implication |->

Assertion checks may only be valid under certain conditions

```
expr1 |-> expr2;
```

Use logical if implication operator |->

- **If** `expr1` is true, **then** `expr2` must be true at the *same* evaluation point




```
property SCI;
  @(posedge CLK) REQ |-> BUSY;
endproperty
```

 © Cadence Design Systems, Inc. All rights reserved.

cadence

At each evaluation point (rising edge of CLK in this example) the if condition is checked (REQ = logic 1). If the condition is false, the assertion is ignored for this clock cycle. If the condition is true, then the then condition is checked (BUSY = logic 1) in the same evaluation cycle. If the then condition is true, then the assertion passes. If the then condition is false, the assertion fails.

Both the if and then conditions must be expressions that return a boolean value.

The if condition is also known as the enabling condition, and the then as the fulfilling condition.

This implication operator is called the overlapping implication operator.

© Cadence Design Systems, Inc. All rights reserved.

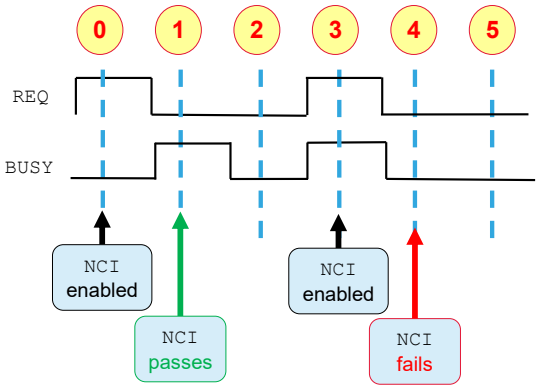
This document is for the sole use of Rajat G of Analog

Next Cycle Implication |=>

Use |=> operator to check a *then* condition in the cycle after an *if* condition

If `expr1` is true, **then** `expr2` must be true at the *next* evaluation point

```
expr1 |=> expr2;
```




```
property NCI;
  @(posedge CLK) REQ |=> BUSY;
endproperty
```

47 © Cadence Design Systems, Inc. All rights reserved.

cadence

At each evaluation point (rising edge of CLK in these examples) the first condition is checked (REQ = logic 1). If the condition is false, the assertion is ignored for this clock cycle. If the first condition is true, then the second condition is checked on at the next evaluation point (BUSY = logic 1 at the next rising edge of CLK). If the second condition is true, then the assertion passes. If the second condition is false, the assertion fails.

The first condition is also known as the enabling condition. The second condition is known as the fulfilling condition. If the enabling condition is true, then the fulfilling condition must be true for the property to pass.

This implication operator is called the nonoverlapping implication operator.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog

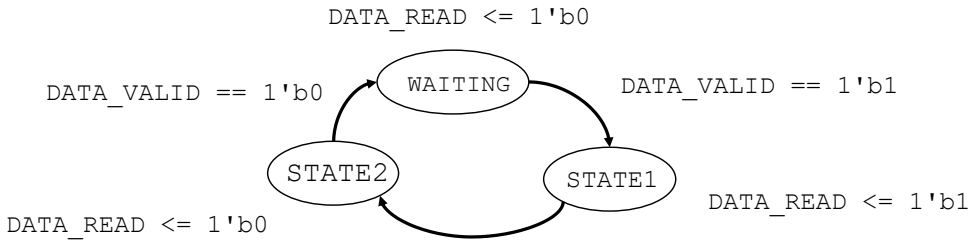
Simple FSM Example Using Implication Construct

Simple FSM

- `DATA_VALID` is an input
- `DATA_READ` is an output

To write SystemVerilog assertions for verifying FSM:

- Use `|=>` to verify state transitions from current state and inputs
- Use `|->` to verify outputs from current state



48 © Cadence Design Systems, Inc. All rights reserved.

cadence

The state diagram for this FSM defines the following behavior:

The states of the FSM are labelled WAITING, STATE1 and STATE2.

The FSM input, DATA_VALID is used to determine when to transition states.

The FSM output, DATA_READ, is assigned according to the current state.

If the current state is WAITING, then the output is 0 and we transition to STATE1 if the input is 1 on the active clock edge.

If the current state is STATE1, then the output is 1 and we transition unconditionally to STATE2 on the next active clock edge.

If the current state is STATE2, then the output is 0 and we transition to WAITING if the input is 0 on the active clock edge.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog

FSM Properties Using Implication Construct

Properties must be asserted to be checked!

```
property FSM1;
  @(posedge CLK) ((STATE == WAITING) && DATA_VALID) |=> (STATE == STATE1);
endproperty

property FSM2;
  @(posedge CLK) (STATE == STATE1) |=> (STATE == STATE2);
endproperty

property FSM3;
  @(posedge CLK) ((STATE == STATE2) && ~DATA_VALID) |=> (STATE == WAITING);
endproperty

property OP_ACTIVE;
  @(posedge CLK) (STATE == STATE1) |-> DATA_READ;
endproperty

property OP_INACTIVE;
  @(posedge CLK) ((STATE == STATE2) | (STATE == WAITING)) |-> ~DATA_READ;
endproperty

AST_FSM_CHK1 : assert property (FSM1);
// ....and so on
```

49 © Cadence Design Systems, Inc. All rights reserved.

cadence

Properties FSM1 to FSM3 check the state transitions. These properties only check that the FSM changes state on the correct input values. They do not check that the FSM does not change state on any other input value. For example, in FSM1, if the current state is WAITING and DATA_VALID is false, then the enabling condition of the property does not complete, and the property does not become active. However, if the state incorrectly changes to STATE1, this is not detected as an error. Further properties may be required to check for incorrect transitions, e.g.,

```
property FSM1A ;
  @(posedge CLK) ((STATE == WAITING) && !DATA_VALID) |=>
    (STATE == WAITING);
endproperty
```

The other properties check that the output is active only in the correct state. These properties will detect incorrect outputs due to errors in the output decoding, but will not detect incorrect outputs from the FSM as a result of being in the wrong state.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog

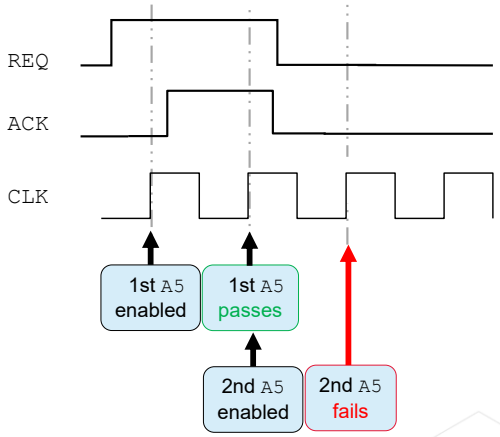
Assertion Overlapping

Assertions are evaluated at every check point, and whenever the enabling condition is true, a new assertion is triggered, allowing many simultaneous copies of an assertion.

REQ true for two cycles triggers two assertions

- More on overlapping later...

```
property A5;
 @(posedge CLK) REQ |=> ACK;
endproperty
```



Caution

Assertions must be carefully written to avoid unexpected failures from overlapping

50 © Cadence Design Systems, Inc. All rights reserved.

The enabling condition of every assertion is checked on every evaluation cycle. If the enabling condition is true, then one copy of the assertion becomes active at the evaluation cycle. If the enabling condition is also true at the next or future evaluation cycle, then another copy of the assertion becomes active. If unintentional, this can lead to confusion and possible false failures as multiple, overlapping copies of a single assertion are active.

Assertions defined with level-sensitive enabling conditions are a common cause of overlapping. In the example above, as REQ is true for 2 evaluation cycles, the enabling condition (REQ) triggers 2 copies of the assertion. For this example, a single ACK fulfils one of the active assertions, but the other fails.

Overlapping is a particular issue for assertions defined using sequences. The next chapter provides more detail.

The above assertion could be made edge-sensitive by using a sequence or a built-in function like $rose.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog

Solution: Prevent Overlap with `$rose` and `$fell`

Built-in functions for detecting value change

Useful for making properties edge-triggered

- One way to avoid overlapping

`$rose(expr)`

- True if `expr` has changed value and is 1 in the current cycle

`$fell(expr)`

- True if `expr` has changed value and is 0 in the current cycle

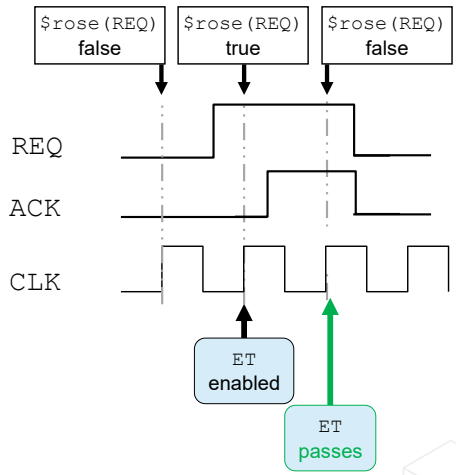


Caution

`$rose()` is also true for X->1 or Z->1 changes

Additional built-in functions useful for different Boolean checks will be covered in the following slides

```
property ET;
 @(posedge CLK) $rose(REQ) |=> ACK;
endproperty
```



51 © Cadence Design Systems, Inc. All rights reserved.

Using $rose or $fell to make the enabling condition of a property edge-triggered rather than level-sensitive is one easy way to avoid overlapping assertions.

This course provides other methods later.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog

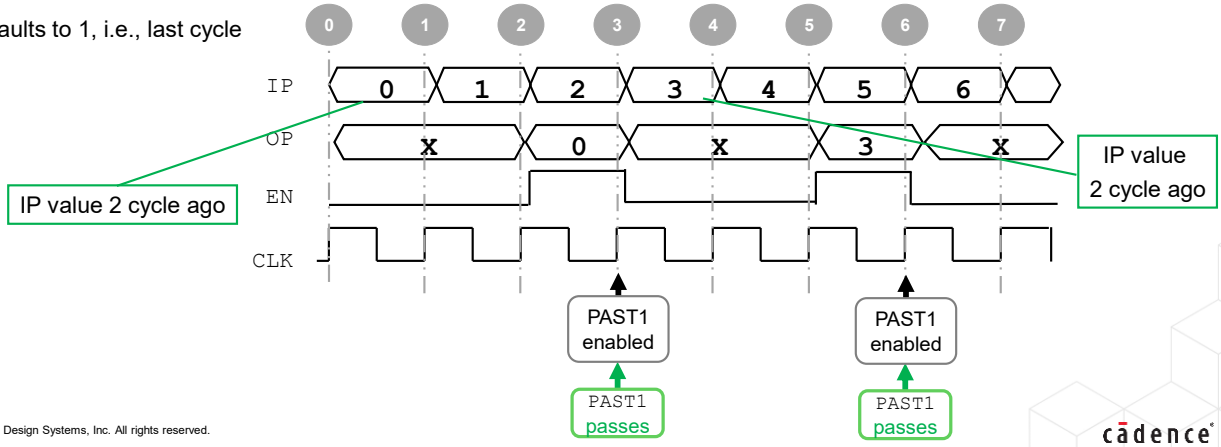
$past() Function

```
$past(A, N);
```

```
property PAST1;
  @(posedge CLK)
  EN |-> (OP == $past(IP,2));
endproperty
```

Returns the value of A from a previous evaluation cycle

- N defaults to 1, i.e., last cycle



52 © Cadence Design Systems, Inc. All rights reserved.

Past has 1 mandatory parameter, the expression whose past value one desires, and 3 optional ones.

Only one of the optional parameters is shown.

Note that the mandatory parameter can be any expression, for example (C && D) || E

See the IEEE 1800-2009 for details.

© Cadence Design Systems, Inc. All rights reserved.

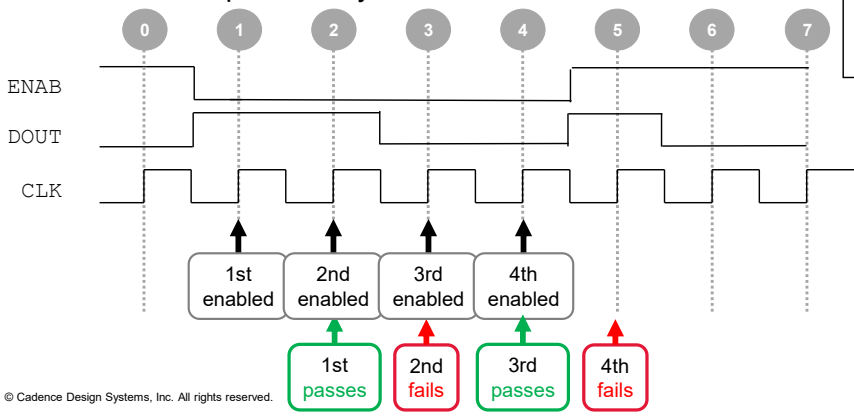
This document is for the sole use of Rajat G of Analog

$stable() Function

```
$stable(expr1);
```

Returns a boolean `true` if `expr1` has the same value as it did in the previous cycle

```
property STABLE1;
  @(posedge CLK)
  ~ENAB |=> $stable(DOUT);
endproperty
```



53 © Cadence Design Systems, Inc. All rights reserved.

The same behavior can be described with the built-in $past() function.

```
property SAME;
   @(negedge CLK)
   ( ~ENAB |=> (DOUT == $past(DOUT))  );
endproperty
```

In contrast to $past(), stable does not have an argument to specify the number of cycles over which the object has been stable.

IEE2009 adds a function $changed(expr) which returns true if expr changed from the previous cycle.

This is the logical inverse of $stable(expr).

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog

$countones() and $isunknown() Functions

$countones(expr1)

- Returns an integer representing the number of 1's in `expr1`
- Any value other than 1 ignored

```
property CHK_ONEHOT;
  @(posedge CLK)
  ($countones(GRANT_VEC) == 1 );
endproperty
```

Caution

Function can be used as an integer for example

$isunknown(expr1)

- Returns a boolean true if any bit in expr1 is x or z

```
property CHK_UNKNOWN;
  @(posedge CLK)
  (~EN |-> $isunknown(TS_BUS));
endproperty
```

Check that if `~EN`, then some bits of `TS_BUS` are driven to `X` or `Z`

54 © Cadence Design Systems, Inc. All rights reserved.

The argument of either function is any reg, wire or net type or any word in a memory.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog

$onehot() and $onehot0() Functions

$onehot(expr1)

- $onehot returns a boolean true only if exactly one bit in expr1 is 1.

```
property OH;
  @(posedge CLK)($onehot(GRANT_VEC));
endproperty
```

$onehot0(expr1)

- $onehot0 returns a boolean true only if, at most, one bit in expr2 is 1.

```
property OH_ZERO;
  @(posedge CLK)($onehot0(GRANT_VEC));
endproperty
```

GRANT_VEC	OH	OH_ZERO
"XX1Z"	Pass	Pass
"0000"	Fail	Pass
"0100"	Pass	Pass
"0Z00"	Fail	Pass
"1111"	Fail	Fail

55 © Cadence Design Systems, Inc. All rights reserved.

The argument is any reg, wire or net type or any word in a memory.

If you wanted to check for a “real” one hot, i.e., GRANT_VEC having the following set of values, 0001, 0010, 0100, 1000.

Then you can combine onehot and isunknown like so:

```
property COMB;
  @(posedge clk)
  $onehot(GRANT_VEC) && (!$isunknown(GRANT_VEC)) ;
endproperty
```

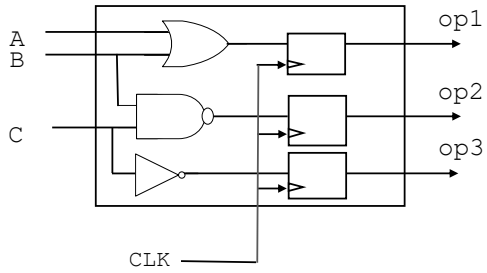
© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog

Quiz

Write assertion(s) to check the behavior of the circuit:

- Don't be concerned about reset for this example
- Use the posedge of `CLK` to clock the properties



56 © Cadence Design Systems, Inc. All rights reserved.

cadence

Hopefully, this quiz gets you in the mindset of writing assertions.

Don't worry if it looks harder than you thought; this is the feeling most people have initially.

It's recommended to try and forget about procedural code like Verilog, C/C++, VHDL,

Knowing when you have written enough assertions to cover all desired behavior is a very difficult task.

In fact, it is so hard that tools cannot do this for you automatically.

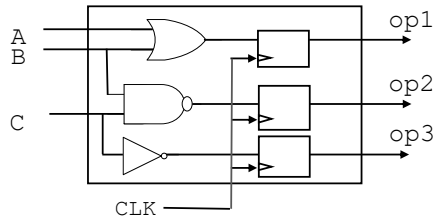
This is called the completeness problem and is applicable to verification in general, not just using properties or formal verification. This course provides more detail later.

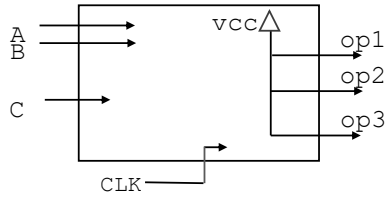
Often with assertions to specify completely the behavior which you want to be checked, you may need to check the output value in the context of previous inputs and the next output value in the context of the current inputs.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog

Quiz Solution






```
P1 : assert property ( @(posedge CLK) A || B |=> op1 );
P2 : assert property ( @(posedge CLK) !(B && C) |=> op2 );
P3 : assert property ( @(posedge CLK) !C |=> op3 );
```

These assertions check some of the desired behavior

However, `op1`, `op2` and `op3` might be stuck at logic 1 every cycle!

You would need coverage to check that `op1` can be zero

- Assuming that you care about the value of `op1` if `A || B` was false on the previous cycle

```
P1_B : assert property ( @(posedge CLK) op1 |-> $past( A || B) );
```

```
P1_B : assert property ( @(posedge CLK) op1 == $past( A || B) );
```

57 © Cadence Design Systems, Inc. All rights reserved.

Given what we know so far, most people would come up with properties like P1, P2 and P3 for this quiz.

However, these are not correct. If we tied op1, op2 and op3 to the power rail, so the outputs are always true, these assertions will not fail. So clearly we didn't capture the properties correctly.

Let's try the other way around and have the output on the left hand side of the implication. So op1 true implies that on the previous clock, A or B must be true. This suffers from the same problem. If we hold all the outputs to the ground, so they are false, then they never get checked.

The only solution, which is not intuitive at all, is to write a Boolean property where op1 equals the previous value of (A or B).

This quiz also illustrates the importance of coverage. Coverage should check op1 has the values 1 and 0. So if our first set of properties all passed, but the outputs were always stuck at 1, then we know we have a problem.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog

Summary

SystemVerilog assertions are based on Verilog boolean conditions

Constructing a simple SVA assertion is straightforward

- Write a Boolean condition using Verilog syntax
- Define when the condition is checked with a default or explicit clock
- Define when the condition should not be checked through a `disable iff`
- Name the condition using the `property` statement
- Assert the property using an `assert` statement

Built-in functions can help

- Avoiding overlapping
- Common checks

Conditional checks are the first step toward more complex assertions

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog

Lab



Lab 1 Simple Assertions to Verify FSM

- In this lab, you will write simple Boolean assertions to an FSM design and analyze the difference between same-cycle and next-cycle implication.

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog



Module 4

Sequences

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog

Module Objectives

After this module, you will be able to

- Explain the fundamentals of Sequences and show Sequence-Based Assertions
- Describe Never Properties and Disable Resolution for controlling assertion
- Apply Cycle Delay Repetition operators to specify timed behavior in sequences
- Utilize Consecutive Repetition operators to model repeated sequential events
- Apply Goto and Non-Consecutive Repetition operators to express flexible sequence behavior

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog

What Are Sequences?

Sequences are building blocks of properties, consisting of a series of boolean expressions separated by cycle delays (##N).

Normally, sequences are defined as unclocked

- They inherit their parent's clock, for example, when used in an assertion

Wide range of powerful operators and features to construct complex sequences

- if-then implication
- Replication
- Named sequences
- Composition operators

```
sequence MYSEQ;
 A ##1 B ##1 C;
endsequence
```

```
AST_SEQ1: assert property
  (@(posedge CLK) START |=> MYSEQ;
```

MYSEQ inherits the clock from the assertion AST_SEQ1

62 © Cadence Design Systems, Inc. All rights reserved.

cadence

Simple boolean assertions are instantaneous. To define complex design properties, you need to define temporal properties where boolean equations are evaluated over successive cycles. Sequences let us do this.

is the cycle delay operator. It joins separate boolean equations into a sequence where the equations are examined on successive evaluation cycles.

© Cadence Design Systems, Inc. All rights reserved.

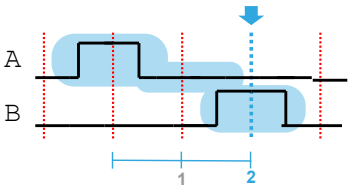
This document is for the sole use of Rajat G of Analog

Sequence Examples

- Sometimes it is necessary to capture a sequence of events in properties
- Most sequences are described using the `##` operator

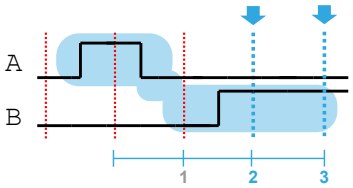
`(A ##2 B)`

`A` happens then exactly 2 cycles later `B` happens



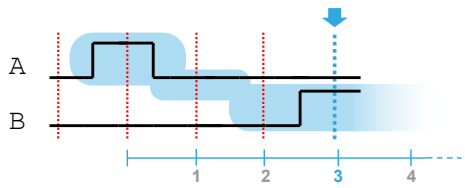
`(A ##[1:3] B)`

`A` happens then 1 to 3 cycles later `B` happens



`(A ##[3:$] B)`

`A` happens then 3 or more cycles later `B` happens



- It is important to remember that a sequence of length 1 cycle is a Boolean

63 © Cadence Design Systems, Inc. All rights reserved.

Just to reiterate at this point, remember that all we're doing when defining sequences, is Booleans separated by a ## and a number.

Cycle delays can also have ranges. The cycle delay number does not need to be ##1, it can be any number, including zero. If we want to say A occurs then exactly 2 cycles later, B occurs, we say A ##2 B. On the slide, the red dotted vertical line represents the property clock evaluation point, whatever that may be. We can also have a range. The second example is a range of between one and three cycles. This sequence will succeed if we have A followed by 1, 2 or 3 cycles, followed by B. So there are 3 solutions which match that sequence. So if we've got A occurring, then B could occur at cycle 1, 2 or 3. The example sequence succeeds, in this case, because B goes high at cycle 2. The upper bound can be infinite, indicated by a dollar sign ($). The third example has a lower bound of 3 cycles, so the earliest we look for B is cycle 3, but an upper bound of $ means B could stay low forever. So a sequence of length one cycle is the same thing, exactly, as a Boolean.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog

Using Sequences in Properties: Conditional Vs. Unconditional



Properties are either:

- Unconditional using instantaneous Boolean expressions, called invariants
- Conditional using implication operators
  - *If* enabling condition, *then* fulfilling condition

Avoid unconditional sequence properties

- Requires `(A ##1 B #1 C)` to be true at *every* evaluation point
- This means:
  - `A` must be true at every cycle
  - `B` must be true at the second and all consequent cycles
  - `C` must be true at the third and all consequent cycles

Unconditional

```
property UNCOND;
  @(posedge clk) (COUNT > 10);
endproperty
```

Conditional

```
property COND;
  @(posedge clk) A |=> C ##1 D;
endproperty
```

if — enabling sequence — then — fulfilling sequence

```
property AVOID;
  @(posedge clk) A ##1 B ##1 C;
endproperty
```

equivalent to:

```
(1'b1 |-> (A ##1 B ##1 C);
```

64 © Cadence Design Systems, Inc. All rights reserved.

cadence

SystemVerilog allows sequence properties to be defined like this:

```
@(posedge CLK)(A ##1 B ##1 C);
```

However, remember that properties are evaluated on every cycle. On the first cycle, a copy of the assertion is triggered, and A must be true. On the second cycle B must be true to keep the first assertion true, but we also trigger a second copy of the assertion, which requires A to be true in the second cycle also. In the third cycle, C must be true for the first assertion, B must be true for the second, and A true again for a new third copy of the assertion. Therefore A, B and C must be true for every evaluation point.

Properties are rarely written in this way. Condition assertions using implication operators are much more useful when using sequences.

Terminology: the enabling sequence or condition is also referred to as the Antecedent or cause. The fulfilling sequence/condition is also referred to as the Consequent or effect.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog

What Is a Never Property?



A Never Property is an SVA construct, used to specify behavior that must never occur in a design.

```
property NEVER_BOOL;
  @(posedge clk) !(COUNT > 10);
endproperty
```

```
property NEVER_SEQ;
  @(posedge clk) not (BUSY ##1 GRANT);
endproperty
```

Often easier to specify bad behavior than every good behavior, for example:

- Count should never be greater than 10
- Should never see `BUSY` followed by `GRANT`

Use the `not` keyword in front of the bad behavior

Do not use with implication operators

- Only with booleans or sequences
- Although it would be syntactically correct



Caution

```
property AVOID;
  @(posedge clk) not (A |=> B) ;
endproperty
```



Not intuitive what this means

A	B	AVOID Result	not AVOID Result
0	0	Pass	Fail
0	1	Pass	Fail
1	0	Fail	Pass
1	1	Pass	Fail

65 © Cadence Design Systems, Inc. All rights reserved.

cadence

Sometimes it is easier to describe behaviour which should never occur, rather than all the different good behaviours that can occur. We can invert the truth of a property in several ways.

If the property is just a Boolean, we can use the normal Verilog invertor operator. However, for a sequence, we can't use the operator. We need to use keyword not. So, the property NEVER_SEQ says we must never observe the sequence BUSY followed one cycle later by GRANT.

Using the not operator also inverts the guidelines for using sequences. We said previous that sequence assertions should always use an implication operator. Although syntactically it is legal to use not with an implication property, it is almost of no practical use because it's hard to understand what actually happens. So the guideline is inverted and you should avoid using not with implication operators.

To understand why, we can examine the truth table for property AVOID. The third column shows the normal behaviour of A |=> B (so AVOID without the not) and column 4 shows the inverse of column 3 (so AVOID with the not). The only condition for a pass in column 4 is A always true every cycle and B always 0 every cycle, which is not intuitive to understand and of no real practical use.

Using implication with not operators falls outside of what is known as a property simple sub-set.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog

Default Disable Function and Usage

```
default disable iff (exp);
```

IEEE 1800-2009 introduced a default disable

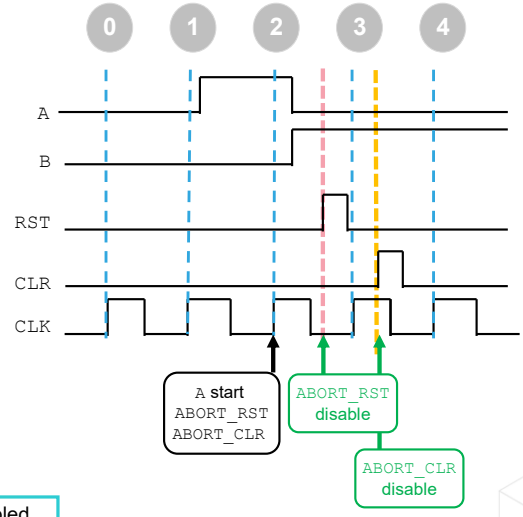
- Similar concept to default clocking

```
default disable iff (RST);
property ABORT_RST;
  @(posedge CLK)
    A |=> B ##1 C ##1 D;
endproperty
property ABORT_CLR;
  @(posedge CLK) disable iff (CLR)
    A |=> B ##1 C ##1 D;
endproperty
property NO_DISABLE;
  @(posedge CLK) disable iff (1'b0)
    A |=> B ##1 C ##1 D;
endproperty
```

default disable iff only applies to the scope in which it is defined

Explicit disable iff overrides the default

Can never be disabled. Effectively masks the default disable iff



66 © Cadence Design Systems, Inc. All rights reserved.

Just as specifying the same clocking expression for every property would be painful, especially if you wanted to change that clock, then specifying the same disable condition is also.

The behavior is very similar to that of default clocking. A default disable clause cannot have a name, unlike default clocking.

The 2009 standard introduced a default disable. You don't want to type out the same disable clause a hundred times for a hundred different properties. If the disable clause changes, you've got to change a hundred places. So default disable iff works much the same as the default clock. You specify it in a scope, inside a module for example,. The default disable only applies to properties in that scope, and there can only be one default disable per scope.

The example shows a default disable as RST being true. Any property which does not have an explicit disable iff will use the default. So property ABORT_RST will use the default. So immediately RST goes high, ABORT_RST is disabled. Property ABORT_CLR uses an explicit disable, which overrides the default. So therefore it's disabled by CLR going high. Again notice both properties are disabled asynchronously. We can also define a property which will not use a disable, even if a default disable is present. For property NO_DISABLE, an explicit disable disable iff (1'b0) will never be true, so property cannot be disabled.

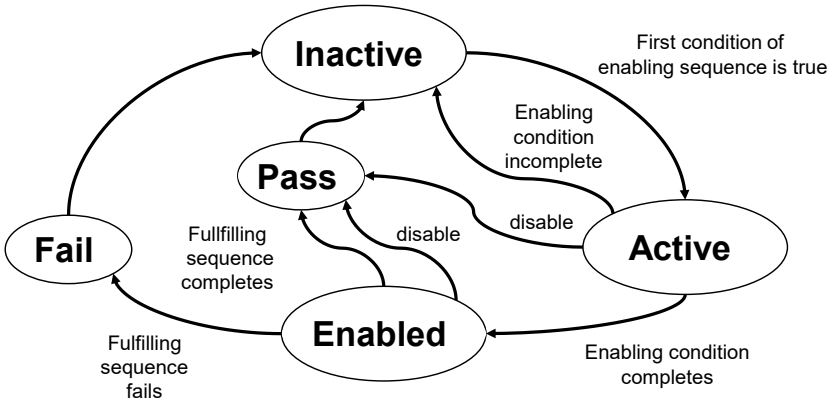
© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog

Assertion Status With `disable iff`



An assertion can be in one of several states:




```
@(clocking) disable iff (dis_expr) enable |=> fulfill;
```

67 © Cadence Design Systems, Inc. All rights reserved.

Assertions start in the inactive state. At each evaluation cycle, the enabling condition (or the first condition of an enabling sequence) is checked. If the condition is true, a copy of the assertion becomes active. In the active state, the assertion checks the enabling condition or sequence. If the enabling condition(s) fails, then the assertion terminates. If the enabling condition(s) complete, then the assertion is enabled. In the enabled state, the assertion checks the fulfilling condition(s). If the fulfilling conditions fail, then in the evaluation cycle the failure was detected, the state of the assertion is fail, after which it becomes inactive. If the fulfilling condition(s) are true, then the assertion state is pass in the evaluation cycle the final fulfilling condition becomes true. An abort can terminate an active or an enabled assertion.

Remember the enabling condition of every assertion is checked on every evaluation cycle; therefore, multiple versions of a single assertion can exist in different states. This can make management and debugging of assertions difficult.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog

Handling Asynchronous Abort Conditions



`disable iff` is asynchronous. In simulation, this will likely lead to problems with reset signals that have glitches.

Solution: Using Synchronous Abort Operators.

For synchronous abort, we choose whether property should pass or fail when abort condition occurs.

`sync_accept_on:`

```
ABORT_PASS_ON_CLR: assert property @(posedge CLK)
          A |=> sync_accept_on(CLR) (B ##1 C ##1 D);
```

- For example, if we observe `A ##1 B && CLR ##1 !C` then the property is deemed to have **passed** on the cycle where `CLR` occurred

`sync_reject_on:`

```
ABORT_FAIL_ON_CLR: assert property @(posedge CLK)
          A |=> sync_reject_on(CLR) (B ##1 C ##1 D);
```

- For example, if we observe A ##1 B ##1 C && CLR ##1 D then the property is deemed to have failed on the cycle where CLR occurred

There are also asynchronous forms of these, `accept_on` and `reject_on`

 © Cadence Design Systems, Inc. All rights reserved.

cadence

The disable iff is asynchronous, therefore there can be problems in simulation with glitches and delta cycles which disable your properties, so you'll never see a failure. Which is dangerous. Also the interpretation of a disabled property is ambiguous. Should it be a pass or a failure?

SVA2009 introduced alternative synchronous property aborts with the ability to select if the property passes or fails when it is aborted..

sync_accept_on takes an abort expression in brackets and is applied to a bracketed sequence expression. So the assertion ABORT_PASS_ON_CLR has A as an enabling expression, and B, followed by C followed by D as the fulfilling sequence. However, if during the evaluation of sequence BCD, CLR is true at an evaluation point (posedge CLK), the assertion is aborted with a pass. So sync_accept_on is synchronous and passes on abort.

There is also sync_reject_on. Assertion ABORT_FAIL_ON_CLR says if I have A, then from the next cycle I must have sequence BCD. But if CLR is true at an evaluation point (posedge CLK), during the evaluation of BCD, the assertion is aborted with a failure. So we are not expecting CLR to occur during BCD. sync_reject_on is synchronous and fails on abort.

There are also asynchronous forms of accept and reject, which just omit the sync_.

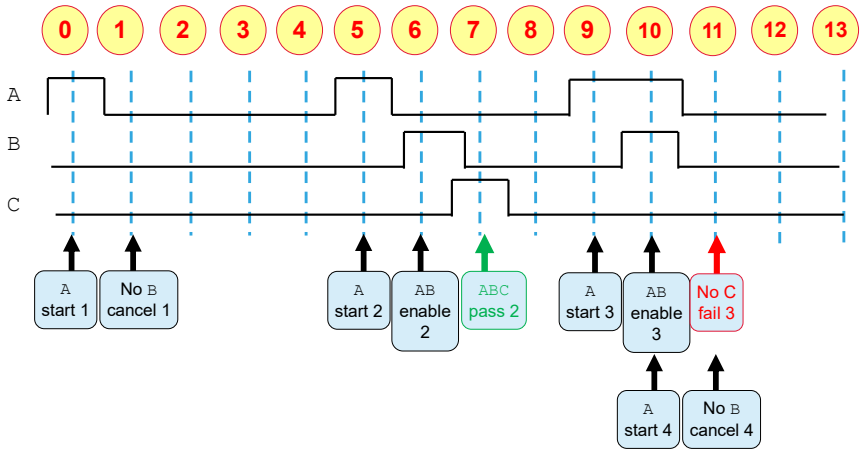
The abort operators can be used anywhere in a property enabling or fulfilling sequence. For example, you can have multiple sync_accept_on terms in a single property, affecting different bracketed individual Boolean or sequence terms. You can have as many as you like in a property. Clearly that gets difficult to understand, following the associativity and precedence rules. But you can do it if you wish.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog

Analysis of Overlapping Sequence Property Evaluations

```
property SQA;
  @(posedge CLK) (A ##1 B) |=> C;
endproperty
```



69 © Cadence Design Systems, Inc. All rights reserved.

Remember that the enabling sequence on the left side of the implication operator must complete before the fulfilling sequence on the right side is checked.

Remember also that assertions can overlap.

At cycle 0, A is true so copy 1 of SQA becomes active. At cycle 1, the second condition of the enabling sequence, B, is false so copy 1 of SQA terminates.

At cycle 5, A is true so copy 2 of SQA becomes active. At cycle 6, B is true and copy 2 of SQA becomes enabled. A is false at cycle 6 so another copy of SQA does not become active. At cycle 7, the fulfilling condition C is true and copy 2 of SQA passes.

At cycle 9, A is true and copy 3 of SQA becomes active. At cycle 10, B is true so copy 3 becomes enabled. A is still true at cycle 10, so copy 4 of SQA becomes active.

So, at cycle 10, there are two copies of property SQA in existence, both at different points in the property evaluation. Copy 3 is enabled from A at cycle 9 followed by B at cycle 10. Copy 4 is active from A at cycle 10.

At cycle 11, C is not true, so copy 3 fails. Also, B is not true, so copy 4 terminates.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog

$rose and $fell in Sequences

$rose and $fell can be used in sequences too

Useful for making sequences edge-triggered and avoiding overlapping

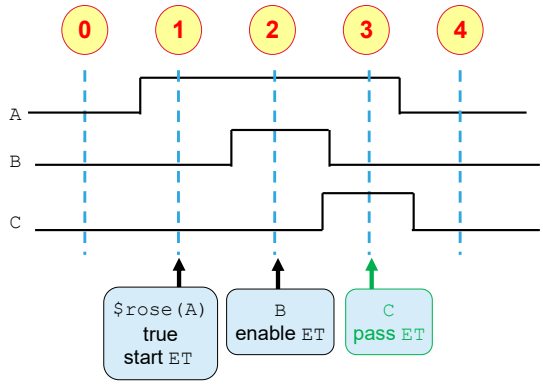
$rose(expr)

- True if expr has changed value and is 1 in the current cycle

$fell(expr)

- True if expr has changed value and is 0 in the current cycle.

```
property ET;
  @(posedge CLK) $rose(A) ##1 B |=> C;
endproperty
```



70 © Cadence Design Systems, Inc. All rights reserved.

Using $rose or $fell to make the enabling condition of a property edge-triggered rather than level-sensitive is one easy way to avoid overlapping assertions.

This course shows other methods later on.

Note $rose(A) is not the equivalent of `(!A ##1 A)`.

It is the equivalent of `(A != 1'b1 ##1 A)`.

For example, it is a transition to 1'b1 from any other value, e.g. 1'bz to 1’b1 or, more commonly, 1'bx to 1’b1. Remember 4-state SystemVerilog signals will initialize to 'bx at the start of simulation.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog

Cycle Delay Repetition

```
expr1 ##N expr2
```

```
property S4;
  @(posedge CLK) A |-> ##3 B;
endproperty
```

```
property S4_EQUIV;
  @(posedge CLK) A |=> ##2 B;
endproperty
```

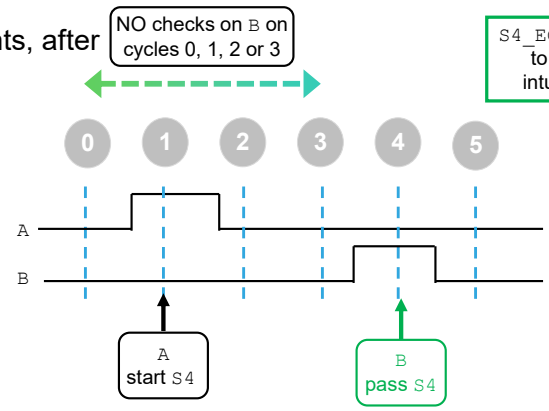
`S4_EQUIV` is exactly equivalent to `S4` but it is slightly less intuitive to count the cycles

`##` can be used with any static integer to define a number of cycle delays

After `expr1`, count N evaluation points, after which `expr2` must be true

`##0` is allowed (and useful)

Zero delay ("fusion")



71 © Cadence Design Systems, Inc. All rights reserved.

Cycle delay ## just counts evaluation cycles without checking any conditions. Any static integer value can be used with the operator. This can be very useful for defining a number of cycle delays between conditions.

Note:

##0 = current evaluation cycle

##1 = next evaluation cycle

##2 = next but 1 evaluation cycle

etc.

Zero delay is also referred to as fusion or overlapping, for example.

(A ##1 B) ##0 (C ##1 D) = (A ##1 B && C ##1 D)

Property S4 states that if A is true (at cycle 1), this implies from the same cycle we have 3 cycles followed by B (cycle 4). We also show the equivalent using next cycle implication. Since next cycle implication consumes one cycle, the cycle delay must be 2 to meet the requirement. Same cycle implication may be more intuitive as the property matches the requirement.

Note that the property does not define anything about A or B during the cycle delay. A and B can have any value. Cycle delays with values other than 0 or 1 should be used with caution. There is usually something you should be checking! For example, should A remain high until B occurs? Should B be low until the third cycle? You need to consider your requirements carefully.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog

Cycle Delay Repetition with Ranges

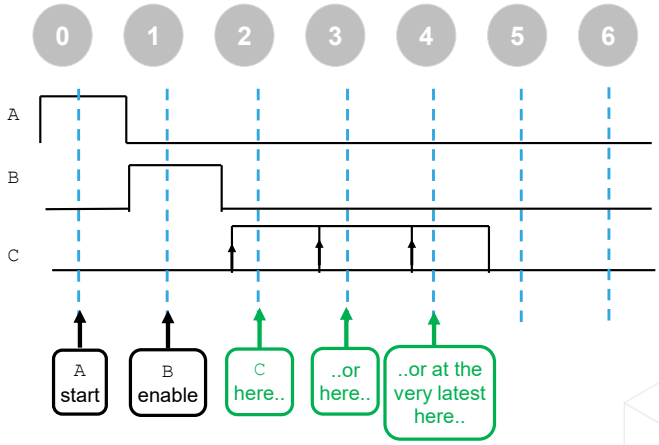
```
expr1 ##[min:max] expr2
```

`##` can also be used with a range of integers

- Range of cycle delays between conditions

After `expr1`, `expr2` must be true between `min` and `max` cycles later

```
property DEL_REP;
  @(posedge CLK) A |=> B ##[1:3] C;
endproperty
```



72 © Cadence Design Systems, Inc. All rights reserved.

A cycle delay can also be defined as a range from a min to a max number of cyclesc.

So, for the property DEL_REP: If A is true, on the next cycle we must get B and then either 1, 2 or 3 cycles later we get C. So, we have a variable delay ranging from 1 to 3 cycles. So, there are 3 possible sequences which satisfy this property:

A ##1 B ##1 C; A ##1 B ##2 C; or A ##1 B ##3 C;

So, in the waveform, C can go high on cycle 2, 3 or 4 for the property to pass. If C hasn't occurred by cycle 4, then the property fails. If C occurs on cycle 2, then the property passes and goes back to inactive. We have a conclusive result. We don't wait around to see if C happens again.

So, a cycle delay range gives a window of opportunity, from a min to max number of cycles, for something to occur (in this case C).

Note that min and max must both be static integers, i.e., defined with literals, constants or constant expressions. They cannot be variables.

Also, min must be less than or equal to max.

© Cadence Design Systems, Inc. All rights reserved.

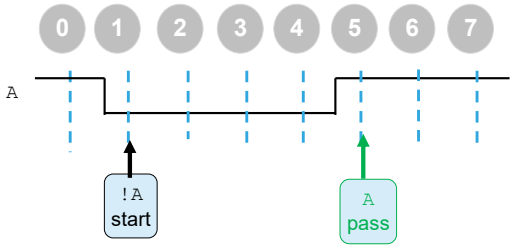
This document is for the sole use of Rajat G of Analog

Consecutive Repetition

```
SEQ_A[*N]
```

Repeated, consecutive sequences can be defined using `[*N]`

- Consecutive repetition operator
- `SEQ_A` repeated `N` times
  - `N` must be constant and known at elaboration



For example:

- `A` is never low for more than 4 cycles:

Instead of this...

```
property SEQ_COUNT;
  @(posedge CLK) !A ##1 !A ##1 !A ##1 !A |=> A;
endproperty
```

Write this...

```
property CONSEC_REP;
  @(posedge CLK) !A[*4] |=> A;
endproperty
```

73 © Cadence Design Systems, Inc. All rights reserved.

cadence

Consecutive repetition allows an individual condition or a complete sequence to repeat, consecutively, a set number of times. Consecutive repetition means one cycle after the next, as defined by the property clock.

For example, if a requirement says A cannot be low for more than 4 cycles, we could write:

(!A ##1 !A ##1 !A ##1 !A |=> A). i.e. if 4 cycles of !A must be followed by A.

However consecutive repetition defines (A[*4]) is equivalent to (A ##1 A ##1 A ##1 A), so we can simplify the property by writing:

!A[*4] |=> A

Note this works for sequences as well as Booleans, e.g.

((A ##1 B)[*2]) is equivalent to (A ##1 B ##1 A ##1 B)

The count value N must be statically computable, i.e., known at compilation time and not defined by a variable or expression. It cannot be negative.

Consecutive repetition can be used with Boolean or sequence operands.

© Cadence Design Systems, Inc. All rights reserved.

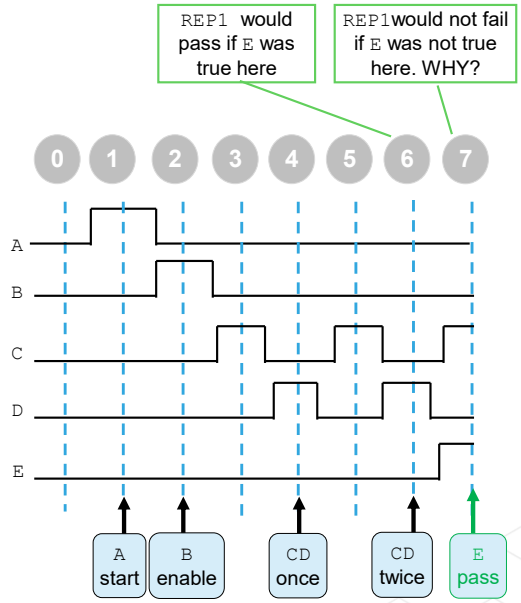
This document is for the sole use of Rajat G of Analog

Consecutive Repetition with Ranges

```
SEQ_A [*min:max]
```

- SEQ_A will repeat from min to max times, where:
  - min and max are both constant and known at elaboration
  - min <= max
- Remember that a sequence of length 1 cycle is a boolean

```
sequence SEQ_CD;
  C ##1 D;
endsequence
property REP1;
  @(posedge CLK) A ##1 B |=> SEQ_CD[*1:3] ##1 E;
endproperty
```



 © Cadence Design Systems, Inc. All rights reserved.

cadence

For a varied number of consecutive repetitions, a range can be defined. Both bounds of the range must be statically computable (like a constant expression) and must be defined in an ascending direction (min less than or equal to max). As special cases, min can be zero and max infinity (see next slide).

The example uses sequence repetition, although again, repetition ranges can be used with Booleans as well.

Sequence SEQ_CD is C ##1 D. Property REP1 says if we have A (cycle 1) followed by B (cycle 2), from the next cycle (3) we get SEQ_CD repeated between one or three times. Cycles 3 and 4 are CD once, our minimum repetition. So, if E was true on cycle 5, the property would pass, but E is not true. However, the property does not fail, because on cycles 5 and 6 we have CD again for the second repetition. On cycle 7 we have E, so the property passes there. Now if E wasn't true on cycle 7, then the property would not fail and the question is why? Well, it is because we could have a third, and maximum repetition of CD. If after the third repetition we do not have E, then the property fails. So, a property only fails when you have run out of ways of it passing. The maximum length of this property where it fails would be CD occurring three times, followed by no E.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog

Consecutive Repetition with Special Ranges

```
B ##1 A[*0:2] ##2 C
```

Lower bound can be 0

- No minimum number of repetitions
- Sequence may be absent
- Expansion:

(B ##1 A[*0] ##2 C) or (B ##1 A[*1] ##2 C) or (B ##1 A[*2] ##2 C)

↓

(B ##1 ##1 C)

↓

(B ##2 C)

This is an empty sequence as it contains A[*0] which is not required to occur.

(empty ##n seq) is equivalent to (##(n-1) seq) for n>0

Upper bound can be $ (infinity)

- No maximum number of repetitions
- Sequence may never end

A[*2:$] ##1 B

(A ##1 A ##1 B) or (A ##1 A ##1 A...)

TB: Empty Sequences in SVA Explained

75 © Cadence Design Systems, Inc. All rights reserved.
cadence

The lower bound can be 0, indicating zero occurrences of an element, meaning it is not required to appear at all.

So, the sequence (B ##1 A[*0:2] ##2 C) can be expanded to 3 possible solutions:

(B ##1 A[*0] ##2 C)

(B ##1 A[*1] ##2 C) = (B ##1 A ##2 C)

(B ##1 A[*2] ##2 C) = (B ##1 A ##1 A ##2 C)

Note the cycle delays: ##1 occurs after B, and ##2 occurs after A.

A[*0] is referred to as an empty sequence, as it includes a term that is not required to occur.

In the case (empty ##N seq) the LRM defines this as equivalent to (##(N-1) seq), so

B ##1 A[*0] ##2 C = B ##1 ##1 C = B ##2 C

For further details, click on the provided link or search for the phrase “Empty sequences in SVA”, which offers an explanation of the concept and its potential implications.

The upper bound can be infinity, $., which means the sequence may never end.

So for A[*2:$] ##1 B, the shortest matching pattern would be A ##1 A ##1 B, and the longest would be A occurring forever, i.e. B never occurs.

Consecutive ranges using $ must be used with caution. Unless additional information is supplied to apply limits to the sequence (see the Sequence Composition topic) an infinite sequence may never complete.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog

Nonconsecutive Repetition

expr1 [=N]

- Repeated, nonconsecutive sequence expressions can be defined using `[=N]`
- Nonconsecutive means **not necessarily** consecutive, `expr1` must occur `N` times

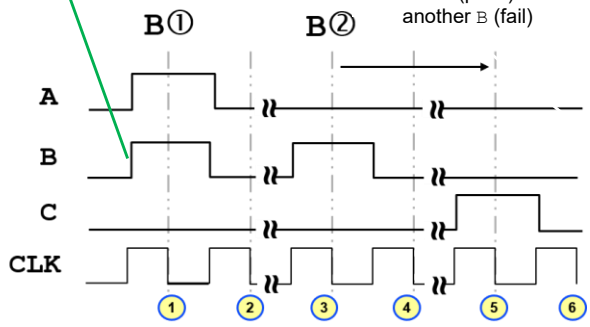
When A is high, then from the same cycle, there must be 2 cycles of B and then C

```
property NONCONSEC_REP;
  @(negedge CLK) A |-> B[=2] ##1 C;
endproperty
```

IMPORTANT : The sub-term `B[=2]` is true from the second occurrence of `B` until the **cycle before** the 3rd occurrence of `B`

`B` does not need to occur on the same cycle as `A`. We can wait any number of cycles for the first `B`.

`B[=2]` remains active until `C` (pass) or another `B` (fail)



No restriction on number of cycles between the first and second `B`
No restriction on number of cycles between second `B` and `C`

Caution

© Cadence Design Systems, Inc. All rights reserved.

A sequence defined with [=N] does not terminate after N occurrences. It remains active until the next condition in the sequence occurs. Therefore, there can be any number of cycles between the [=N] boolean condition and the following boolean.

The property NONCONSEC_REP is enabled by A. From the same cycle as A, it looks for B. From the cycle after the second occurrence of B, the property looks for C. There can be any number of cycles after A before the first B. The two occurrences of B can take any number of cycles. After the second occurrence of B, there can be any number of cycles before C.

This is a weak property, i.e., if C never occurs, the property will not fail as long as there are only 2 occurrences of B after A. Also, if there are never 2 occurrences of B after A, then again, the property will not complete.

In summary, after the second occurrence of B, the property is looking for either:

C, which would be a pass

Another occurrence of B, which would be a fail.

The property will remain enabled until either one of those conditions is found. As a weak property, neither of those two terminating conditions must occur.

Nonconsecutive repetition can only be used with Boolean operands.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog

Go-to Repetition

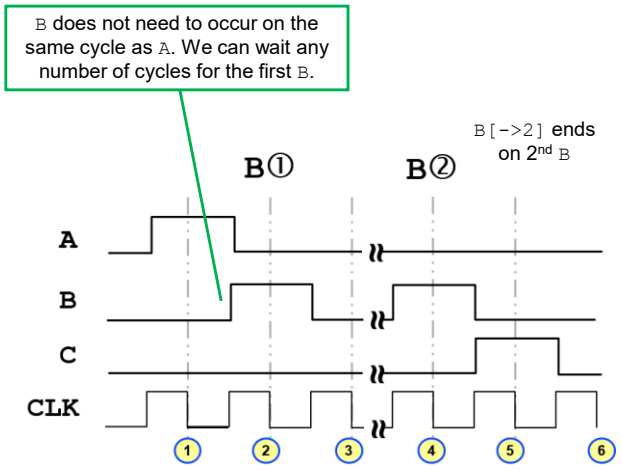
expr1[->N]

- The go-to repetition operator `[->N]` is similar to the `[=]` operator
- However, `expr1[->N]` ends on the Nth occurrence of expr1

When `A` is high, then from the same cycle, there must be 2 cycles of `B` before `C`, with `C` **immediately following** the second `B`

```
property GOTO_REP;
  @(negedge CLK)  A |-> B[->2] ##1 C;
endproperty
```

IMPORTANT : The sub-term `B[->2]` ends on the second occurrence of `B`



No restriction on number of cycles between the first and second B
Last occurrence of `B` must be followed immediately by `C`

Caution

77 © Cadence Design Systems, Inc. All rights reserved.

A sequence defined with [->N] terminates after N occurrences. The next condition in the sequence must occur immediately on the next cycle (or same cycle if using ##0).

The property GOTO_REP is enabled by A. From the same cycle as A, it looks for B. The cycle after the second occurrence of B, C must be true. There can be any number of cycles after A before the first B. The two occurrences of B can take any number of cycles. However, the go-to term B[->2] ends on the second occurrence of B, so C must occur in the cycle immediately following the second B.

This is a weak assertion, i.e., if 2 occurrences of B are never seen, the property will not fail.

Go-to repetition can only be used with Boolean operands.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog

Go-to and Nonconsecutive Ranges

Ranges can be used with both operators

- From `A`, there must be between one and 2 occurrences of `B` followed by `C`
- From `D`, there must be between 3 and 4 occurrences of `E` followed by `F`. `F` must immediately follow the 3rd or 4th occurrence of `E`

Upper bound of both operators can be `$`

- Use with care!

Lowest bound can be 0

- No minimum number of occurrences

You will see examples with waveforms on the next slide ...

```
A ##1 B[=1:2] ##1 C
```

```
D ##1 E[->3:4] ##1 F
```

```
A ##1 B[=0:1] ##1 C
```

matches `( A ##1 C)` or `(A ##1 B[=1] ##1 C)`

```
D ##1 E[->0:1] ##1 F
```

matches `( D ##1 F)` or `(D ##1 E[->1] ##1 F)`

78 © Cadence Design Systems, Inc. All rights reserved.

Go-To and nonconsecutive repetition can have ranges as well.

So from A we must have between 1 and 2 occurrences of B, which don't necessarily have to be consecutive, followed, at some future cycle, by C.

From D, we could have, from the next cycle, between 3 and 4 occurrences of E, followed immediately by F.

See the next slides for waveform examples.

The upper bounds can be infinity in both cases, and the lower bounds can be zero as well. This makes it really difficult to evaluate, so think very carefully before you use 0 or infinite range bounds.

© Cadence Design Systems, Inc. All rights reserved.

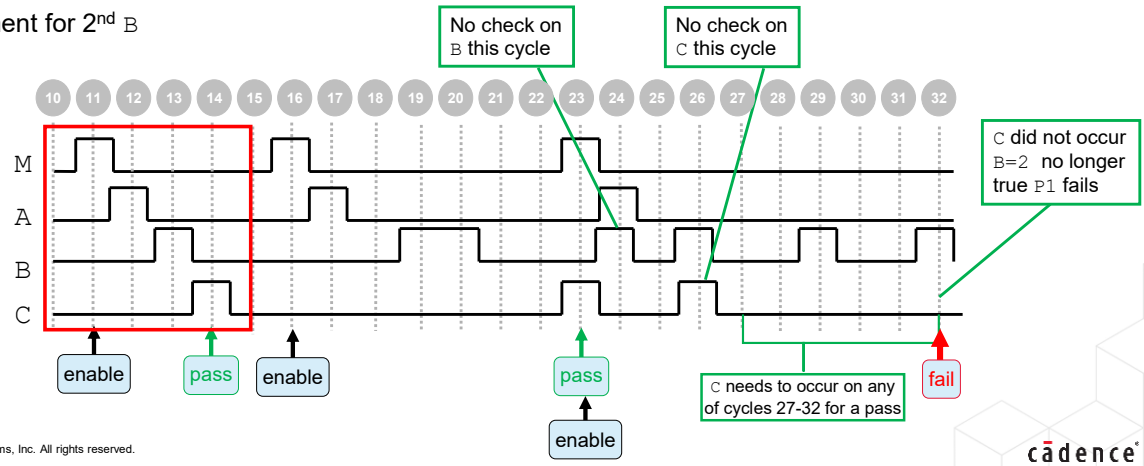
This document is for the sole use of Rajat G of Analog

Nonconsecutive Repetition Range Example

This property states:

- No limit on # cycles between A and 1st B
- No limit on # cycles between 1st and 2nd B
- No limit on # cycles between 2nd B and C
- No requirement for 2nd B

```
property P1;
   ( M |=> A ##1 B[=1:2] ##1 C);
endproperty;
```



79 © Cadence Design Systems, Inc. All rights reserved.

Cycles 11 to 14 show the quickest way in which property P1 could pass. M followed by A followed by one B (the minimum bound of the nonconsecutive repetition range), followed by C.

From cycle 16, we have M followed by A, and we can wait any number of cycles for the first B. The first B occurs at cycle 19 and the second B in the next cycle (20). Nonconsecutive doesn't mean they can't be consecutive. From the second B at cycle 20, we see no further Bs before C at cycle 23 so the property passes at cycle 23. The earliest that C could occur would be cycle 20, after the first B. If C had occurred on cycle 20 or 21 or 22, they would be passes. Once P1 passes it becomes inactive.

From cycle 23 we have M, then A. Notice the B on cycle 24 does not count, because we are checking A on that cycle. The first B is on cycle 26. Notice on cycle 26, C is high but this is not relevant, because we only start looking for C in the cycle after the first occurrence of B. From cycle 27, we are looking for either C for the property to pass, or another B to continue our nonconsecutive repetition count. We have a second B at cycle 29, so from cycle 30 , we are looking for either C for the property to pass, or another B for the property to fail. On cycle 32, we have a third B with no C, so the property fails. C could occurr on any cycle from 27 to 32, for the property to pass. Specifically for cycle 32, the final term of the property is just looking for C, and says nothing about B. So C occurring on cycle 32 would be a pass, even though B is high for a third time.

© Cadence Design Systems, Inc. All rights reserved.

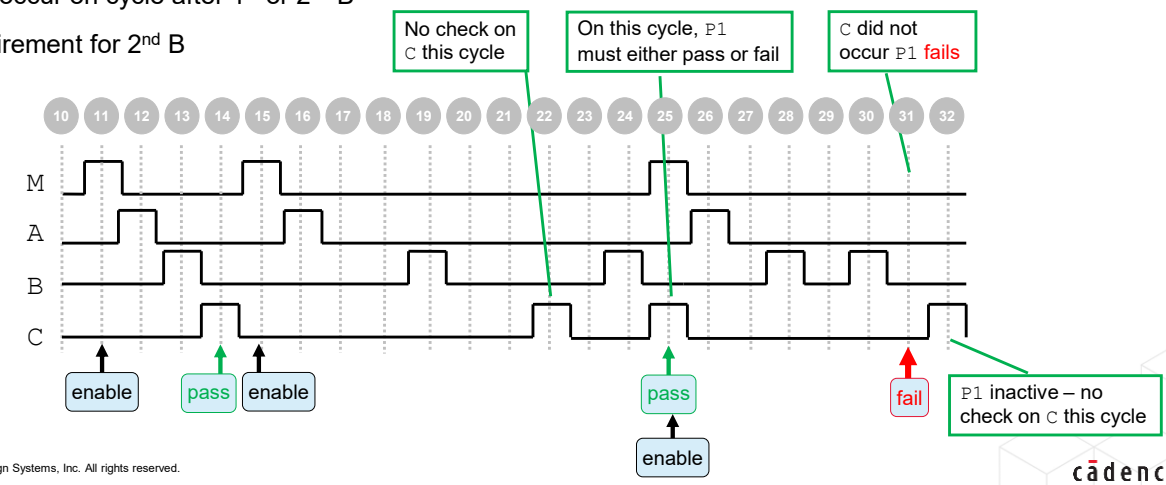
This document is for the sole use of Rajat G of Analog

Go-to Repetition Range Example

This property states:

- No limit on # cycles between A and 1st B
- No limit on # cycles between 1st and 2nd B
- C **must** occur on cycle after 1st or 2nd B
- No requirement for 2nd B

```
property P1;
   ( M |=> A ##1 B[->1:2] ##1 C);
endproperty;
```



80 © Cadence Design Systems, Inc. All rights reserved.
cadence

This is exactly the same property, but using go-to repetition for B instead. The quickest way in which P1 passes is exactly the same as before. M followed by A followed by one B followed by C.

From cycle 15, we have M followed by A and then the first B on cycle 19. The C on cycle 22 has no effect. If C had occurred cycle 20, on the cycle after the first B, P1 would have passed. Go-to repetition requires C to occur one cycle after the first B or one cycle after the second B. At cycle 24 is the second B, so the go-to repetition term completes on that cycle. So on cycle 25, we need C for the property to pass, otherwise P1 fails.

From cycle 25, we have M followed by A. Cycle 28 is our first B. On cycle 29 we need C for the property to pass, or we start looking for another B to continue our go-to repetition count. On cycle 30, we see the second B, so go-to repetition term completes on that cycle. We need C on cycle 31, but C is false so P1 fails on cycle 31. The C on cycle 32 is too late. The property has already failed and is inactive.

Nonconsecutive and go-to repetition can be one of the most difficult to understand features of SVA. Please study these examples carefully to make sure you comprehend the concepts.

```
<<[->N to M]>> <<[->N: M]>>
```

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog

Repetition Operators Shorthand



Shorthand for common sequence repetitions

- Can also be used in cycles range delays
- Need to use IEEE 1800-2009 switch or later in tool

`myseq[*0:$]`	Zero or more repetitions of myseq	`myseq[*]`
`myseq[*1:$]`	One or more repetitions of myseq	`myseq[+]`
`A ##[0:$] B`	B in the same cycle as A or any future cycle after A	`A ##[*] B`
`A ##[1:$] B`	B in cycle following A or any future cycle after A	`A ##[+] B`
1800-2005, 1800-2009 1800-2012, or 1800-2017		Only 1800-2009 1800-2012, or 1800-2017

81 © Cadence Design Systems, Inc. All rights reserved.

cadence

These shorthand notations can be used with any form of consecutive repetition – cycle, Boolean or sequence.

This shorthand notation is exactly the same as IEEE 1850, Property Specification Language (PSL).

Hence, this further reduces the differences in semantics and the learning curve between SVA and PSL.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog

Guidelines for Writing Property



Verification fails when assertions copy RTL, repeat, or conflict, or are underspecified, causing missed bugs and incorrect behavior to slip through.

Properties should *not* duplicate the Verilog implementation.

Properties should be created *from* the specification *using* the implementation.

- Not vice-versa

```
...
if (ADD)
  C <= A + B;  // Add A & B

property CHECK_ADD;
  @(posedge CLK) ADD |=> (C ==(A+B));
endproperty

A1 : assert property (CHECK_ADD);
```

Avoid Copying RTL Verilog

Properties should be defined alongside stimulus in a test plan

- What are you testing?
- How are you going to test it?
  - Stimulus
- How do you check that you have tested it?
  - Coverage
- How are you going to verify results?
  - Assertions

 © Cadence Design Systems, Inc. All rights reserved.

cadence

Blindly duplicating RTL code in SystemVerilog Assertions can mean:

Errors in the RTL are duplicated in SystemVerilog Assertions.

Assertions are written at such a low level they become ineffective. (As in the example above – the add is not going to fail, so why check it?)

Indeed, who would write the comment //Add A & B? No one because it is too obvious.

Ideally assertions should be defined as part of a formal test plan.

Note that the snippet code is not accurately translating RTL to SVA. The correct SVA is:

```
ADD |=> (C == ($past(A)+$past(B)));
```

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog

Incorrectly Specifying Assertions

Properties can be redundant

```
property OC1; (A != B); endproperty
property OC2; (B != C); endproperty
property OC3; (C != A); endproperty
```

If OC1 and OC2 fail, then so will OC3

This doesn't mean OC3 couldn't tell us something we wouldn't otherwise know if OC1 and OC2 pass

Properties can potentially conflict

```
property CON1; !A; endproperty
property CON2; !B; endproperty
property CON3; (A || B); endproperty
```

- A simulator or formal tool would not warn if these were assertions
- A formal tool would warn if these were assumptions

 © Cadence Design Systems, Inc. All rights reserved.

cadence

Incorrectly specifying can lead to duplicated or redundant assertions that are explicitly or implicitly covered by previous assertions.

Incorrectly specifying can also lead to conflicting assertions, where assertions will always fail.

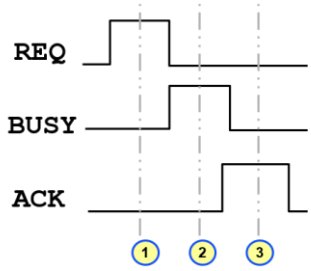
© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog

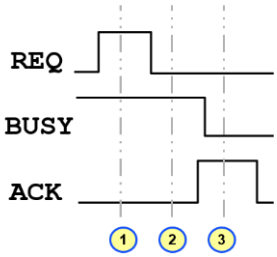
Under-Specifying Assertions

Write a sequence to detect this waveform

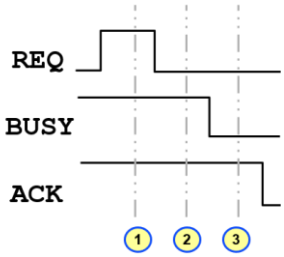
```
(REQ ##1 BUSY ##1 ACK)
```



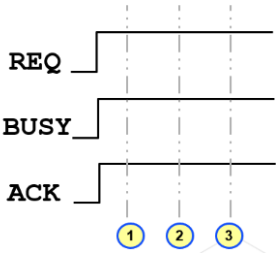
However, this waveform also satisfies this sequence...



as does this ...



and even this ..



84 © Cadence Design Systems, Inc. All rights reserved.

cadence

The above sequence checks REQ on the first cycle, BUSY on the next and ACK on the last. REQ is not checked after the first cycle. Likewise, BUSY and ACK are not checked outside of their respective cycles. The sequence may look straight-forward, but there are a large number of waveforms that satisfy the sequence. If all these are acceptable, then there is no problem. If, however, some of these sequences are unacceptable, then you must define the sequence more strictly.

If the first waveform was the only permitted solution, then your sequence should be:

```
(REQ && !BUSY && !ACK) ##1 (!REQ && BUSY && !ACK) ##1 (!REQ && !BUSY
&& ACK)
```

© Cadence Design Systems, Inc. All rights reserved.

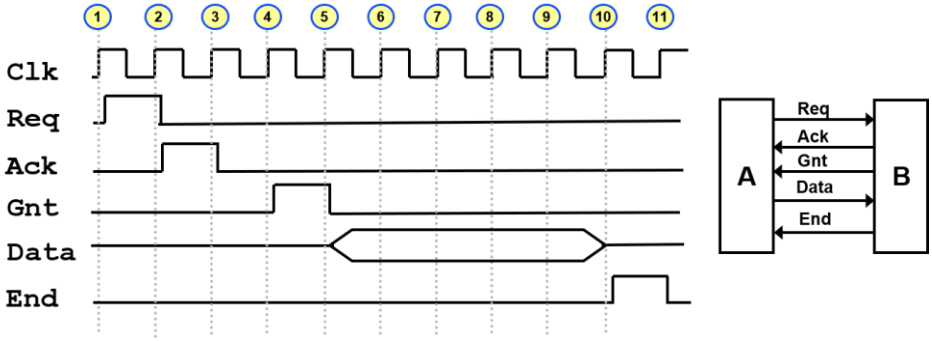
This document is for the sole use of Rajat G of Analog

Quiz

After REQ in one cycle followed by ACK in the next what must then occur is:

- GNT must be low in the same cycle as ACK, and goes high between 1 and 2 cycles *after* ACK goes high
- END must remain low for 5 CLK cycles after GNT as DATA clocked in
- END then goes high

Write an assertion to check the behavior described above



 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

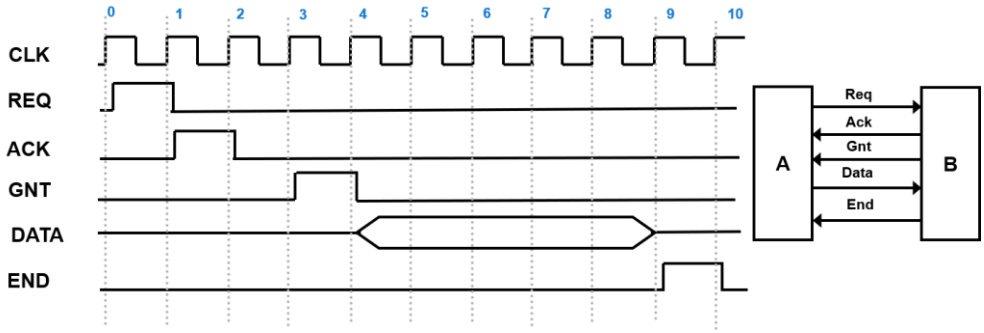
© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog

Quiz Solution

```
@(posedge CLK) (REQ ##1 ACK) |-> !GNT[*1:2] ##1 GNT ##1 !END[*5] ##1 END;
```

```
@(posedge CLK) (REQ ##1 ACK) |-> !GNT ##1 [1:2] GNT ##1 !END[*5] ##1 END;
```



86 © Cadence Design Systems, Inc. All rights reserved.

Enabling sequence is (REQ ##1 ACK). You use the same cycle implication for the fulfilling sequence as GNT must be low in the same cycle as ACK. GNT must be low for 1 to 2 cycles before going high, (!GNT[*1:2] ##1 GNT). After GNT, look for 5 cycles of END low followed by END high (!END[*5] ##1 END). So the full property is:

```
property P1;
  @(posedge CLK)
  (REQ ##1 ACK) |-> !GNT[*1:2] ##1 GNT ##1 !END[*5] ##1 END;
endproperty
A1 : assert property (P1);
```

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog

Summary

Sequences allow complex, multi-cycle properties to be easily expressed

- Boolean expressions separated by cycle delay `##N`

SVA has several operators and features to aid definition of multi-cycle and repeated sequences

- Ranged and un-ranged repetition operators
  - Consecutive repetition `[*N]`
    - Applies to sequences and individual boolean expressions
  - Nonconsecutive repetition `[=N]`
    - Applies to individual boolean expressions
  - Go-to repetition `[->N]`
    - Applies to individual boolean expressions

Ranges can be defined for cycle delay and repetition operators

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog

Lab



Lab 2 Sequence-Based Properties

- In this lab, you will evaluate simple sequence-based assertions in simulation and extend them to support more complex sequences.

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog



Module 5

Coverage

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog

Module Objectives

After this module, you will be able to

- Describe the application of SVA to Sequence-Based Coverage
- Explain what coverage is and why it is essential for measuring Verification Completeness
- Differentiate between structural coverage and functional coverage in verification
- Describe the purpose and usage of the SVA cover directive
- Utilize coverage results to debug assertion behavior

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog

Measuring Test Effectiveness Using Code Coverage



How do we know whether our test vectors are effective?

- A basic way to measure test effectiveness is code coverage
  - It shows how many times each line of code is executed by the test data
  - **But unexecuted code => holes in test coverage => undetected bugs**

Statement Coverage

```
          always @(posedge clk or posedge rst)
    20      if (rst)
     1        op <= 8'h00;
    19      else
    19        case (opcode)
    19          ADD : op = a + b;
   *0*          SUB : op = a - b;
              ...
              endcase
```

process never stimulated with `opcode = SUB`

91 © Cadence Design Systems, Inc. All rights reserved.

cadence

Code coverage identifies lines of code that are not executed for a given set of test vectors.

Untested code shows holes in test coverage. Untested code may contain bugs that have not yet been detected. In addition, untested code may represent redundant logic or un-testable code.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog

Functional Coverage: A Complement to Code Coverage

Structural metrics like code coverage are static

- Check every code branch exercised
- Check every value of an object applied
- No design knowledge required to define this check

Functional metrics use "dynamic" coverage

- Check sequence of values applied to an object
- Check sequence of events (transactions) has occurred
- Design knowledge and intent required to define this check

Functional coverage analyzes design use scenarios

- Complementary to code coverage

```
typedef enum {add, sub, divide} t_opcode;
```

Static value coverage

add	☑
sub	☑
divide	☑

Dynamic sequence coverage

add;sub;divide	☑
add;divide;sub	☑
add;add;add;add	☑
sub;sub;sub;divide	☒

92 © Cadence Design Systems, Inc. All rights reserved.

cadence

Code coverage is static. It will check that every line of code or every opcode has been tested, but it will not check that sequences of opcodes have been tested. This is important as our design may have a problem where 2 subs followed by a divide causes an overflow error.

Code coverage is a basic measure, checking that every line of code has been exercised. Functional coverage, as the name suggests, checks if the functionality of the design has been exercised by the stimulus. You cannot exercise every combination and sequence of events in a design in simulation, but you can identify which critical combinations to exercise to provide confidence in the stimulus.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog

Functional Coverage Using SystemVerilog Assertions (SVA)



Functional coverage requires specifying legal and illegal sequences, triggering and terminating coverage events, and multi-cycle behavior over time.

This demands a powerful and expressive language!

Solution: SystemVerilog Assertions can do this.

```
typedef enum {add, sub, divide} t_opcode;
t_opcode opcode;
```

```
sequence opsub; opcode == sub; endsequence
sequence opdiv; opcode == divide; endsequence
...
sequence opseq1;
  opsub ##1 opsub ##1 opsub ##1 opdiv;
endsequence
```

Sequence coverage

add;sub;divide	☑
add;divide;sub	☑
add;add;add;add	☑
sub;sub;sub;divide	☒

93 © Cadence Design Systems, Inc. All rights reserved.

cadence

Specifying the sequences to be checked during functional coverage can be difficult. Commercial functional coverage tools use proprietary languages to handle this. But SVA has the ability to specify complex multicycle functional sequences in a concise, powerful language.

SVA can be applied to functional coverage definitions as well as dynamic assertion checking.

Note that SystemVerilog has another form of coverage based on covergroups. This is a data-based coverage approach and is covered in the Cadence® SystemVerilog for Design and Verification course.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog

Cover Directive

```
cover property (property_spec);
```

- The assert directive is not suitable for coverage.
- Use the cover directive to count the number of times a sequence occurs
- Cover can be used with either sequences or properties
  - Declaring sequences for use in both assertions and coverage is very efficient
- Coverage must be clocked!

Sequences almost always defined without a clock

```
sequence opsub; opcode == sub; endsequence
sequence opdiv; opcode == divide; endsequence
sequence opadd; opcode == add; endsequence

sequence opseq1;
  opsub ##1 opsub ##1 opsub ##1 opdiv;
endsequence

C1 : cover property ( @(negedge clk) opsub ##1
                      opdiv ##1 opadd);
C2 : cover property ( @(negedge clk) opseq1);
```

`opseq1` clock inherited from cover `C2`

94 © Cadence Design Systems, Inc. All rights reserved.
cadence

The assert directive is not suitable for coverage. If you want to check the coverage of 3 subs followed by a divide, attaching an assert directive to the sequence only reports errors when the sequence does not complete.

SVA has the cover directive. Cover can be applied to sequences (as above) or properties.

Named or explicit sequences, or any combination of these, can be used in a cover statement.

The sequence cover records how many times the sequence started and how many times it was successfully completed.

Property cover records how many times the property started, failed, and succeeded. If a property is asserted, a good simulator should record property coverage automatically without the need for separate coverage.

Note: Be careful on clocking. Sequences are usually unclocked as the clock is defined in the property where the sequence is used. In the absence of a default clock, an explicit clock must be defined in the cover statement.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog

Cover Statement

Statements can be attached to cover directives for simulation

- Executed when cover completes
- Multiple completes in same cycle gives multiple executions of statement

```
sequence TRANSFER;
  (GntA or GntB) ##1 (Busy throughout Reset[->1]);
endsequence

C3 : cover property ((@(posedge CLK) TRANSFER)
      $display("Test 34 found: reset during transfer");
```

95 © Cadence Design Systems, Inc. All rights reserved.

cadence

By omitting the semicolon from the end of the cover statement, we can add an action block which will be executed when the cover completes. Multiple statements must be enclosed in begin-end.

Named cover statements, like assertion properties, are much easier to manage and debug.

An appropriate default clock should be defined if an explicit clock is not included.

© Cadence Design Systems, Inc. All rights reserved.

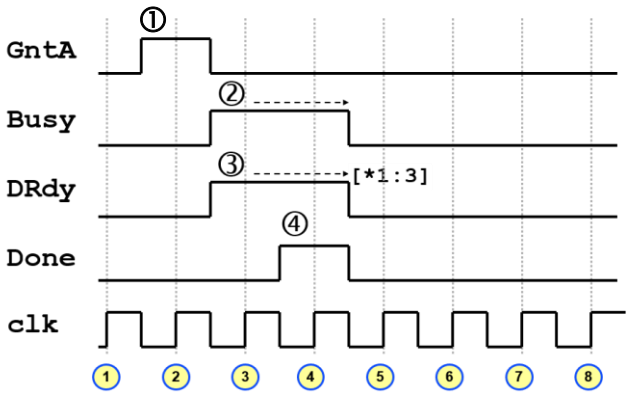
This document is for the sole use of Rajat G of Analog

Example: Covering Variable-Length Transfer

Test when a transfer includes between 1 and 3 successive data ready cycles

```
C4 : cover property (@(posedge clk)
     (GntA || GntB) ##1 (Busy throughout DRdy[*1:3]) ##0 Done );
```

① ② ③ ④



After `GntA` or `GntB`,
`DRdy` occurs 1-3 consecutive times
while `Busy` is high,
ending, and overlapping, with `Done`

96 © Cadence Design Systems, Inc. All rights reserved.

cadence

This cover directive monitors for transfers involving 1, 2, or 3 successive data ready (DRdy) cycles.

Remember with a cover, there is no concept of a property passing and failing. We just want to observe how many times the property occurred.

This example introduces a new keyword throughout. The left-hand side operand is a Boolean and the right operand is a sequence. Throughout states that the LHS Boolean has to be true throughout the duration of the RHS sequence.

The cover C4 is either GntA or GntB followed by Busy true throughout the sequence of 1 and 3 consecutive repetitions of DRdy true. The final occurrence of DRdy must be on the same cycle as Done is true. Remember `##0` is called a fusion, which means the next term (Done) must be true in the last cycle of the previous term (1-3 consecutive repetitions of DRdy).

In the waveform example, we have Busy occurring during DRdy being high 2 times consecutively, with Done true at the same cycle as the second DRdy. So, the cover starts at cycle 2 and ends at cycle 4.

An appropriate default clock should be defined as an explicit clock is not included.

© Cadence Design Systems, Inc. All rights reserved.

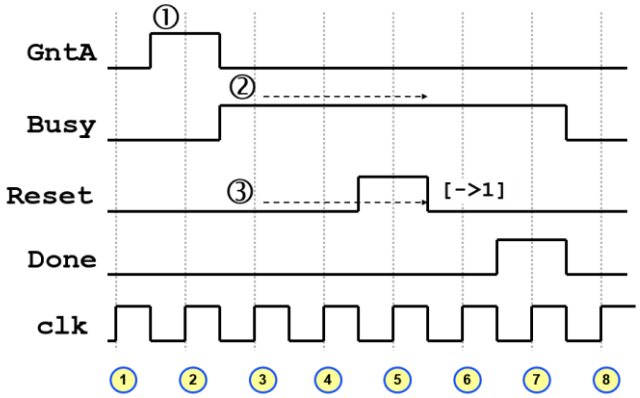
This document is for the sole use of Rajat G of Analog

Example: Covering Reset-Interrupted Transfer

Test the case in which a transfer is interrupted by a bus reset

```
C6 : cover property (@(posedge clk)
    (GntA || GntB) ##1 (Busy throughout Reset[->1]) );
```

① ② ③



`Reset` occurs while `Busy` is still active

97 © Cadence Design Systems, Inc. All rights reserved.

Another coverage scenario

We want to know if a transfer was interrupted by a reset, to check our system can recover correctly.

Here we use the `[->]` operator to match an occurrence of Reset that happens while Busy is high.

Remember Reset `[->]` defines a sequence that finds and ends with the first occurrence of Reset.

So (Busy throughout Reset`[->]`) defines a sequence where Busy is high up to and including the first occurrence of Reset, when the sequence terminates.

So, if we have a grant on A or B, we must have Busy high up to and including the first occurrence of Reset.

In the example, the cover starts at cycle 2 with GntA high. Then from cycle 3, Busy is high until cycle 5 when the first occurrence of Reset is found. So, the cover terminates at cycle 5. Note the throughout operator requires Busy to be high in cycle 5 when the go-to repetition operator on Reset finishes.

In this case, Done is not used in the cover.

An appropriate default clock should be defined, because an explicit clock is not included.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog

Cover in Simulation Versus Formal



In simulation `cover property (my_property)` means:

- From what was observed in a specific simulation count how many times `my_property` was observed to complete

In formal `cover property (my_property)` means:

- Show me an example of `my_property` completing
- This is normally the shortest possible trace

 © Cadence Design Systems, Inc. All rights reserved.

cadence

So coverage in simulation and formal treats the same SVA code differently.

In simulation, when you cover a property, it means count how many times the property was observed to complete during the verification.

In formal, when you cover the same property, it means show an example of the property completing. This example will be a set of waveforms and this would normally be the shortest possible trace. That is useful in itself, knowing the shortest path in which something can happen. So, for example, if I want to know how quickly can I fill a buffer and then empty it, a cover in formal can show you this.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog

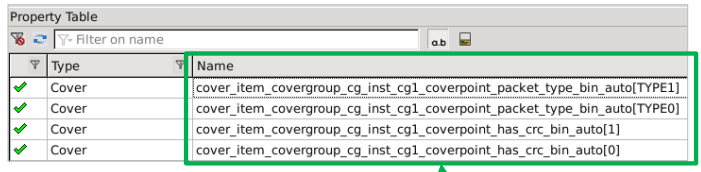
Cover Groups

```
covergroup covergroup_name(clock);
.....
endgroup
```

General SystemVerilog construct not used solely in SVA

- Defined with keywords `covergroup` and `endgroup`

Widely used in simulation testbenches but can also be used in formal.




```
covergroup cg @(posedge clk);
  packet_type:    coverpoint my_packet.packet_type;
  has_crc:        coverpoint my_packet.has_crc;
  cross_type_crc: cross packet_type, has_crc;
endgroup
cg cg_inst_cg1 = new;
```

Jasper® automatically creates one property (bin) for each possible value of the signal

© Cadence Design Systems, Inc. All rights reserved.

cadence

Covergroups are supported in Jasper

Language support limitations are documented in the Command Reference Manual.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog

Using Coverage for Assertions Debugging



When assertions have long or alternate enabling sequences, it becomes difficult to understand:

- Whether the enabling sequence starts
- Where it breaks
- Which alternate paths occur

Coverage can be used to gain this visibility.

- Use coverage to detect the start of an enabling sequence
- Use coverage to identify when an enabling sequence does not complete
- Use coverage to observe which parts of alternate enabling sequences occur
- Use coverage to observe assertion pass and fail behavior

Tool support for this varies...

```
property P1;
  A ##1 B ##1 (C or D) ##1 E |=> END);
endproperty

C0: cover property
  (A);

C1: cover property
  (A ##1 B);

C2: cover property
  (A ##1 B ##1 C);

C3: cover property
  (A ##1 B ##1 D);

C4: cover property
  (A ##1 B ##1 C ##1 E);

C5: cover property
  (A ##1 B ##1 D ##1 E);
```

100 © Cadence Design Systems, Inc. All rights reserved.

cadence

Coverage can also help in debugging long-sequence assertions.

Systematic coverage can check each stage of the enabling sequence if the assertion is not becoming active in simulation, or can check that each alternative sequence of a fulfilling sequence does occur.

Property P1 is probably a good example of what not to write. There is multiple ways in which the left-hand side enabling sequence can complete. If we find the property is never checked because the enabling sequence is never true, the question is "why not"? How is the enabling sequence failing to complete? We need to manually write covers to understand why the left-hand side doesn't occur . No tool will do this for you.

So, we cover A occurring. Then A followed by B. After B there is sequence or (C or D) which gives two alternative paths, so we must cover ABC and ABD. And then either of those 2 paths can be followed by E. So, this allows us to discover what is happening in the enabling sequence. And we had to write these covers ourselves, manually, because we created the problem by writing a much too complicated left-hand side.

An appropriate default clock should be defined as an explicit clock is not included.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog

What Does Cover Property Mean?



A cover property measures the completion of a property, including vacuous success.

In simulation, the answer seems so intuitive one never stops to consider it.

Count how many times I observe `A` followed by `B`

```
C10 : cover property ( A ##1 B);
```

Count how many times I observe `A` followed by `B`

```
C11 : cover property ( A |=> B);
```

Do they really mean the same thing?

- Until the 2012 LRM, the answer was ambiguous
- To most simulators and formal tools, the answer was YES

101 © Cadence Design Systems, Inc. All rights reserved.

Normally you cover a sequence, e.g. (A ##1 B). However originally in SVA, it was syntactically legal to cover an implication (which is a property expression) e.g. (A |=> B).

Do these mean the same thing? Historically the answer was yes, or most tools interpreted these as the same. However, users gradually realized this was wrong. Unfortunately, the issue was not fully fixed in the 2009 LRM and it was not until the 2012 LRM that it made explicitly clear these covers were not the same. That causes a lot of problems with backwards compatibility, and specifically with users writing covers with implication (incorrectly but in good faith) finding their covers were re-interpreted in the subsequent LRM versions.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog

What Does Cover Sequence Mean?



Cover sequence measures actual sequence matches only, with no implication ambiguity.

Cover sequence was introduced in the 2009 LRM

The argument can only be a sequence

- i.e., no implication allowed
- Indeed, just as in PSL

Count how many times I observe `A` followed by `B`

```
C10 : cover sequence ( A ##1 B);
```

Illegal syntax in 2009 and later

```
C11 : cover sequence ( A |=> B);
```



102 © Cadence Design Systems, Inc. All rights reserved.

cadence

The 2009 LRM introduced a partial fix for this issue, allowing the keyword sequence to be used with cover. So, you can say cover sequence (A ##1 B) because (A ##1 B) is a sequence.

However, the implication (A |=> B) is a property expression, not a sequence. So that will be a syntax error to use implication with cover sequence.

However, 2009 LRM still allowed you to cover a property, so you could still write cover property (A |=> B); (although you should avoid this).

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog

Property Vs. Sequence Coverage: LRM 2012/2017 Clarifications

The Formal Syntax of LRM 2012 makes it explicit what this means.

```
C12 : cover property ( A |=> B);
```

Show/count a trace of the property “passing/completing”

This means either:

1. `A` followed by `B`
2. `A` not occurring (remembering what an implication means)

However, 2. is not very useful

Recommendation: If you want to cover the sequence `(A ##1 B)` then use:

```
C13 : cover sequence ( A ##1 B);
```

 © Cadence Design Systems, Inc. All rights reserved.

cadence

The 2012 LRM makes it explicit, where it wasn't before, that cover property (A |=> B); succeeds in two circumstances.

A followed next cycle by B, which is what we always thought it meant.

A not occurring. So, if the enabling sequence of an implication is not true, it is a vacuous pass, and the fulfilling sequence is not checked.

Obviously, it is not very useful to have a cover of A not occurring, but that's what the Language Reference Manual has decided.

If we use the 2009 cover sequence form, we would never use an implication operator because that would be a syntax error. However, if your properties were written using the original 2005 cover property form, you may have implication in the property (because that was legal for 2005 and interpreted same as a sequence). So now in the 2012 LRM, you have a backwards compatibility problem in that cover property using implication does not mean the same thing anymore.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog

Backward Compatibility Issue with `cover` Verification Directive

Prior to the 1800-2009 LRM the only way of writing a cover directive was like this:

```
C_W_IMP : cover property ( A |=> B);
C_SEQ   : cover property ( A ##1 B);
```

Unfortunately, many people wrote cover directives with implications in them, i.e., like `C_W_IMP`

- …and they kept on writing them like that

This leads to undesirable consequences from LRM 1800-2012 onwards, as we have seen

This begs the question, “Why cause backward compatibility problems?”

The answer is so that you can write an infinite-length cover, which can do things like:

- “Show (cover) that it is always **possible** for signal `B` to go high in the future”
- This is not the same as saying “`B` must always go high in the future” (which would be an assertion)
- For more information, search for:



TB: Can I Cover Infinity With SVA Properties

104 © Cadence Design Systems, Inc. All rights reserved.

cadence

So why was the change required? There must have been a reason to introduce backwards compatibility issues?

The answer is to do with infinite length covers, which can be really useful in formal. An infinite length cover allows you write a cover which checks it is always possible for (e.g.) signal B to go high in the future. This is not the same thing as saying B must always go high in the future because that's an assertion. Saying it always possible for "something to go high" is not the same as saying it "should always go high". The non backward compatibility changes were made so you can define an infinite length cover.

There is more information on infinite length covers in the link on the slide, or search for "Can I cover Infinity with SVA properties?" for a video which explains the topic in detail.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog

Summary

Assertions only catch bugs that occur during verification

- Conditional assertions must be enabled before they can fail

Coverage Monitors ensure that verification is thorough

- Monitors check that all interesting behavior is exercised
- Functional coverage is more effective than code coverage

SVA includes both

- Assert Directives (assertions, i.e., checks)
- Cover Directives (coverage monitors)

 © Cadence Design Systems, Inc. All rights reserved.

cadence

Assertions are half the battle – they catch bugs that occur

Stimulus is the other half of the battle – the right stimulus is necessary to trigger bugs

Coverage is a measure of whether we've stimulated a design thoroughly, to trigger all bugs

Coverage Monitors observe the design during simulation and report what behavior is exercised

The "cover" directive creates a Coverage Monitor, using the same behavioral description features as used for assertions

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog



Module 6

Liveness Properties

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog

Module Objectives

After this module, you will

- Define a strong liveness property independent of LRM version
- Outline strong and weak properties and identify their usage
- Identify whether a property is strong or weak based on the LRM version and default strength
- Distinguish Strong and Weak Operators in the LRM

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog

Liveness Compared with Safety Properties



A liveness property is one where at least one way of demonstrating a failure requires an infinite-length waveform.

- Liveness property claims that something **good** happens, eventually
- Has no meaning in simulation
  - Can't simulate for an infinite number of cycles
- Strong creates Liveness properties



A safety property is bound in time, which means all failures are demonstrated by a finite-length waveform.

- Safety property claims nothing **bad** happens
- Failure can only be demonstrated by a finite waveform
  - Hence, all failures can be shown in simulation
- Weak creates Safety properties

Note: 1800-2009 introduced weak and strong properties. Strong and weak relates to how failures of properties can be shown

 © Cadence Design Systems, Inc. All rights reserved.

cadence

A liveness property is one where at least one way in demonstrating a failure requires an infinite length waveform. Or alternatively a liveness property claims that something good happens eventually. For simulation this has no meaning, because there is no such thing as eventually or an infinite waveform in simulation. You cannot simulate for an infinite number of cycles.

In contrast, safety properties are bound in time, which means all failures are demonstrated by a finite length waveform, which means all failures can be shown in the simulation. Alternatively, a safety property claims nothing bad happens.

In the original SVA 2005 LRM, there was no concept of weak or strong. Everything was strong. In SVA 2009 they introduced the concept of weak. This can lead to backwards compatibility issues between the original and later SVA standards.

Strong and weak relate to how the failures of the properties can be demonstrated. Strong creates liveness properties and weak creates safety properties.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog

Example: Liveness Vs. Safety Properties

`SAFETY1` holds if and only if:

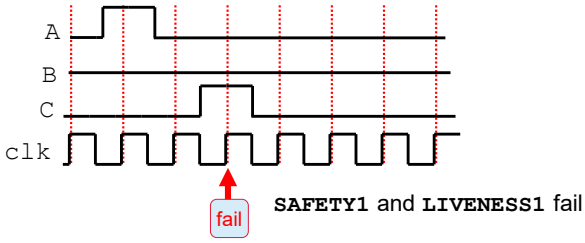
- `A` can never occur
- `A` occurs and then `B` occurs before `C`
- `A` occurs but `B` or `C` never occur
  - Makes no requirement for `B` or `C` to occur

```
SAFETY1 : assert property (
    @(posedge clk) A |-> weak(!C[*] ##0 B) );
```

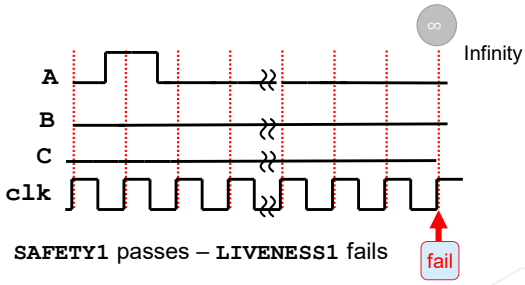
`LIVENESS1` holds if and only if:

- `A` can never occur
- `A` occurs and then `B` occurs before `C`
  - `B` must occur or property fails

```
LIVENESS1 : assert property (
    @(posedge clk) A |-> strong(!C[*] ##0 B) );
```



SAFETY1 and **LIVENESS1** fail



SAFETY1 passes – **LIVENESS1** fails

109 © Cadence Design Systems, Inc. All rights reserved.

Here is the same property, except the safety property uses keyword weak and the liveness property uses keyword strong. The property states if A is true, then from the same cycle C is low repeated 0 to an infinite number of cycles before B is high. C must still be low in the same cycle B is high . So, this says if A occurs, then we must observe B before we observe C.

So, what's the difference between the strong and the weak versions? There are two ways in which this property can fail. On the left-most waveform, we have A occurring and then we have C before B, so both properties fail. This is a finite length failure. So, both the safety property and the liveness property will fail.

The other failure is if A is true and then C stays low forever and B never occurs. The safety property identifies the whole fulfilling sequence as weak using brackets. Weak means we do not require this sequence to complete. As long as C stays low, we do not require B to occur. The liveness property identifies the fulfilling sequence as strong, which means we must see this sequence complete, i.e. B must occur. If A occurs and C stays low forever and B never occurs, that's a failure for the strong property.

We cannot verify to infinity in simulation, but we can in formal.

So, the strong property is a mixed liveness. It has both finite length counter examples and infinite length counter examples.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog

Default Strength of SVA Directives



- In 2005 LRM the concept of weak does not exist
- From 2009 LRM onwards, if strong or weak are omitted the default strength depends upon usage
- The default is weak for these verification directives
  - assume
  - assert
- The default is strong for these verification directives
  - cover
  - restrict
- Properties shown are equivalent in 1800-2009

```
SAFETY1 :assert property(
  @(posedge clk)
    A |-> (!C[*] ##0 B) );
```

≡

```
LIVENESS1 :assert property (
  @(posedge clk)
    A |-> weak(!C[*] ##0 B) );
```

 © Cadence Design Systems, Inc. All rights reserved.

cadence

In the 2005 LRM, weak did not exist, so everything is strong.

From 2009 onwards if strong or weak are omitted, there is a default that depends upon which verification directive is used.

For assume and assert, the default is weak.

For cover and restrict the default is strong.

So, the two examples are equivalent in the 2009 LRM because the default for asserts is weak.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog

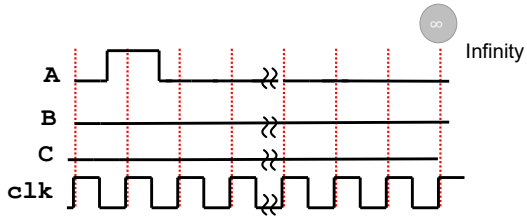
Standard Incompatibilities

- Default strength means different behavior in 2005 and 2009
- In IEEE 1800-2005, the property fails
  - IEEE 1800-2005 only had strong semantics
- In IEEE 1800-2009, the property passes
  - Default strength for assertion is weak

```
P1 :assert property(
  @(posedge clk)
    A |-> (!C[*] ##0 B) );
```

Caution

1800-2009 pass – 1800-2005 fail



111 © Cadence Design Systems, Inc. All rights reserved.

cadence

Default strength gives different behaviour with different standards.

In SVA2005, property P1 fails if B does not occur as the only strength is strong.

In SVA2009, property P1 passes, because the default strength for an assert is weak, so we don't require B to occur.

So, the version of the Language Reference Manual which is used for compilation determines whether the same property passes or fails.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog

What Are Linear Temporal Logic (LTL) Operators?



Linear Temporal Logic (LTL) operators are temporal operators used in SystemVerilog assertions to describe how a property should behave over time, making temporal relationships easier.

- There were many new operators added in IEEE-2009
- Most do exactly the same as their PSL namesake
- Cannot define anything you couldn't already do with sequences
  - Just makes it easier and more intuitive (and in many cases, that is very questionable)
- There are strong and weak forms
- Strong forms prefixed with s_

Strong	
s_until	s_eventually
s_until_with	s_implies
s_always	s_nexttime

Weak	
until	eventually
until_with	implies
always	nexttime

 © Cadence Design Systems, Inc. All rights reserved.

cadence

SVA2009 introduced many new Linear Temporal Logic (LTL) operators. These, largely, do not do anything you can't define already with sequences. They were added to make it easier to write more intuitively, which in many cases is very questionable, as we shall see in Part 2 of this course, for simulation users only, where we describe some of these operators in more detail.

Each operator has a strong form and a weak form. The s_ identifies the strong form.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog

Example: `eventually` Versus `s_eventually`

- These keywords only exist in IEEE-2009 or later
- `eventually` **must** have a bounded range, i.e., can't use `$` as upper bound
- `s_eventually` may or may not have a range
  - The range may or may not be bounded
- In virtually every case, it is not of pragmatic use for any form of `eventually` to have a bound
- Hence, `s_eventually` with no bound is used almost all of the time

```
W_EVEN : assert property (
  REQ |-> eventually[1:3] GNT);
```
≡
```
P1 :assert property (
  REQ |-> weak(##[1:3] GNT) );
```

```
S_EVEN_BND : assert property (
 REQ |-> s_eventually[1:3] GNT);
```
≡
```
P2 :assert property (
 REQ |-> strong(##[1:3] GNT) );
```

```
S_EVEN_UBND : assert property (
 REQ |-> s_eventually GNT);
```
≡
```
P3 :assert property (
 REQ |-> strong(##[0:$] GNT) );
```

 © Cadence Design Systems, Inc. All rights reserved.

cadence

The most frequently used LTL operator is eventually.

The weak eventually, without s_, requires a bound and cannot use $ as the upper bound. The weak eventually is of little use. W_EVEN says if we have REQ then eventually, within one to three cycles, we get GNT. Which is exactly the same as writing P1.

The strong s_eventually may or may not have a range. Putting a range on s_eventually is uncommon, because a sequence form may be easier and because eventually is usually open-ended. S_EVEN_BND is exactly the same as P2.

An unbounded s_eventually is the most likely to be used. If we have a REQ then, eventually at some point in the future, we must have a GNT and the GNT must occur. S_EVEN_UBND is the same as P3, but is a lot more readable and unambiguous. You don't have to explicitly include strong. So, for that reason, you'll see s_eventually all the time.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog

Example: Fairness Constraint with `s_eventually`

- Assumption requirement at every `posedge clk`:
  - `STALL` input to FSM must be false at some point in the future
- You are stating that you do not want to be shown a property fail by formal tool holding `STALL` true forever. This is known as a **Fairness constraint**
- All Properties shown are equivalent
  - Only if correct standard release is followed

1800-2009

```
P2009_A :assume property (
    @(posedge clk)
        s_eventually !STALL) );
```

≡

1800-2005

```
P2005 :assume property (
    @(posedge clk)
        1'b1[0:$] ##1 !STALL );
```

≡

1800-2009

```
P2009_B :assume property (
    @(posedge clk)
        strong( 1'b1[0:$] ##1 !STALL ));
```

114 © Cadence Design Systems, Inc. All rights reserved.

cadence

An FSM design may have a STALL input which prevents the FSM from changing state. An FSM assert may say if we are in state one, eventually we get to state three. A formal tool can show this assertion failing by holding the STALL input high all the time, so the FSM never makes any transitions, In fact a formal tool can show all FSM assertions failing, simply by holding the STALL input true forever.

In order to prevent that kind of false negative, we have to use a fairness constraint.

All these properties are equivalent, but the most common form is to use s_eventually. At every posedge clk, s_eventually, i.e. at some point in the future, not STALL must be true. Now the formal tool cannot show a failure with STALL held high forever. That's why it's called a fairness constraint, because STALL held high forever is unfair.

s_eventually is the most intuitive form although if you compile for SVA2005, this will be a syntax error.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog

Summary

- The 2009 standard changes the behavior of some liveness assertions
- It's essential that:
  - You understand the difference
  - You know which version of the LRM that you are compiling with
- Using `s_eventually` is the most intuitive way to express the desired behavior unambiguously and independent of the LRM version

 © Cadence Design Systems, Inc. All rights reserved.

cadence

We strongly recommend you don't use compile for the 2005 version of the LRM, because of the backward compatibility problems.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog



Module 7

Property Reuse

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog

Module Objectives

In this module, you

- Describe a few language features and methodologies to facilitate the reuse of properties
- Apply parameterized sequences and properties for reusable assertions
- Implement efficient assertion reuse using generate constructs and verification components
- Apply the bind statement to reuse properties without changing RTL code
- Adapt reusable properties across simulation and formal

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog

Sequence Arguments



```
sequence N_CYCLE_RESP(n, p);
  ( !p[*1:n-1] ##1 p );
endsequence
```

Declare common sequences using formal arguments

Map actual to formal arguments at instantiation

- Tools check expanded sequence is legal

Can be instantiated inside property or another sequence

instantiation

```
( $rose(req) ##1 N_CYCLE_RESP(10,ack) ##1 done )
```

result after text expansion

```
( $rose(req) ##1 !ack[*1:9] ##1 ack ##1 done )
```

118 © Cadence Design Systems, Inc. All rights reserved.

cadence

A common sequence that applies to different signals can be defined as a named sequence with arguments. Different objects can then be associated with the sequence arguments when used. Integer values can be passed as arguments, as in the example above, to vary the number of cycles checked by the sequence.

A sequence with simple untyped arguments can be considered as a text macro. A tool will expand a sequence instance using the defined arguments and check the expansion is legal.

An actual argument can replace an:

Identifier

Expression

Event control expression

An actual argument can be $ (infinite upper range for repetition), such as:

```
N_CYCLE_RESP(ack, $)
```

You do not need to pass everything that you want to use in the sequence definition as an argument to the sequence. You still have visibility of all signals in the same scope as the sequence is defined without them being explicitly passed as arguments.

However, you must be aware that reusability of a sequence is limited if you define a sequence using items that are not passed as arguments. This is because you would be relying on local signals with specific names existing in all places where you want to reuse the sequence.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog

Property Arguments

Note the signal CLK is not passed explicitly as an argument

This relies upon "side effects". This is BAD practice. It only works if there is a signal named CLK in the local scope. You should pass everything as an argument for reuse.

Parameterized property declaration

```
property ResultAfterN (START, N, RESULT, STOP);
  @(posedge CLK) disable iff (STOP)
  START |=> ##N RESULT;
endproperty
```

Instantiation

```
A1 : assert property (ResultAfterN(WRITE_REQ, 3, ACK, CANCEL));
```

Result after text expansion

```
A1 : assert property (
  @(posedge CLK) disable iff (CANCEL)
            WRITE_REQ |=> ##3 ACK);
```

Recommended for you to research in the LRM:

- Properties can have more kinds of arguments than sequences, e.g., property arguments can be events or other properties
- Sequence and property arguments can be defined as being typed and having a direction

TB: SVA Local Variables Practical Examples (6.44 mins onwards)

119 © Cadence Design Systems, Inc. All rights reserved.

cadence

Properties can have a few more arguments than sequences. For example, arguments can be events, not just Booleans. Here is a property definition with START, N, RESULT and STOP as the arguments. We define an instance of this property and map WRITE_REQ to START, 3 to N, ACK to RESULT and CANCEL to STOP. Again, because we have simple arguments with no typed or direction, the instance is just a text macro expansion. The tool will check the syntax after expansion.

In this example CLK is not passed as an argument. This is legal syntax and will work as long as there is a signal visible in the local scope named CLK. This is called side effects, where we rely CLK being visible. This is usually considered bad practise, because it can affect reuse. Usually, a property with arguments should declare everything used in the property as an argument. This avoids local side effects and issues with reuse.

Properties can have different kinds of arguments than sequences, such as events, and an argument could be another property. So, you can define one property in terms of another. Although we would recommend against that. It is not really of practical use and creates difficulties in understanding whether it's working correctly or not and debugging it.

Sequence and property arguments can be defined as being typed and having a direction, although this has lots of limitations. Please click or search for "SVA Local Variables Practical Examples" for further information.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog

Generating Assertions

```
generate
  generate_item
endgenerate
```

`generate` can be used to create multiple or conditional assertions

- Verilog2001 statement

Useful for properties with arguments

- Here `generate for` creates an assertion for each bit of a vector

Conditional, case and nested generates can also be used

Remember `generate for` requires:

- Loop variable declared as a `genvar`
- A `begin` label to uniquely identify each generated `assert`
  - You should label each generate block

```
reg [7:0] REQ, ACK;

property VRES (N);
  @(posedge CLK)
  $rose(REQ[N]) |=> ACK[N];
endproperty

genvar i;
generate
  for (i=0; i<8; i=i+1)
  begin :genvres
    A1 : assert property (VRES(i));
  end
endgenerate
```

```
...genvres[0].A1
...genvres[1].A1
...
...genvres[7].A1
```

Each generated assertion with have a unique hierarchical name

120 © Cadence Design Systems, Inc. All rights reserved.

cadence

The Verilog generate statement is very useful for reuse.

In this example we have 8 bit vectors REQ and ACK. The property VRES takes a single argument N as an index to these vectors. Again, this is a bad example in that we should really be passing CLK, REQ and ACK as arguments, but this is just to demonstrate the generate. So VRES says if a particular bit of REQ rises, then the next cycle, that bit of ACK is set. VRES only examines one bit, because N has one discrete value. The generate loop allows us to instantiate VRES for multiple bits. Generate is not procedural code. It is a concurrent statement. A generate loop variable (i) has to be declared using the keyword genvar. Then the generate for iterates i from 0 to 7. You need a begin inside the generate for, with a label because that forms the name of our property. Each iteration creates an instance of VRES, using the index i as the argument to the property. So the loop generates eight properties instances, each with a different value for i from 0 to 7.

Each generated instance is uniquely named by concatenating the hierarchical pathname, e.g. top.U1.U2, the generate label (genvres), the generate loop index value (e.g. [0] and the assertion instance name (A1). The generate label and index number are the parts which uniquify each instance label.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog

What Are Verification Components (Checkers)?



Verification components are generic, reusable modules that encapsulate verification only code, such as properties and auxiliary logic and provide a structured infrastructure for reusable properties across commonly used interfaces and components.

- Verification Components (VCs) function similarly to UVCs in a UVM testbench, instance and `bind` to where they are needed in the DUT.
- We could use the SystemVerilog `checker` construct instead of a normal `module`.
  - If the VC ports are all logic signals, then we can simply replace `module` with `checker`.

```
module fr_cnt_chkr (
        input logic  CLK,
        input logic  rst_n,
        input logic  CNT_ENB,
        input logic[2:0]  CNT_VAL);

  default clocking DEFCLK @(posedge CLK1);
  endclocking
  C1 : cover property ( CNT_ENB && CNT_VAL == 3'b101);
  AST_CNT_NE_6 : assert property (CNT_VAL != 6 );
endmodule
```

```
checker fr_cnt_chkr (
        input logic  CLK,
        input logic  rst_n,
        input logic  CNT_ENB,
        input logic[2:0]  CNT_VAL);

  default clocking DEFCLK @(posedge CLK1);
  endclocking
  C1 : cover property ( CNT_ENB && CNT_VAL == 3'b101);
  AST_CNT_NE_6 : assert property (CNT_VAL != 6 );
endchecker
```

121 © Cadence Design Systems, Inc. All rights reserved.

For reuse, it is recommended to encapsulate all your verification code inside of a container. We will call this container a Verification Component (VC), but these are also known as checkers. VCs are just generic modules with input port only. They are used to define reusable properties and verification code for easy reuse.

VCs are just a normal Verilog module with input ports only. A VC is just for verification, so should not be driving any outputs. The port list contains all signals required to define our properties. We have a default clocking block which applies just to this verification component. The VC will contain covers, asserts and any auxiliary code, which is normal Verilog code to help us write properties. Auxiliary code is covered in the next section.

The 2009 LRM introduced checkers. Checkers are similar to a module. If all the VC the ports are logic types, we can simply replace the keyword module with the keyword checker.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog

Selecting the Right Construct for Verification

At first reading, SystemVerilog Checkers seem to have advantages over modules, for example:

- An EDA tool knows that a checker is there purely for verification purposes
- This allows, for example, checkers to be hidden in the design hierarchy diagram

Checkers can have more types of arguments than a normal SystemVerilog Module, for example:

- Can infer clock and reset from the context in which they are used
- Can be instantiated in procedural code
- Can have properties, events, sequences, etc., in the port list

However, SystemVerilog checker constructs are rarely used for some basic practical reasons.

- Checkers cannot have parameters
- The “extra” features complicate verification and, in practice, are of little pragmatic use

It is recommended to continue using SystemVerilog modules for your VCs for reuse.

 © Cadence Design Systems, Inc. All rights reserved.

cadence

Checkers seem to have advantages over modules.

For example, EDA tools know that checkers in your design hierarchy are purely for verification code and can be ignored for (e.g.) synthesis. Checkers can infer clocks and resets from the context in which they are instanced. Checkers can be instantiated it in procedural code. Checker ports types can be events, sequences, properties etc. whereas module VC ports can only be logic types. However, checkers cannot have parameters. That is the main issue why hardly anyone uses them. All the extra features sound attractive but are of limited practical use. Checkers are a good idea butut are not commonly used. So, it's recommended you use modules for VCs.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog

Assertion Binding Using `bind`

```
bind bind_target_instance
bind_instantiation;
```

- `bind` statement can literally be placed anywhere
  - Including on a line by itself in a separate file
- Instantiates a VC in instance(s) of a hierarchical design module
- Works the same both with modules or SystemVerilog checkers
- Advantages
  - No changes to design module
  - Easier to modify and update
- Can bind to all or specific instances of a design module
  - To bind to every instantiation of `mone`

    ```
    bind mone mone_prop...
    ```
  - To bind only to instantiation `m1` of `mone`

    ```
    bind mone:m1 mone_prop...
    ```

```
module mone_prop(input a,b,clk);
  property P1;
    @(posedge clk) $rose(a) |=> b;
  endproperty
  A1 : assert property (P1);
endmodule
```

Verification Component

```
module mone(input logic clk, req,
            output logic ack,
             ...);
  ...
endmodule
```

Design module

```
module top;
...
  // instantiate design modules
  mone m1 (clk, req1, ack1...);
  mone m2 (clk, req2, ack2...);

  bind mone mone_prop mp1(req,ack,clk);
...
```

Signal names in design module scope

Design module | Verification Component | Instantiation name | Design module variables | Testbench

123 © Cadence Design Systems, Inc. All rights reserved.

cadence

Bind allows you to insert an instantiation of a VC (or indeed any module) into any design module in the hierarchy, by adding a bind construct to, typically, the topmost module in a design.

Hence, you can add modules into a design hierarchy from the testbench without having to edit the design modules.

The bind command effectively inserts a VC instantiation in every (or selected) instance of a named design module.

Syntax for bind:

```
bind
mone                 // Design module name or module name:instance name(s)
mone_prop            // Verification Component module name
mp1                  // Instantiation name of VC module
req,ack,clk);        //Map of VC ports to design module signals
```

You can also bind a VC to a specific instantiation of a design module by using the instantiation name instead of the module name:

```
bind m1 mone_prop mp1(req,ack,clk);
```

© Cadence Design Systems, Inc. All rights reserved.

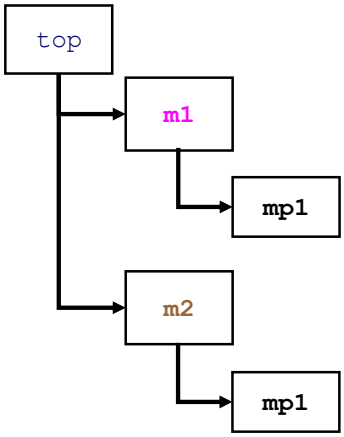
This document is for the sole use of Rajat G of Analog

Assertion Binding in Hierarchical Designs

`bind mone mone_prop...`

Equivalent to an instantiation of the VC inside `top.m1` and `top.m2`

- Linked into design by way of hierarchical path names




```
module mone_prop(input a,b,clk);
  property P1;
    @(posedge clk) $rose(a) |=> b;
  endproperty
  A1 : assert property (P1);
endmodule
```

Verification Component

```
module mone(input logic clk, req,
            output logic ack,
             ...);
  ...
endmodule
```

Design module

```
module top;
...
  // instantiate design modules
  mone m1 (clk, req1, ack1...);
  mone m2 (clk, req2, ack2...);

  bind mone mone_prop mp1(req,ack,clk);
...
```

Testbench

 © Cadence Design Systems, Inc. All rights reserved.

cadence

Bind creates an implicit instantiation of a VC and links this to all instances, or a specific instance, of a design module via hierarchical variable names.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog

Applying Properties Conditionally Using `ifdef`

```
`ifdef ifdef_condition block_of_text
 .........
`endif
```

- Use Verilog `ifdef` statement
- You might want to conditionally include these items:
  - Cases for Formal
  - Abstract Properties for Formal
  - Sanity Checks
- You might want to conditionally exclude these items:
  - EDA tools which don't support SystemVerilog Assertions
  - When verifying at a higher level and these properties are already proven
  - ..and so on

```
module mod1 (input logic CLK,
                ...
             output logic EN1, EN2);

`define SVA_ENABLE

`ifdef SVA_ENABLE
  property PROP1;
    @(posedge CLK) (EN1 | EN2);
  endproperty

  ASSERT1: assert property (PROP1);
`endif
...
```

125 © Cadence Design Systems, Inc. All rights reserved.

cadence

SystemVerilog allows concurrent assertions to be placed in any hierarchical block, including module, interface, package, clocking block or compilation unit scope.

It is essential to have a means of conditionally enabling or disabling properties. For example:

You may have tools which do not support SVA. Formal tools have cases which are deliberate and temporary over constraints to speed up the current verification task. You may need to switch this on or off depending on the current verification goal. Different abstract properties may only work in formal but not in simulation. Temporary checks to test your environment is correct. Maybe you are verifying at a higher level of integration, and you've already proven certain properties, so you don't need them anymore.

Many companies have predefined ifdef value lists. Separate values for simulation only or formal only properties. Or, for example, ifdef sim_and_formal for both.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog

Reusing Properties Across Formal and Simulation



- Properties specify some desired behavior:
  - Behavior of the DUT – Assertions
  - Behavior of DUT's environment – Assumptions
- These say nothing about whether you use Formal or Simulation
- Hence, they should be reusable with either tool
- This is mostly true, but you need to be aware of the exceptions

 © Cadence Design Systems, Inc. All rights reserved.

cadence

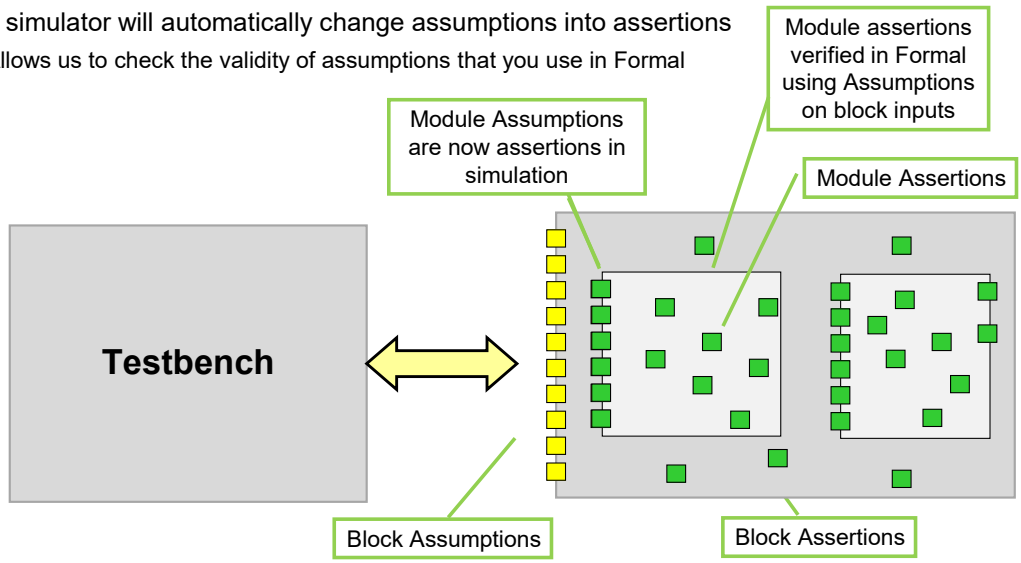
Properties specify some desired behaviour of the design. These are asserted if we wish to prove the design behaviour. However, properties can also specify the behaviour of the DUTs environment, namely what the inputs are allowed to do. These are assumptions when they're used in formal. The properties have no concept of whether you are using formal or simulation. They should be interchangeable between either of these tools. That is generally true, but you need to be aware of the exceptions.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog

Validating Reused Properties Across Formal and Simulation

- Both assertions and assumptions are verified in the simulation
- The simulator will automatically change assumptions into assertions
  - Allows us to check the validity of assumptions that you use in Formal



127 © Cadence Design Systems, Inc. All rights reserved.
cadence

During simulation, the block and module assumptions are checked to ensure that the assumptions used during Formal Analysis still hold during simulation.

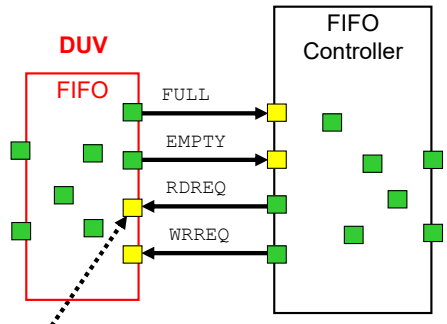
In formal analysis, the design inputs can be driven to any value in order to cause assertions to fail. Assumptions are rules for how the inputs are allowed to be driven, which we use to avoid false assertion failures due to unrealistic input values. Our results are dependent upon these assumptions being correct. How do we know they are correct? The formal tool is not going to check assumptions for us. Well, we can check assumptions in simulation and that is why the simulator will change assumptions into asserts. So, when you integrate a block into a bigger system and simulate it, the block assumptions are turned into asserts. It is important these asserts never fail during a simulation because if they fail, it means the inputs can be driven in a way that violate our assumptions, which means our formal results are probably wrong. Invalid. So, one way of verifying our assumptions is in simulation.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog

Cases Only Applicable to Formal

- You want to formally verify some WRITE functionality of the FIFO
- To simplify the formal verification task, add pin constraints
  - For example, hold `RDREQ` at `'0'`
  - Deliberate overconstraining to reduce state space
  - This is a temporary measure to make verification more convenient
- This is called a **case** in Formal Analysis
- In simulation, this property will certainly fail
- Keep cases separate from normal assumptions, so they can be easily disabled




```
ASM_NOREAD: assume property @(posedge CLK)(!RDREQ);
```

Assertions

Assumptions

 © Cadence Design Systems, Inc. All rights reserved.

cadence

Formal tools have cases which are deliberate and temporary over constraints to speed up the current verification task.

For example, we need to formally verify the write functionality of a FIFO. So, as a temporary measure, we can write an assumption which prohibits read requests. This can exclude all the logic for read operations, which means the tool has less work to verify the writes. So that is a temporary restriction for convenience. We must clearly keep these cases separate. We do not want that property in simulation because clearly it will fail. We can disable the cases by using ifdefs.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog

Formal Complexity Issues



- SVA in Simulation is intuitive and generally WYSIWYG
  - Simulator builds a state machine to check property
- Formal has techniques with which simulation cannot cope
  - For example, FSM deadlock
  - SVA has no constructs to deal with this, but PSL does
- Formal has to perform exhaustive proofs
  - With simulation, all you know is what you tested works
  - Properties that are OK for simulation may vastly increase Formal Complexity
    - Hence, Formal Run times
    - Especially true of assumptions
      - Having a state (repetition) on RHS of the implication is very costly for assumptions

```
property P1;
  ( @(posedge CLK)
          START |=> POP[->1] ##0 (TAG==2 or (BUSY[*1:$] ##1 (!BUSY && SKIP))));
endproperty
```

129 © Cadence Design Systems, Inc. All rights reserved.

cadence

SVA in simulation is generally more intuitive and What You See Is What You Get. Write any property, and the tool builds a state machine to check it during simulation. But formal has completely different techniques that a simulator can't handle. For example, FSM deadlocks. A formal tool can check waveforms to infinity. Formal has got to do exhaustive proofs. With simulation, you can know what you tested worked, but you can't test everything. You can run tens of thousands of tests to maximise your chances of finding bugs, but you can't test everything.

Properties that are good for simulation may be really hard for formal by creating a complexity problem like state space explosion.

The example properties may work well in simulation, but if you try this in formal, there are so many open-ended sequence terms, the tool won't have enough time to give you a proof.

In principle, everything is provable in formal. It just depends on how long you're willing to wait for an answer. And in the case of something like this property example, it's probably too long to make it useful.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog

Summary

You now have some powerful constructs for property reuse

- Parameterized sequences and properties
  - Allow use of common sequence definitions on different signals
- Binding
  - Easier association of assertions with existing models
- `ifdef`
  - Exclude or include properties according to requirements
- Differences in properties for simulation and Formal Verification

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog



Module 8

Auxiliary Code

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog

Module Objectives

At the end of the module, you will be able to:

- Identify verification scenarios that cannot be solved using SVA alone
- Create auxiliary HDL code to support writing effective and efficient SVA properties
- Implement one-shot and state-tracking mechanisms using auxiliary code
- Evaluate when to use auxiliary code versus SVA to achieve accurate, maintainable formal verification

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog

What Is Auxiliary Code?



Auxiliary code is helper code used alongside assertions to make properties easier to write, help the tool evaluate verification problems more efficiently, and perform tasks that assertion languages alone cannot express.

Location of Aux code

- In Verification Components(VC's) / Checkers
- Normal Verilog modules that have only inputs
  - VC's are merely a container for Aux code, assertions, assumptions, cover statements, i.e., only verification code
- Connected to the DUV, even legacy code, using the SV `bind` statement
  - Just instance and connect them where you need them
    - Just like UVCs in a UVM testbench
- Embedded in RTL
  - Must be surrounded with synthesis pragmas or `` `ifdef ``
  - Not recommended

 © Cadence Design Systems, Inc. All rights reserved.

cadence

Auxiliary code (also known as helper code) has three main purposes.

To make it easier to write properties.

To make the problem easy for the tool to evaluate, especially in formal. It often makes assertions easier to debug and understand as well.

It helps writes properties which assertion languages cannot do alone.

Where does auxiliary code go? As discussed in the section on reuse, it goes inside a Verification Component. Remember VCs are modules which only have inputs and are a container for verification code. We can use the bind statement to attach VCs to design modules where they are needed. We could embed them in RTL code. That is supported in the language, but it's not recommended because you would need to hide the code from synthesis using synthesis pragmas, and ifdefs.

All the examples on the following slides are from real-life customer examples.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog

Overlapping REQ/GNT Protocols



Write assertion for REQ/GNT protocol where events must be matched within a bounded time window, allowing overlapping, and prohibit unmatched responses. This initially seems easy in SVA. Why won't it work as planned?

Required request/grant interface behavior:

- If you have a `REQ`, then you must have a `GNT` within 2 to 8 cycles
- Requests can overlap
  - For example, you can have another request before the first grant received
- Each `REQ` must have a corresponding `GNT`
- No `GNT` without a `REQ`

```
A1: assert property(@(posedge clk) REQ |-> ##[2:8] GNT));
```



 © Cadence Design Systems, Inc. All rights reserved.

cadence

Here is a formal description of the example:

Grant Timing: Each request must be followed by a corresponding grant within a defined latency window of 2 to 8 clock cycles.

Overlapping Requests: Multiple requests may be issued before earlier requests have been granted.

Request-Grant Pairing: Every request must be matched with a corresponding grant. Grants must not occur independently; they must be the result of a preceding request.

This specification appears straightforward you would expect SVA to easily support this kind of behavior.

As a first attempt, we could write the example above: If REQ occurs, then GNT must follow within 2 to 8 cycles.

Although this assertion looks reasonable, it fails to function correctly. The failure is not due to complexity or syntax, it is due to fundamental limitations in SVA's ability to express this behaviour. Specifically, there is no viable method within SVA to enforce this protocol as described.

There are two reasons why the assertion will not work:

Overlapping

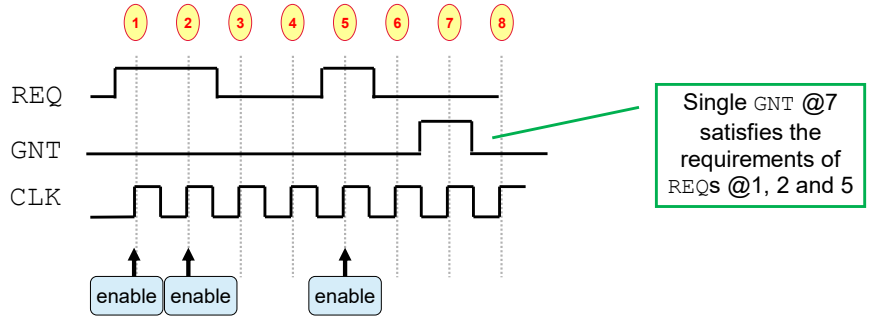
Requirement for a global tally of REQ/GNT handshakes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog

Understanding the Overlapping Assertion Problem in SVA

- Assertion languages have no way of describing what you want
- A single occurrence of GNT can satisfy multiple requests
- You need to model using Auxiliary code (HDL helper code)
  - This will also be much more efficient for formal




```
A1: assert property(@(posedge clk) REQ |-> ##[2:8] GNT));
```

135 © Cadence Design Systems, Inc. All rights reserved.

One reason our simple assertion example will not work is overlapping.

So, remember for implication properties, any posedge at which REQ is true (enabling sequence) we require GNT (fulfilling sequence).

The REQ at cycle 1 enables the first copy of the assertion. REQ still high at cycle 2 enables the second copy of the assertion. The REQ at cycle 5 enables a third copy of the assertion. So, at cycle 6 there are 3 copies of the assertion enabled, all at different stages of completion because they will be at different cycle counts for the GNT window. The single GNT at cycle 7 passes all 3 copies. Clearly that is not what the specification said. This is fundamental behaviour for SVA, try as hard as you wish, but you cannot write an assertion which does that check for you.

So, what is the solution? Well, we need auxiliary code to help write the assertion, and this auxiliary code will also have the side-effect of making formal analysis much more efficient.

Note that auxiliary code is usually very simple. It is mainly setting flags, clearing flags or counting up and counting down and then writing assertions about those flags and counters.

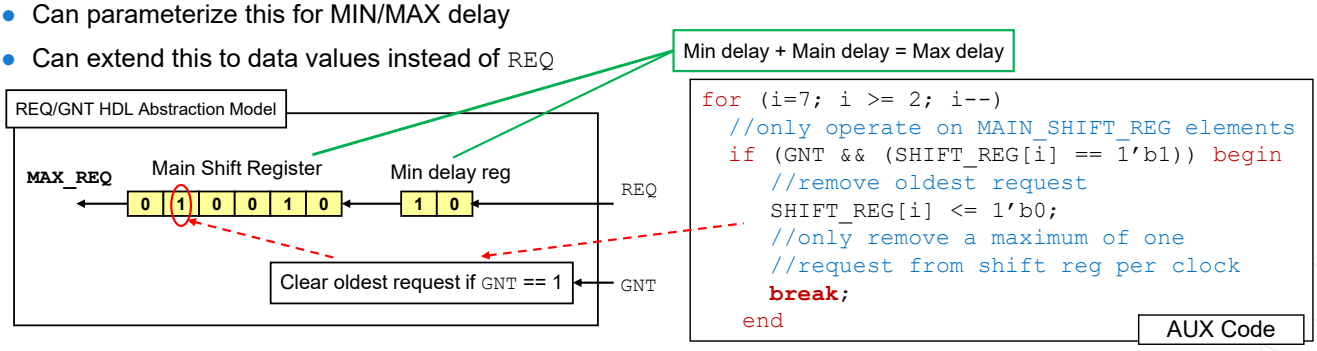
© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog

Solution: Using a REQ/GNT Abstraction

Requirement of the Aux code model:

- If you have a request, you must receive a GNT within 2-8 cycles
- 2 represents the minimum delay for the GNT response
- 8 represents the maximum delay for the GNT response
- Can parameterize this for MIN/MAX delay
- Can extend this to data values instead of REQ




```
A1: assert property ( MAX_REQ |-> GNT );
A2: assert property ( GNT |-> $countones(MAIN_SHIFT_REG) != 0 );
```

☑

136 © Cadence Design Systems, Inc. All rights reserved.

cadence

Auxiliary code can create an abstraction model of our verification problem, which will work for overlapping grants.

We create a shift register of length 8 (the maximum response delay). The lowest two bits of the register are partitioned off as the minimum delay. On every cycle, the register shifts left, REQ is clocked into bit 0, and bit 7 is shifted out to the signal MAX_REQ. Now if we see GNT, we need to remove the oldest REQ. So on each cycle, we iterate through the shift register from left to right, from bit 7 to bit 2. If GNT is true and the current register bit is 1, then we clear that bit and exit the loop. That removes the oldest request. We do not check the lowest 2 bits of the register as they contain requests earlier than the minimum response. Now our properties are really easy. Boolean operands with same cycle implication.

If MAX_REQ is 1, a REQ has reached the maximum cycle for a response and so GNT must be true.

If GNT is true, there must be a REQ in the main shift register, representing between 2 and 8 cycles. We can use $countones to check the main register is not zero. If it is zero, that means we had GNT with not REQ.

So simple auxiliary code and simple assertions solve our problem. The code and the assertions are much easier to understand and really easy for the tools to evaluate, especially formal tools. Much of auxiliary code looks like exactly like this in formal. Simple abstract models of the verification problem allowing us to write simple properties.

Note the auxiliary code looks like just SystemVerilog synthesisable RTL code. Formal tools can generally only process synthesizable RTL code. Note we could also parameterize the auxiliary code for different response windows and extend the model to using data values instead of just request bits.

© Cadence Design Systems, Inc. All rights reserved.

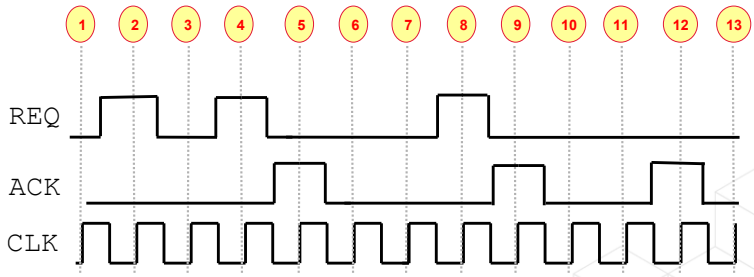
This document is for the sole use of Rajat G of Analog

Overlapping REQ/ACK Handshake Protocol

Write assertions to verify an Overlapping REQ/ACK Handshake protocol, limit outstanding occurrences, ensure eventual matching responses, and prevent unmatched responses. How can you write this with SVA?

A new requirement now…

- Every `REQ` must have an `ACK` sometime later
- Maximum of 5 outstanding requests allowed
- The `ACK` can occur any time
  - There does not have to be an `ACK` before another `REQ`
  - No limit on number of cycles from `REQ` to `ACK`
- No `ACK` without `REQ`



137 © Cadence Design Systems, Inc. All rights reserved.

Another example with a different requirement. Every REQ must have an ACK sometime later. There's a maximum number of five outstanding requests allowed. The ACK can occur any time and there does not have to be an ACK before another REQ, so we can make 2 REQs before the first ACK, for example. And there's no limit on how long we wait between a REQ and an ACK.

Here is an example set of waveforms. A REQ on cycle 2. Another REQ on cycle 4. So, two outstanding REQ on cycle 4. An ACK on cycle 5 for the first REQ from cycle 2. Another REQ on cycle 8. An ACK on cycle 9 for the REQ on cycle 4. A final ACK on cycle 12 for the REQ from cycle 8.

So, a correspondence of REQ to ACK three times in that set of waveforms. There must be a one-to-one correlation of the handshakes. There cannot be an ACK for no reason.

Again, in SVA, we have the same problem here. We cannot write a property which globally counts REQs and ACKs, and deals with the overlapping problem.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog

Auxiliary Counter-Based Model for REQ/ACK Verification

No way to write this in just SVA

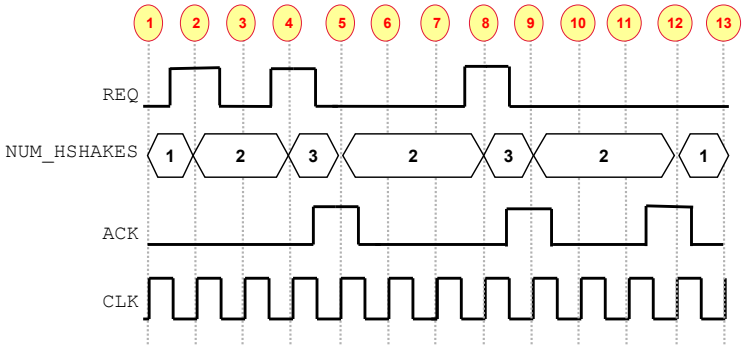
You need Aux code to do the *global counting*

Use the smallest counter size possible, not an integer (32 bits)

```
reg[2:0]  NUM_HSHAKES;
always @(posedge CLK, posedge RESET)
 begin
  if  (RESET)
    NUM_HSHAKES <= 1;
  else
   begin
    if (REQ & !ACK)
       NUM_HSHAKES<= NUM_HSHAKES+1;
    if (ACK & !REQ)
       NUM_HSHAKES<= NUM_HSHAKES-1;
   end
end
```

Start unsigned count at 1

AUX Code




```
LESS_THAN_6_REQS: assert property (
 @(posedge CLK)  ( NUM_HSHAKES !=7) );

NEVER_MORE_ACKS_THAN_REQS : assert property (
  @(posedge CLK) (NUM_HSHAKES != 0) );

NEVER_ACK_WITHOUT_PREV_REQ : assert property (
  @(posedge CLK)
   (REQ && ACK |=> NUM_HSHAKES != 1) );
```

? What would be good cover statements to write?

Question

```
C1: cover property (
  @(posedge CLK)
  (NUM_HSHAKES ==6) ##[+] (NUM_HSHAKES ==1) );
```

138 © Cadence Design Systems, Inc. All rights reserved.

cadence

NUM_HSHAKES is the number of outstanding handshakes. We count from 1 to avoid negative numbers. REQ and not ACK is an outstanding handshake, so we increment the count. An ACK and no REQ is an matched handshake, so we decrement. The spec allows a maximum of five outstanding transactions. Starting from 1, a count to 7 represents more outstanding requests than allowed.

The example waveform shows the counter. On cycle 2, there is a REQ and no ACK, so we increment to 2. A REQ and no ACK on cycle 4 increments to 3. The ACK and no REQ at cycle 5 decrements to 2. Cycle 8 REQ increments to 3, and the ACKs on cycles 9 and 12 decrement back to the initial value of 1.

The first assertion says counter should not equal 7, as that is 6 outstanding transactions (from 1). In the second assertion, NUM_HSHAKES cannot be 0, as that is more ACKs than REQs or ACK for no reason.

We have to consider REQ and ACK on the same cycle. We cannot acknowledge a request in the same cycle the request is raised. For REQ and ACK at the same cycle, the AK must be for an outstanding request, so the count cannot be 1, the initial value, which means no outstanding requests.

In simulation, our assertions would pass if we didn't actually do anything. Coverage can check the number of outstanding handshakes reaches the maximum and then goes back to zero. Remember [##+] means one to an infinite number of cycles. In formal, this cover shows two things. One, it's possible, which gives more confidence that the tool has checked every combination. And two, it shows the quickest way in which that can be done, which in itself might be revealing.

In formal, it is important we use the minimum counter size possible, otherwise that will increase state space. In this case, the maximum required count is 7, so the counter is declared as 3 bits.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog

Auxiliary Code to Implement One-Shot Properties



SVA will not provide semantics for one-shot properties, properties that must be checked only once, making it impossible to directly express this behavior in situations where PSL provides native support.

Requirement:

- From the beginning of verification, we require `A` to be high up to and including the cycle `B` occurs
- After `B` occurs, we don't care about what either `A` or `B` is doing.

Solution:

```
reg B_SEEN;
always @(posedge CLK or negedge RSTn)
  if (!RSTn)
     B_SEEN <= 0;
  else if (B)
     B_SEEN <= 1'b1;
```

```
ONESHOT_A_UNTIL_B: assert property (@(posedge CLK) ( !B_SEEN |-> A) );
```

139 © Cadence Design Systems, Inc. All rights reserved.

cadence

Another issue for SVA is one shot properties. SVA does not have the semantics to describe these.

The specification says that from the beginning of verification, we require A to be high up to and including the cycle where B occurs. Once B occurs, there are no further restrictions on A or B. So, we only need to check this behavior once at the beginning of verification.

We use auxiliary code to declare a flag called B_SEEN, which indicates whether B has occurred or not. We clear the flag at reset. We set the flag if B is true. Notice there's no way of clearing B_SEEN other than going to reset again. Once B_SEEN is set, it stays set. Then we can write an assertion which says if B has not been seen, this implies A is true in the same cycle. Once B goes high, B_SEEN is true all the time and the property will never be checked again.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog

Best Practices for Using SVA and Auxiliary Code



Use small auxiliary code rather than complex SVA property to make behavior easy to track, so assertions stay simple, clear, and easy to understand.

A common question for beginners to formal is

- "What is the point of SVA if I have to write more RTL?"
  - Aux code will just be more work
- Avoid writing overly complex SVA properties; they are difficult to understand, debug, and maintain.
- Use auxiliary code to simplify verification logic and prevent overlapping or ambiguous assertions.
- Prefer clear, modular auxiliary constructs (flags, counters, shift registers) instead of long, dense SVA sequences.
- Do not attempt to encode all behaviors in a single assertion; break checks into smaller, verifiable components.
- Rely on auxiliary code to improve formal tool efficiency and reduce state-space complexity.
- Treat SVA as a concise expression layer, not a replacement for synthesizable modeling where required.

 © Cadence Design Systems, Inc. All rights reserved.

cadence

So why do we need to write more RTL code to make SVA work? In SVA we can describe very complicated behaviour in very few characters, which is obviously a source of strength but also weakness. Remember, we need to debug our properties and be confident our assertions work as intended.

Two things can happen with a complicated property.

It fails and we can't work out why.

It passes and we are not sure whether it's checking what we intended it to.

Auxiliary code is really about efficient SVA coding.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog

Summary

Writing complex SVA is counter-productive

- It is hard for the tool to evaluate
- It is hard for other engineers to understand

Almost every problem will require the Aux code

- Allows the problem to be abstracted
- Does not create overlapping copies of itself
- More verbose but easy to understand
- Typically checks more behavior than the intended complex SVA
- Easier to write covers and even allows the tool to automatically infer covers

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog

Lab



Lab 3 Auxiliary Code

- In this lab, you will apply auxiliary code, assertions, covers and verification components to verify design, using formal verification tool.

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog



Module 9

Sequence Operations

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog

Module Objectives

After this module, you will be able to

- Explain reuse of named sequences to build complex SVA behavior
- Describe sequence clocking and clock inheritance
- Explain advanced sequence operators and features to simplify sequence and property definition in SVA
- Explain relational sequence operators `"or", "and",` and `"intersect"`
- Apply `throughout` and `within` operators to constrain sequence behavior over time
- Use `first_match` and `sequence.triggered` to control sequence activation and completion

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog

Simple Named Sequences

```
sequence NAME;
  (<sequence>);
endsequence

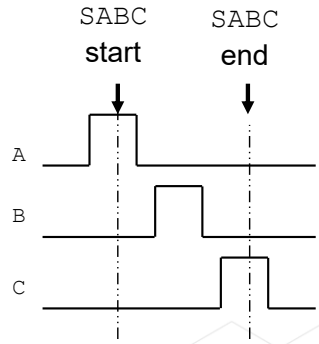
sequence NAME;
 (<expression>);
endsequence
```

```
sequence ENAB;
 (COUNT > 10);
endsequence
sequence SABC;
  (A ##1 B ##1 C);
endsequence
property PS;
    ENAB |=> SABC;
endproperty
```

- Common sequences or simple Boolean conditions can be defined once using a unique name.
- These named sequences can be reused as building blocks to construct more complex sequences and properties.
- Sequences can be parameterized to make them flexible and reusable across different scenarios.

In this example

- Sequence `SABC` defines a sequence of `A` followed by `B` followed by `C`
- Sequence `ENAB` is a boolean condition, a sequence of length 1 cycle



 © Cadence Design Systems, Inc. All rights reserved.

A fundamental approach to describing complex assertions is to break up complex properties into smaller sequences, define these separately as named sequences, and build up a definition for the complex sequence using these as building blocks. The sequences are easier to understand, require fewer lines of code and can be used in more than one complex sequence.

Named sequences can be considered as “subprograms” for sequence definition. They can even have arguments (parameters) (see section on Advanced Features).

Sequences are usually declared without any temporal condition (clocking). They inherit the clocking from the context in which they are used.

Property PS states that if COUNT is greater than 10 (the 1 cycle long sequence ENAB has completed), then from the next cycle the 3 cycle sequence `A ##1 B ##1 C` must start and subsequently complete.

There is no explicit clocking shown for this property. We assume there is default clocking statement in the scope of PS.

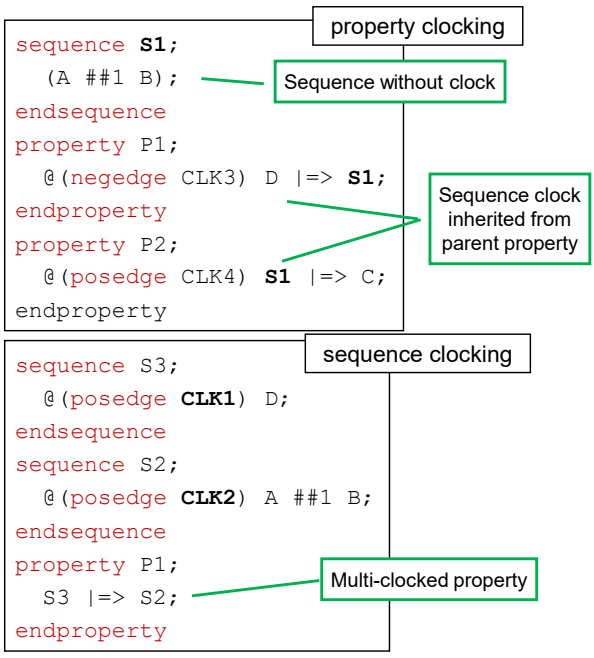
The example waveform which would satisfy the sequence SABC. Also, A, B and C always high for every cycle would also satisfy SABC.

Remember a sequence of length one cycle is the same as a Boolean, because a sequence is Booleans separated by some number of cycles.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog

Sequence Clocking and Clock Inheritance in SVA



Sequence declarations do not normally contain clocks

- Clocking is inherited from the property or assertion where the sequence was used
- Allows different clocking for same sequence

Unclocked properties are not allowed

Sequences can contain clocks

- Property can inherit clock from sequences

Sequence and property clock can differ

- Multiclock properties are difficult to understand



Multi-clocked property

Caution

 © Cadence Design Systems, Inc. All rights reserved.

cadence

Sequences are usually declared without any temporal condition (clocking). They inherit the clocking from the context in which they are used.

The sequence S1 is declared without a clock. When S1 is used in property P1, it inherits the property clock, so the ##1 in S1 is the negedge of CLK3. When S1 is used in property P2, it inherits the clock from P2, so now ##1 is the posedge of CLK4.

Sequences can have their own clock. However, if a property where a sequence is used also specifies a clock which is different, then it is a multi-clocked property. Rules for the evaluation of such properties are complex.

The slide shows examples of sequences with clocks, but it is very rare to do this.

Sequence S3 is using the posedge of CLK1. Sequence S2 is using the posedge of CLK2. When we use both sequences in the property P1 underneath, this creates a multi-clock property. Multi-clock properties are hard to understand. Once the sequence S3 is completed, evaluated on the posedge of CLK1, the next cycle implication means we are waiting for the next posedge of CLK2, the clocking event of S2.

You will see other details of multi-clock properties later in this course.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog

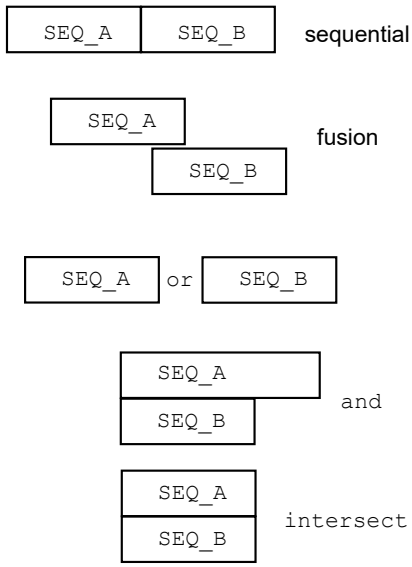
Sequential Relationship Operators in SVA

Cycle delay ## allows sequential sequences

- Also permits fusion where sequences overlap by 1 cycle (##0)

Use these operators to define alternative and concurrent sequences

- or
  - 1 or more alternative sequences hold
- and
  - Concurrent sequences start, but need not complete in the same cycle
  - Non-length matching
- intersect
  - Concurrent sequences start and must complete in same cycle
  - Length matching



147 © Cadence Design Systems, Inc. All rights reserved.

So far, the cycle delay operator ## has allowed us to define serial or fused sequences.

There are also operators which form relationships with other sequences. So, you can describe how two sequences are, or should interact or are related.

An or is an alternative of one sequence or the other, or both. This is a sequence or rather than a logical or.

and means two sequences start at the same cycle, but do not need to end at the same time necessarily. This is called non-length matching as the length of both sequences does not need to match.

intersect means two sequences start at the same cycle, and they must both end together at some future cycle together. This is called length matching as the length of both sequences must match.

So, the keywords are and, intersect and or. There is no keyword fusion or sequential.

All these help in creating complex properties.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog

Sequence Fusion

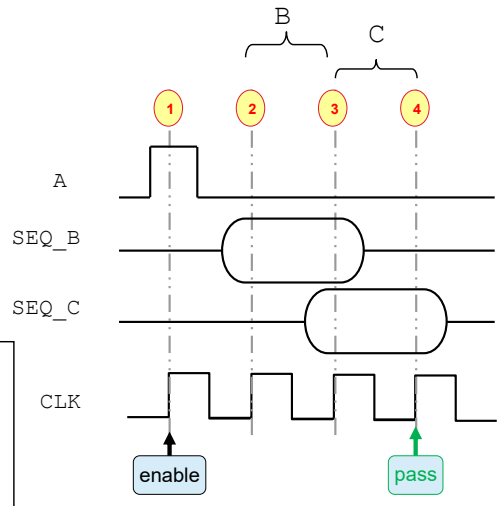
```
SEQ_B ##0 SEQ_C
```

Where sequences overlap, they can be defined with zero cycle delay

`SEQ_C` starts in the same cycle as `SEQ_B` completes

Extremely useful where `SEQ_B` is of variable or unknown length

```
sequence SEQ_B;
  ...
endsequence
sequence SEQ_C;
  ...
endsequence
property SEQ_FUSION;
  @(posedge CLK)
    A |=> SEQ_B ##0 SEQ_C;
endproperty
```



148 © Cadence Design Systems, Inc. All rights reserved.

cadence

Fusion is a cycle delay ##0. This is overlapping.

In the example, SEQ_B lasts for two cycles and SEQ_C also lasts for two cycles. It doesn't matter how long the sequences are. The property says if we have A, then on the next cycle, we require SEQ_B to start and complete. In the final cycle of SEQ_B, SEQ_C must start. ##0 means the sequences overlap by one cycle. Explicitly, the final term of SEQ_B is true in the same cycle as the first term of SEQ_C.

The example waveform shows this property passing. SEQ_B completes in cycle 3 and SEQ_C starts also in cycle 3.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog

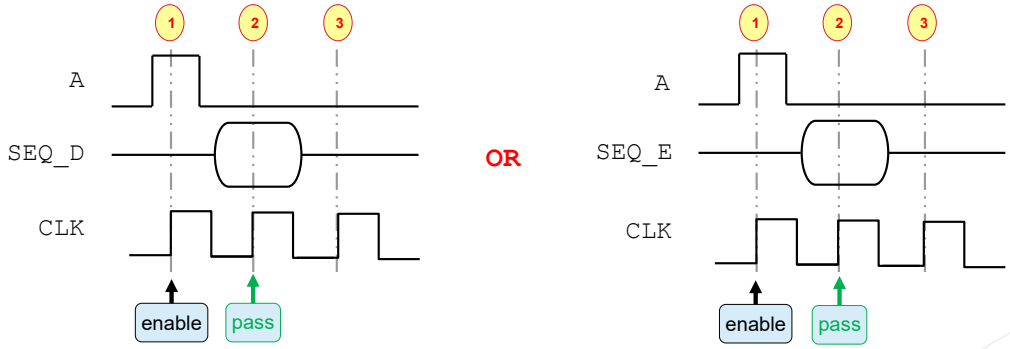
Sequence `or`

```
(SEQ_D) or (SEQ_E)
```

```
property SEQ_OR;
  @(posedge CLK) (A) |=> ((SEQ_D) or (SEQ_E));
endproperty
```

Defines alternative sequences

Either `SEQ_D` or `SEQ_E` holds at the current cycle



149 © Cadence Design Systems, Inc. All rights reserved.

Sequence or is one alternative or the other of a sequence.

So, in this example, SEQ_D and SEQ_E happen to be sequences of length one cycle, to save space on the slide, but they could be of any length. If we have A occurring, then from the next cycle either SEQ_D or SEQ_E must start and complete. It doesn't matter which one, or if both of them complete, but one of the two alternatives must complete for the property to pass. The sequences are not mutually exclusive. Either one or the other or both.

This is a sequence or, rather than logical or. You can't use keyword or in a Boolean expression in SystemVerilog.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog

Sequence and

```
(SEQ_F) and (SEQ_G)
```

Defines concurrent sequences

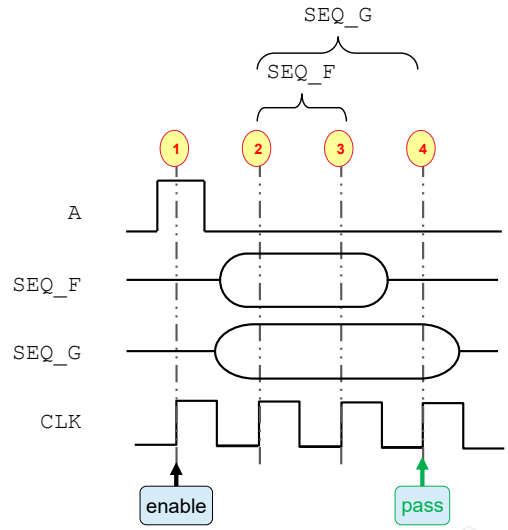
Sequences to start at the same time

Sequences may finish in *different* cycles

- Non-length matching

`SEQ_F` and `SEQ_G` start in the same cycle, but can end in different cycles

```
property S11;
   @(posedge CLK)
      (A) |=> (SEQ_F and SEQ_G);
endproperty
```



150 © Cadence Design Systems, Inc. All rights reserved.

cadence

Sequence and is non-length matching.

For property S11, if we have A occurring on cycle 1, then on cycle 2, SEQ_F and SEQ_G must both start. They can, but don't necessarily need to, complete in the same cycle. As long as both sequences complete, the property passes. In the example waveform, SEQ_F lasts for two cycles and SEQ_G lasts for three cycles. They both start on cycle 2, as required , and run to completion. The property passes on cycle 4 when SEQ_G completes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog

Sequence `intersect`

```
(SEQ_J) intersect (SEQ_K)
```

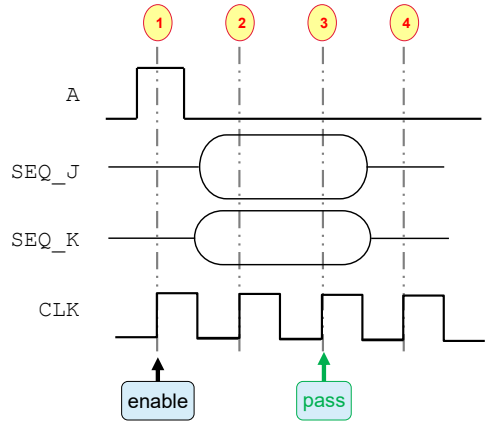
Sequences must start *and* finish in the same cycle

- Length matching `and`

`SEQ_J` and `SEQ_K` start in the same cycle and end in the same cycle

- Useful for placing cycle limits on otherwise "open-ended" sequences

```
property S12;
   @(posedge CLK)
      (A) |=> (SEQ_J intersect SEQ_K);
endproperty
```



151 © Cadence Design Systems, Inc. All rights reserved.

Sequence intersect is length matching.

For property S11, if we have A occurring on cycle 1, then on cycle 2, SEQ_J and SEQ_K must both start. They must complete in the same cycle for the property to pass. In the example waveform, both sequences last for two cycles. They both start on cycle 2, as required , and both complete on cycle 3, as required. So, the property passes at cycle 3.

intersect is useful for placing a limit on a sequence which otherwise will be open-ended. We can intersect a sequence of variable length or unknown length, with one of a known fixed length,, as we shall see in subsequent slides.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog

Examples: Sequence Operator

1. From EN1, DPULSE must occur between 4 and 8 times over the next 15 cycles.

```
EN1 |=> (DPULSE[=4:8]) intersect (1'b1[*15])
```

2. If START occurs, then on the next POP either TAG is 2 or, the BUSY signal is turned on and when it is turned off, the signal SKIP is on.

```
START |=>
   POP[->1] ##0 (TAG==2) or (BUSY[*1:$] ##1 (!BUSY && SKIP));
```

152 © Cadence Design Systems, Inc. All rights reserved.

A default clock must be declared.

If EN1 is true, then from the next cycle DPULSE must occur between four and eight times over the next 15 cycles. Nonconsecutive repetition is used to look for 4 to 8 occurrences of DPULSE. Remember nonconsecutive repetition is open-ended. There can be any number of cycles before DPULSE is true and any number of cycles between each occurrence of DPULSE. There is normally no limit on the duration of that repetition, but here we're intersecting it with 1'b1, which is always true, for 15 consecutive cycles So the nonconsecutive repetition is limited to a cycle count of 15 using length matching intersect.

If START is true, then look for a rising edge on POP. On the same cycle as POP is true, ##0, we check that either TAG=2 or BUSY is turned on and when it's turned off, the signal SKIP is on. So, SKIP must be true in the same cycle as BUSY is false. Note that POP[->1] and BUSY[*1:$] are both open-ended sequence terms of potentially infinite length. So, we don't require POP to occur or BUSY to go low.

The second property is a good example of what not to write in formal because the open-ended sequence terms create a huge state space for the tool to analyse There are better ways of describing it in formal. In the Jasper Formal Fundamentals course, the first section covers SVA formal friendly coding which covers issues like this. In simulation, the property will work fine. However, it may still be difficult to understand, to know it's working correctly, to debug or to maintain.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog

Sequence throughout

```
(expr1) throughout (SEQA);
```

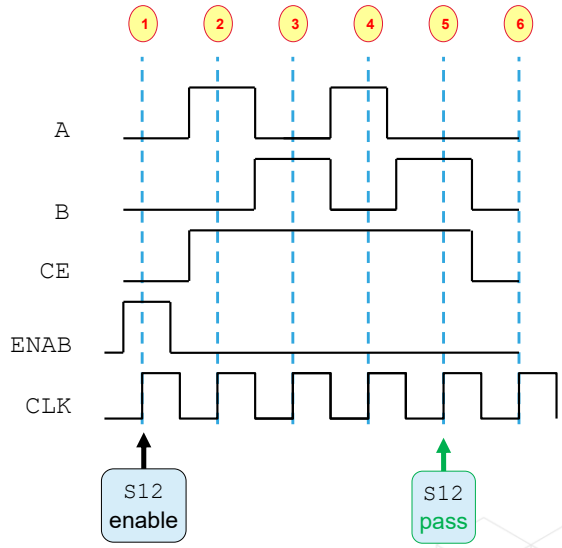
Defines that a Boolean expression is true for the duration of a sequence

- `expr1` is true on every cycle of `SEQA`
- Equivalent to:

```
(expr1)[*0:$] intersect (SEQA)
```

```
sequence SAB;
  (A ##1 B);
Endsequence

property S12;
  @(posedge CLK)
    ENAB |=> (CE) throughout (SAB[*2]);
endproperty
```



153 © Cadence Design Systems, Inc. All rights reserved.

cadence

throughout defines that the left-hand Boolean operand (expr1) must be true for every cycle of the right-hand sequence operand (SEQA).

The LRM defines throughout as equivalent to the Boolean expression repeated 0 to an infinite number of times, length matched with the sequence.

Property S12 says that if ENAB is true, then from the next cycle, CE must be true for 2 repetitions of the sequence SAB, which is A followed by B.

So, in the waveform, S12 is enabled by ENAB at cycle 1. CE is high from cycle 2, and stays high for 2 repetitions of SAB. A at cycle 2 and B at cycle 3 is the first repetition of SAB, and cycles 3-4 are the second. So S12 passes at cycle 5.

We saw some examples of throughout in the previous coverage module.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog

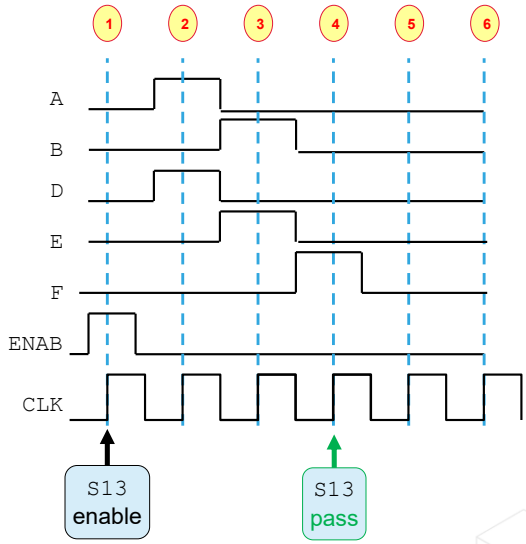
Sequence `within`

```
(SEQA) within (SEQB)
```

Defines that one sequence is enclosed by another

- `SEQA` begins at or after `SEQB` begins
- `SEQA` completes at or before `SEQB` ends

```
sequence SAB;
  (A ##1 B);
endsequence
sequence SDEF;
  (D ##1 E ##1  F);
endsequence
property S13;
  @(posedge CLK)
  ENAB |=> (SAB) within (SDEF);
endproperty
```



154 © Cadence Design Systems, Inc. All rights reserved.

cadence

Within requires that the left-hand sequence operand (SEQA) is contained within the bounds of the right-hand sequence operand (SEQB). So SEQA is enclosed by SEQB. The resulting sequence begins and ends with SEQB. SEQA must start at or after the cycle in which SEQB starts. SEQA must complete before or in the same cycle as SEQB completes.

within is equivalent to:

```
(1[*0:$] ##1 SEQA ##1 1[*0:$]) intersect (SEQB)
```

i.e. a sequence composed of any number of cycles (including zero) followed by SEQA followed in turn by any number of cycles (including zero) is length matched (intersect) with SEQB.

Sequence SAB is a two-cycle sequence. Sequence SDEF is a three-cycle sequence. Property S13 says if ENAB is true which is at cycle 1, then from the next cycle sequence SAB is contained within sequence SDEF. So SDEF starts at cycle 2 and ends at cycle 4. Within cycles 2 to 4, we must see sequence SAB. A occurs at cycle 2 and B at cycle 3, so we have SAB within SDEF and S13, which gets enabled at cycle 1, passes in cycle 4.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog

`within` Example: Ack Response Inside a Window

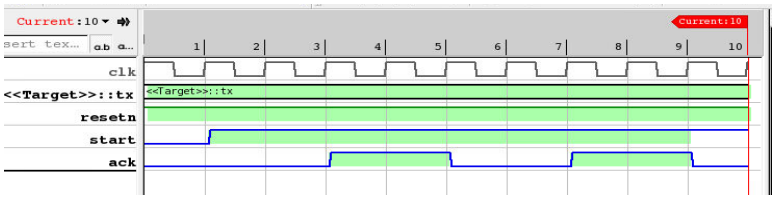
Requirement:

- If I observe the rise of `start` then I must observe between 1 and 3 `ack`'s within a 10-cycle window
- This 10-cycle window begins in the same cycle in which I detect a rise of `start`
- The `ack`'s need to occur in *consecutive* cycles but at any time in the 10-cycle window

```
S_ACK_WIN_10: assert property (
           @(posedge clk) $rose(start) |-> ack[*1:3] within 1'b1[*10] );
```

Looks like a sensible property at first glance.

However, if I ask Jasper® to cover the property (`witness`), then this is what I see:



It is not expected design behavior to see 4 `acks` in the 10-cycle window.

Why does Jasper show this design behavior?

 © Cadence Design Systems, Inc. All rights reserved.

cadence

within looks straightforward, but it can give unexpected results.

In property S_ACK_WIN, if we observe the rise of start, then there must be between 1 and 3 acks within a 10-cycle window. The 10-cycle window begins in the same cycle where we detect the rise of start. The 1 to 3 acks need to be in consecutive cycles, but they can happen any time in the 10 cycles defined by 1'b[*10]. Given what we've learned already, this property seems like a reasonable thing to write, but if we were to observe a witness of that property in Jasper, we would see something unexpected.

The witness shows the rise of start detected at cycle 2. We have 2 consecutive acks at cycles 4 and 5, but we have 2 more consecutive acks at cycles 8 and 9. Both of those blocks of ack are within the 10-cycle window. Why does Jasper show us this waveform as a solution to the property?

By default, Jasper will cover the left-hand side of implication only, namely $rose(start) in our S_ACK_WIN_10.

If we wish to observe a cover of the entire property passing, known as a "witness," then we need to use the elaborate command option:

```
elaborate -create_related_covers witness
```

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog

Why the `within` Property Doesn't Work?

These 2 properties are equivalent:

```
S_ACK_WIN_10: assert property (
  @(posedge clk) $rose(start) |-> ack[*1:3] within 1'b1[*10] );
```

```
S_ACK_WIN_10_EQUIV: assert property (
  @(posedge clk) $rose(start) |-> (1[*0:$] ##1 ack[*1:3] ##1 1[*0:$]) intersect 1'b1[*10] );
```

Hence:

- A single `ack` anywhere in the window satisfies all requirements
- `ack` can have any other behavior in the remaining 9 cycles of the window

Easy to make mistakes while trying to understand `intersect` and `within`

Auxiliary code is easier to understand for "Count occurrences in a window" type properties

- It's also easier for the tool

 © Cadence Design Systems, Inc. All rights reserved.

cadence

So, within is not quite as straightforward as it looks.

The LRM states these two properties are equivalent. So, S_ACK_WIN_10 is equivalent to a consecutive repetition of ack between 1 and 3 cycles, sandwiched between true from zero to an infinite number of cycle on either other side, which is then intersected with that 10 cycle window. As the minimum repetition bound is one ack, a single ack occurring anywhere in the 10 cycles window satisfies all property requirements. ack can have any other behaviour in the remaining 9 cycles. Hence the appearance of a second block of ack in the Jasper witness.

intersect, within and most of the sequence operators can be difficult to use. It is easy to make incorrect assumptions about their behaviour. Auxiliary code and simple sequences can be easier to understand than remembering the intricacies of the operators. If you know how to define a sequence, you can define the required behaviour without using these operators. So often as well in formal, especially, the auxiliary code is much easier to understand and more efficient to analyse.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog

Limiting Event Occurrences in a Time Window



The requirement is to ensure no more than three occurrences of signal B within any 10-cycle window.

The initial attempt to implement this requirement :

```
A1: assert property (not strong( (1 [*10]) intersect (B[=4]) ) );
```

Property `A1` meets the requirement, but …

- Is not particularly easy to conceive or understand – Why do we need `strong`?
- Is not that easy for the tool to evaluate

 © Cadence Design Systems, Inc. All rights reserved.

cadence

So, let's take a closer look at the example placing a limit on the number of occurrences of an expression within a specific cycle window.

This is not the same requirement as the previous slide - this is simplified for better understanding.

This requirement is for no more than three occurrences of B in any 10-cycle window.

Property A1 meets the requirement by intersecting a 10-cycle window with 4 non-consecutive repetitions of B, which is one more than the maximum allowed. We invert the intersect because we should not observe 4 occurrences of B and we use strong to ensure both sequences complete. So, the inversion and strong do not make this an intuitive solution to the problem. And this property not easy to evaluate in formal.

© Cadence Design Systems, Inc. All rights reserved.

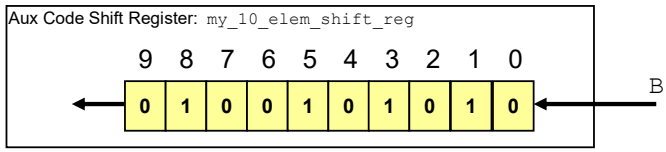
This document is for the sole use of Rajat G of Analog

Simplified Verification Approach Using AUX Code

Remodel with a simple shift register and a simple Boolean property

Approach

- Shift the value of B into a 10-bit register every clock cycle.
- Use $countones(my_10_elem_shift_reg) to count how many times B is high in the past 10 cycles.
- Assert that this count must never equal 4.




```
A2: assert property ($countones(my_10_elem_shift_reg) != 4);
```

158 © Cadence Design Systems, Inc. All rights reserved.

It is simpler to remodel the verification problem using auxiliary code. All we need is a simple 10 element shift register. We shift B in on every cycle and write a simple property to check that the number of ones in the shift register should not equal 4. As we are shifting B in every cycle, 4 Bs over the 10 cycles stored in the shift register would be too many. So, a simple shift register and a simple Boolean property meet the requirements and is more efficient in both formal and simulation.

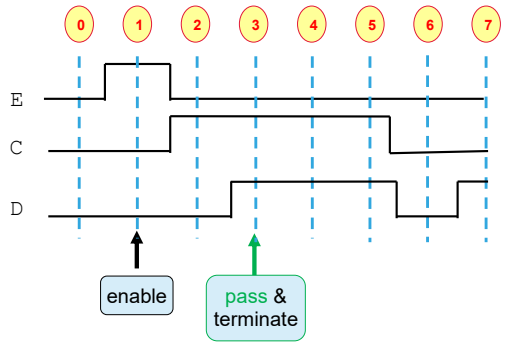
© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog

first_match() Operator

```
first_match(SEQ_A)
```

If you think `first_match()` is the solution, then you probably lack understanding of the requirements




```
sequence SEQ_CD;
  C[*1:3] ##1 D;
endsequence
property SEQ_RHS;
  E |=> SEQ_CD;
endproperty
property SEQ_RHS_EQUIV;
  E |=> first_match(SEQ_CD);
endproperty
```

There is an implicit `first_match` of any sequence on **RHS** of an implication operator. Hence, both properties are equivalent.

TB: SVA first_match Operator: Why Doesn't PSL Have One?

159 © Cadence Design Systems, Inc. All rights reserved.
cadence

If you think first_match() looks like the solution, it's probably a good indicator that you're doing something wrong.

first_match() implies we look for the first occurrence of something.

We will first analyse first_match() use on the right-hand side of an implication operator, before looking at use on the left-hand side.

SEQ_CD is a variable length sequence of between 1 and 3 occurrences of C followed by D. Property SEQ_RHS says E implies from the next cycle, SEQ_CD. In the waveform, E occurs at cycle 1 and the first C at cycle 2 and then D at cycle 3. On cycle 3, property SEQ_RHS passes and becomes inactive. We have a conclusive result as the behaviour we described has been observed. Even though we see a second C at cycle 3, followed by D at cycle 4, and the third C at cycle 4, with D at cycle 5, these are not additional passes. Once we have a result we do not keep looking for all the other times it might pass.

Property SEQ_RHS_EQUIV behaves exactly the same way as SEQ_RHS. There is an implicit first_match() on the right-hand side of every implication operator.

If you are interested in more details on first_match() please select the link or search for the Training Byte title.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog

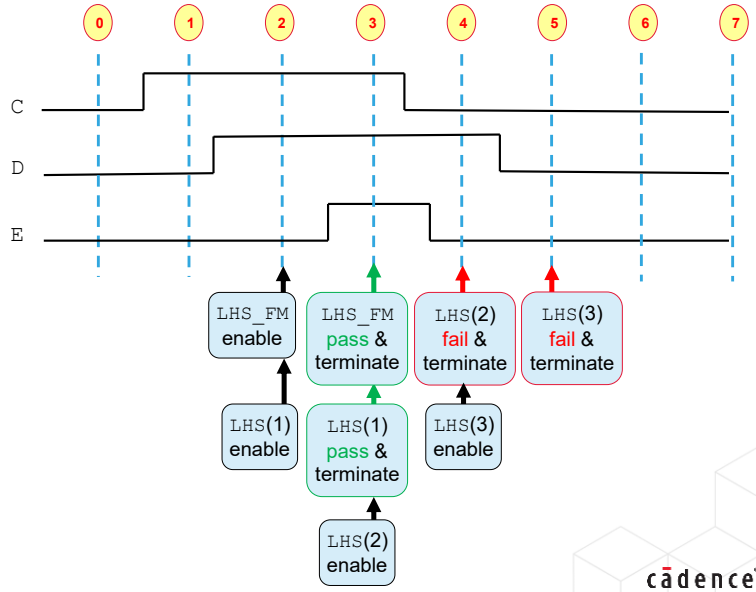
`first_match()` Operator: Removing Undesired Failures

`first_match()` is only relevant when used on **LHS** of implication operator

```
sequence SEQ_CD;
  C[*1:3] ##1 D;
endsequence

property LHS_FM;
 first_match(SEQ_CD) |=> E;
endproperty

property LHS;
  SEQ_CD |=> E;
endproperty
```



160 © Cadence Design Systems, Inc. All rights reserved.

Here's a typical example where people try to use first_match(). Here we have swapped the terms on either side of the implication operator, so SEQ_CD is now the enabling sequence, with first_match() for property LHS_FM.

Let's examine property LHS first. Copy 1 of LHS becomes active at cycle 1 when C is high, enabled by the D at cycle 2, and passes with the E at cycle 3. However, at cycle 2, C remains high. Two Cs still match the variable length consecutive repetition of SEQ_CD, so copy 2 of LHS becomes active. LHS copy 2 is enabled by D at cycle 3, but fails at cycle 4 as E is low. C is still high at cycle 3. This third C still matches C[*1:3], so copy 3 of LHS becomes active at cycle 3, enabled by D at cycle 4 and fails due to a lack of E at cycle 5. So, because we had a variable length consecutive repetition as the first term of the enabling sequence SEQ_CD, all 3 possibilities were checked, which gave us 3 copies with 1 pass and 2 failures for this waveform. So, we have a result for every possible match we could make for the sequence. There is no implicit first_match() on the left-hand side of the implication operator.

However, using first_march() in property LHS_FM means as soon as we get the first match of SEQ_CD, which is C at cycle 1 and D at cycle 2, we do not check for a second C at cycle 2 or a third at cycle 3. So only one copy of LHS_FM is enabled at cycle 2 and passes at cycle 3.

If we did not have D at cycle 2, only then would we match the second C at cycle 2 for the evaluation of SEQ_CD. If there was no D at cycle 3, we would match the third C at cycle 3.

So first_match() may have a use on the left-hand side of an implication operator, but only in specific circumstances, such as for variable length sequences. We can usually avoid such issues by careful definition of our sequences.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog

Sequence `triggered` Operator

```
SEQ_A.triggered
```

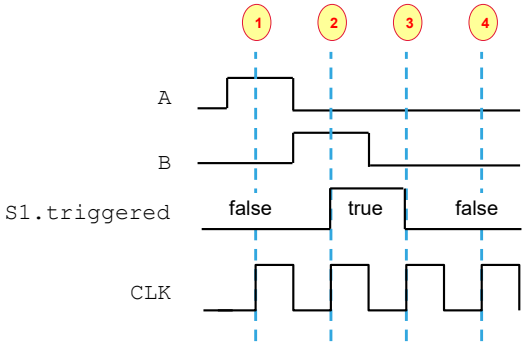
Useful to detect when a sequence completes

- Called the endpoint

Sequence method `triggered` returns true at sequence endpoint

- Remains true until simulation time advances

```
sequence S1;
  @(posedge CLK) (A ##1 B);
endsequence
```



 © Cadence Design Systems, Inc. All rights reserved.

Every sequence has a built-in method called triggered, and we can use this to capture the endpoint of a sequence. triggered returns true whenever the sequence completes, and that depends on its clock.

The triggered method evaluates to true if the given sequence has reached its endpoint in the current time step, and false otherwise. The triggered status of a sequence is set during the Observe region of the simulation event scheduler and persists through the remainder of the time step (i.e., until simulation time advances).

Sequence S1 is A followed by B. S1.triggered will be true in any cycle where S1 completes. It will be true until the next time we evaluate that sequence, which will be the next posedge clk for S1. So, for A at cycle 1 and B at cycle 2, S1.triggered is true at cycle 2 until the next posedge clk, at cycle 3.

If A and B are high all the time, then S1.triggered will be true for every cycle.

In IEEE 1800-2005 there was another sequence method called ended which was true only in the observed timeslot of the endpoint of the sequence. This meant it could not be used in an assertion disable or procedural wait. This made its usefulness limited, hence ended was deprecated in the IEEE 1800-2009 standard. So, sequence method ended does not exist in the latest SystemVerilog standard, although you may see it used in legacy code.

Many tool vendors continue to support ended for backward compatibility.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog

Example: Sequence `triggered` Operator

Method `triggered`

- True from observe timeslot until simulation time advances
- Can be used in an assertion
- Can be used to disable an assertion
- Can be used in procedural code

There is also a `sequence.matched()` method, which allows you to synchronize sequences with different clocks

- Refer to the LRM for details

```
`define true 1
sequence EP2;
  (req ##1 ack ##2 `true);
endsequence

property SEQ;
  @(negedge clk) $rose(A) |=> EP2.triggered;
endproperty

property SEQ_DIS;
  @(negedge clk) disable iff (EP2.triggered)
  ...
endproperty

always @(posedge clk)
  begin
  wait(EP2.triggered);
  $display("2 cycles after handshake");
  end
```

162 © Cadence Design Systems, Inc. All rights reserved.

cadence

We can use triggered in a variety of places. Remember it is just a Boolean.

We can use triggered in a property. In SEQ, if we see rose(A), then from the next cycle EP2 must have triggered. Sequence EP2 lasts for 4 cycles, so we have to be evaluating EP2 before property SEQ becomes enabled, 2 cycles before rose(A).

We can use triggered in the disable iff clause of a property.

We can even use it in normal procedural code to wait until some particular sequence is finished, for example.

There are also other sequence methods, which are not widely used. Please refer to the Language Reference Manual for details.

Note that sequences can be extended so that the endpoint is true in later cycles (see sequence EP2 above).

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog

Using Sequence Endpoints

An endpoint can be used in a sequence

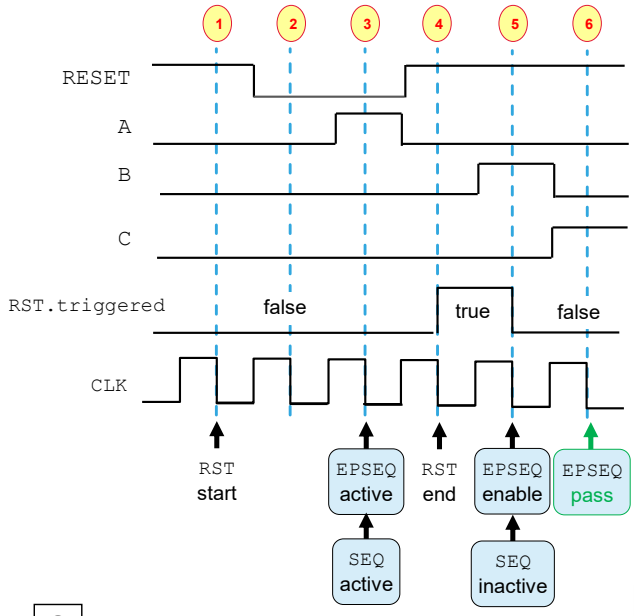
An endpoint is not equivalent to the sequence of its definition

- In `EPSEQ`, `RESET` will be looked for before `A`
- In `SEQ`, the `RESET` will only be checked after `A`

```
sequence RST;
  (RESET ##1 !RESET[*2] ##1 RESET);
endsequence

property EPSEQ;
  @(negedge CLK)
    ((A ##1 RST.triggered ##1 B) |=> (C));
endproperty

property SEQ;
  @(negedge CLK)
    ((A ##1 RST ##1 B) |=> (C));
endproperty
```



Question

? When does the `SEQ` enabling sequence become incomplete?

163 © Cadence Design Systems, Inc. All rights reserved.

cadence

Sequence RST is RESET high, followed by RESET low for 2 cycles, followed by RESET high.

Property EPSEQ uses the triggered method for RST. If we see A followed by the end of sequence RST, followed by B, then we expect C on the next cycle.

Therefore, the RST sequence must run in parallel with A. In fact, because the RST sequence is of fixed length, we can fully expand EPSEQ as follows:

```
(RESET ##1 ~RESET ##1 ~RESET && A ##1 RESET ##1 B ##1 C)
```

In property SEQ, the RST sequence must begin after A and be followed by B. So, the expansion is:

```
(A ##1 RESET ##1 ~RESET ##1 ~RESET ##1 RESET ##1 B ##1 C)
```

In the waveform, RST starts on cycle 1 and completes on cycle 4, where RST.triggered is true.

So EPSEQ becomes active at cycle 3 with A, sees RST.triggered at cycle 4, is enabled by B at cycle 5 and passes with C at cycle 6. So, RST.triggered was evaluated from cycle 1, before EPSEQ was even active at cycle 3. triggered methods need to be evaluated on every cycle regardless of whether the property in which they are used is being evaluated.

The enabling sequence of SEQ never occurs. It gets cancelled on cycle 5 because RESET is not low.

This shows that an endpoint is not a replacement of the sequence which defines the endpoint.

Endpoint can be useful when describing multiple concurrent or open-ended sequences. Separate sequences can be individually defined and named, and a property declared to check endpoints in the correct order.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog

Quiz



1. Write sequences to describe the following properties:
   a. 4 non-consecutive occurrences of `PULSE` in 10 cycles
   b. `GO` is followed by either `ERR` or 3 to 7 consecutive cycles of `ENB`

Solution:

a. 4 non-consecutive occurrences of `PULSE` in 10 cycles

```
(PULSE[=4]) intersect (1'b1[*10])
```

b. `GO` is followed by either `ERR` or 3 to 7 consecutive cycles of `ENB`

```
  GO ##1 (ERR) or (ENB[*3:7])
```

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog

Quiz (continued)



2. Write an assertion for the following specification:

A sequence beginning with `START`, followed by 6 **consecutive** occurrences of `PUSH`, during which `KILL` does not occur, must be followed by 6 **nonconsecutive** occurrences of `POP` before `STOP` can occur.

Solution:

```
property FRAME;
  @(posedge CLK)
  START ##1 (PUSH && !KILL)[*6] |=> !STOP throughout POP[->6];
endproperty
A1 : assert property (FRAME);
```

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog

Summary

- Named sequences let you define small, reusable SVA behaviors that can be combined to build clear and complex properties.
- Sequences typically have no clock of their own and inherit clocking from the property or assertion where they are used.
- Sequence operators:
  - `or`: Either sequence can satisfy the behavior.
  - `and`: Sequences start together, end independently.
  - `##0`: Second sequence starts in the same cycle as the first ends.
  - `intersect`: Sequences start and end together.
  - `throughout`: Condition must hold for every cycle of the sequence.
  - `within`: One sequence must fully occur inside another.
  - `first_match`: Uses only the earliest matching occurrence of a sequence.
  - `triggered`: Indicates that a sequence has successfully completed.

 © Cadence Design Systems, Inc. All rights reserved.

Let’s conclude this module. We started by understanding that named sequences provide a structured way to build complex assertions by breaking them into smaller, reusable components. We also discussed that sequences typically do not have their own clock and instead inherit clocking from the property or assertion where they are used, allowing flexibility across different verification scenarios. We then explored important sequence operators such as or, which allows alternative behaviors, and, which enables concurrent start with independent completion,##0 for overlapping sequences, and intersect, which enforces strict alignment of sequence execution. In addition, we looked at operators like throughout, which ensures a condition holds for the entire duration of a sequence, and within, which constrains one sequence to occur entirely inside another. Finally, we covered advanced features such as first_match, which considers the earliest valid sequence occurrence, and triggered, which helps detect when a sequence has completed. Together, these concepts enable us to write concise, expressive, and effective assertions for modeling complex hardware behavior.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog

Lab



Lab 4 Serial Peripheral Interface Verification

- In this lab, you will analyze and verify transactions on an SPI bus variant, using SVA constructs.

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog



Module 10

Advanced SVA Features

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog

Module Objective

After this module, you will be able to

- Analyze how assertions are evaluated across SystemVerilog simulation scheduling regions to avoid race conditions
- Implement action blocks to execute verification-only logic on assertion pass or failure
- Construct assertions using local variables for deserialization and dynamic data tracking scenarios
- Apply conditional property clocking to evaluate assertions only under valid conditions

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog

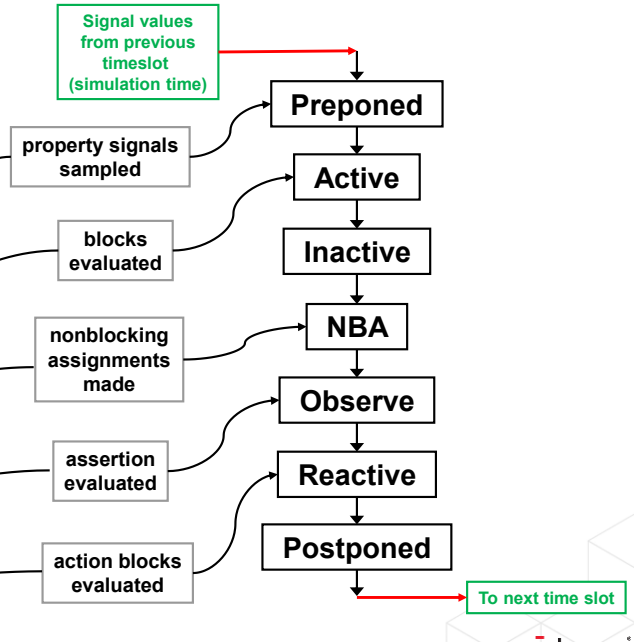
How Do SystemVerilog Scheduling Regions Control Assertion Evaluation?

SystemVerilog assertions rely on the simulation scheduling mechanism to ensure predictable and race-free evaluation.

A timeslot is a unique simulation time. Each SystemVerilog simulation timeslot is divided into a series of scheduling regions.

- Hence, all regions are executed sequentially within the same timeslot.

```
property PIPE;
  @(posedge clk)
    ip -> ##2 op;
endproperty: PIPE
always @(posedge clk)
  temp <= ip;
always @(posedge clk)
  op <= temp;
A_PIPE : assert property (PIPE)
    $display("pipe passed");
  else
    $warning("pipe failed");
```



170 © Cadence Design Systems, Inc. All rights reserved.

cadence

This is a simplified description of the time slots within a SystemVerilog simulation timeslot. There are many more additional scheduling regions between the ones listed above, primarily for PLI callback support.

The execution of SystemVerilog code at a specific point of simulation time is strictly defined as a series of scheduling regions.

In the preponed region, all property signals are sampled. Active executes always and initial blocks until suspension, and updates blocking assignments. Nonblocking assignments executed during Active are updated during NBA. It is possible that NBA updates can trigger the further execution of always or initial blocks, if they appear in event expressions of a block. This is known as a delta cycle. So there is a feedback path (not shown in the diagram) from NBA to Active. It may take multiple delta cycles (Active -> NBA) until the design reaches a steady state. Then in Observe we evaluate assertions triggered by e.g. a posedge of clk made in NBA ,using the signal samples from Preponed. In Reactive, we execute any action blocks triggered by passing or failing assertions. So assertion evaluation may be triggered by a rising edge clock assignment from NBA and will use signal values from Preponed, i.e. signal values from before the clock edge. These Preponed signal values may be different than the current values in Observe if the signals have been updated in Active or NBA.

The key advantage for assertions is that they are evaluated at the end of a simulation step, after all assignments have been scheduled, thus avoiding race conditions.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog

Action Blocks for Concurrent Assertions



Action blocks can be used with concurrent assertions

- Also, in immediate assertions

Statements executed upon assertion pass or fail

- `else` block executed on assertion failure
- Pass block can be omitted

Severity levels can also be attached

```
property HS;
  @(posedge CLK) REQ ##1 ~ACK |-> BUSY;
endproperty

A1 : assert property (HS)
       // pass block
       $display("A1 passed");
     else
       // fail block
       begin
         $warning("A1 failed);
         err_count++;
         -> ack_err_event;
       end
```



Action block code ignored by synthesis tools

Caution

171 © Cadence Design Systems, Inc. All rights reserved.

cadence

Actions blocks allow code to perform additional operations on the pass or failure of an assertion.

Note that the else clause is executed on the failure of the assertion, therefore it is common to omit the pass block and just include the failure else clause.

An action block can be any of the following:

- Nothing (omitted)
- Pass statements only
- Fail statements only (after else)
- Both pass and fail statements

WARNING: since immediate assertions are ignored by synthesis tools, code in the action block will not be synthesized. Therefore, verification code only should be placed in the action blocks.

The optional assertion label can be accessed via the %m format specifier.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog

Local Variables for De-Serialization

Local variables are dynamically created variables inside sequences or properties that hold intermediate values during a single assertion evaluation.

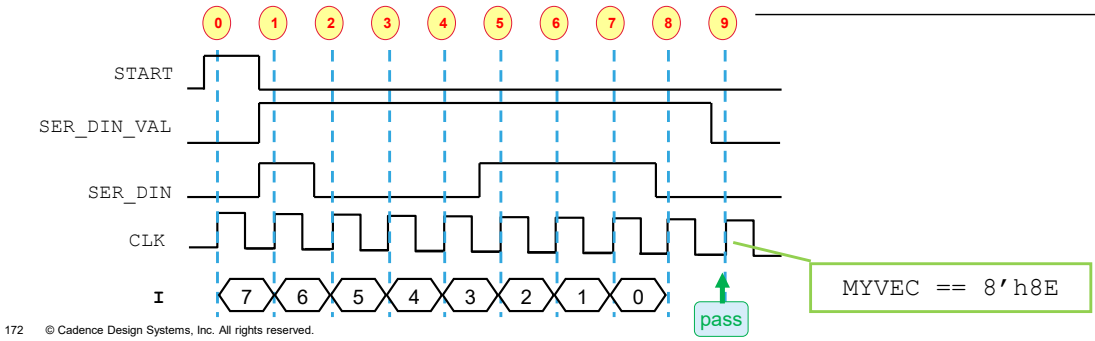
A property can define local variables

- There is no limit on the number of local variables

This example shows how to collect serial data.

```
localparam [7:0] EXPECTED_VAL = 8'h8E;
...
property MYPROP;
    int I;
    logic [7:0] MYVEC;
  @(posedge CLK)
    (START, I = 7) |=>
    (SER_DIN_VALID, MYVEC[I]=SER_DIN, I--)[*8]
     ##1 MYVEC == EXPECTED_VAL;
endproperty
```

Declare local variables inside property



172 © Cadence Design Systems, Inc. All rights reserved.

Properties can declare local variables. Although these may appear useful, they have limited application in practice. One useful application is deserialization, as shown here.

Property variables are declared after the property identifier and before the temporal condition. Variable operations can be executed at any point in the property evaluation, by adding the operation to an individual property term, separated by a comma (,). Multiple variable operations can be added to each term, as long as each is separated by commas. In fact, you can have any non time-consuming procedural action as an executable statement after the comma.

In the example, when the enabling condition is met (START is true on cycle 0), then we set the assertion variable I to the highest index of MYVEC (7).

From cycle 1, we expect SER_DIN_VAL true for 8 consecutive cycles ([*8]) until cycle 8.

On each of these 8 cycles, we:

Capture the serial data by writing the current value of SER_DIN to bit I of MYVEC.

Decrement I.

On cycle 9, we expect the collected serial data in MYVEC to be equal to 8E hex, as defined by the localparam EXPECTED_VAL.

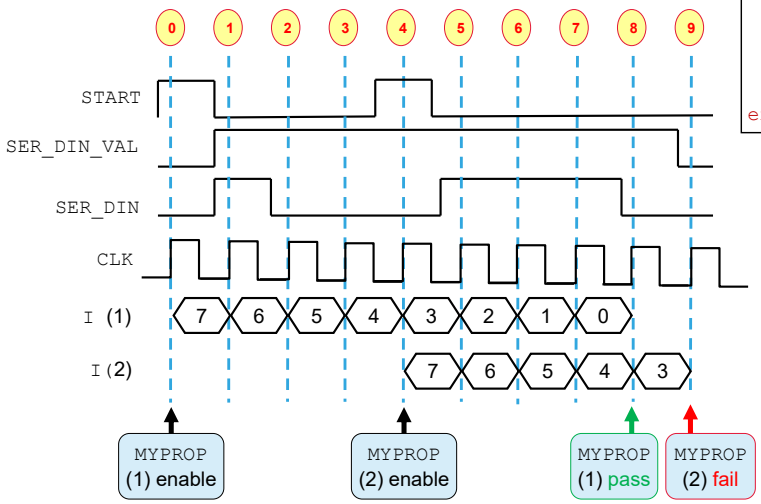
© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog

What Happens with Overlapping Properties?

Overlapping creates multiple assertion copies
Copies cannot communicate their own local variables

```
localparam [7:0] EXPECTED_VAL = 8'h8E;
...
property MYPROP;
    int I;
    logic [7:0] MYVEC;
  @(posedge CLK)
    (START, I = 7) |=>
    (SER_DIN_VALID, MYVEC[I]=SER_DIN, I--)[*8] ##1
       MYVEC == EXPECTED_VAL;
endproperty
```



173 © Cadence Design Systems, Inc. All rights reserved.

Remember that each time that the enabled condition occurs (START is true), then a new property copy is spawned, and the fulfilling condition is required. Each copy of the property has its own unique set of assertion variables, and each has its own fulfilling condition requirement. Remember also behaviour at a single cycle can satisfy the fulfilling conditions of multiple property copies.

This is the same property as the previous slide, but the waveform shows START occurring twice.

START at cycle 0 activates the first copy of MYPROP. Variable I of MYPROP(1) is initialised to 7. SER_DIN_VALID is high for the next 8 cycles and on each-cycle we capture SER_DIN into the next bit of MYVEC of MYPROP(1). MYPROP(1) passes on cycle 8 as its MYVEC equals 8'h8E.

START at cycle 4 actives a second copy of MYPROP with its own, independent set of property variables. Variable I of MYPROP(2) is initialised to 7. From cycle 5, MYPROP(2) captures SER_DIN into its MYVEC variable and expects 8 consecutive repetitions of SER_DIN_VALID. However, at cycle 9, SER_DIN_VALID is low so MYPROP(2) fails.

So, each activated assert copy has it's own independent set of property variables. The separate overlapping assertion copies cannot communicate with each other. Each one is in isolation. It can be very difficult to debug issues with overlapping properties containing local variables.

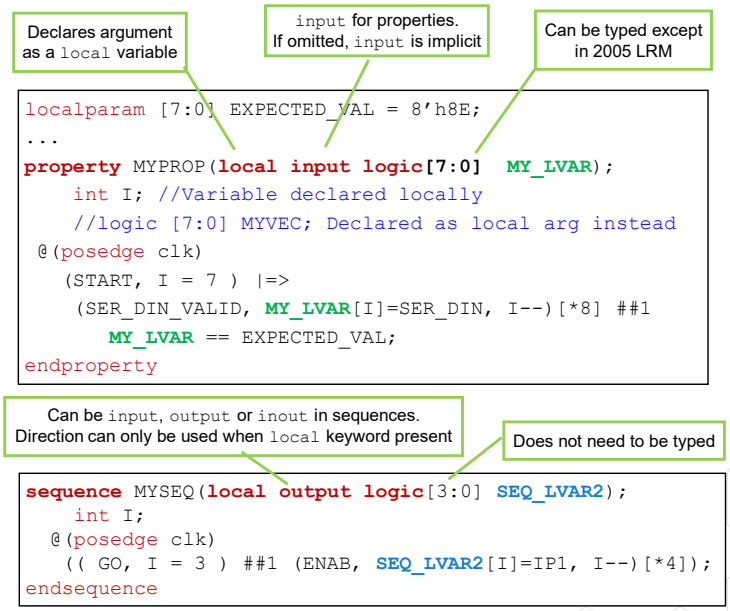
© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog

Why Passing Local Variables as Arguments Is Usually a Bad Idea?

Pragmatically, very problematic for these reasons

- What is legal has Language Reference Manual (LRM) dependencies
- Difficult to debug or be confident your intent is correctly specified
- Sequences and Properties have different rules
- Difficult to be aware of the scheduling issues
- Difficult to comprehend all the behaviors of overlapping threads
  - One property may instance a sequence which in turn instances another sequence, and so on
  - Overlapping threads cannot communicate with each other
- `input` local variable arguments are not visible outside of the sequence/property
- Cannot used `triggered()` or `matched()` sequences methods
- Other considerations, e.g., with sequence `and` and sequence `or` operators
- The list of restrictions continues …
  - Advantages don't often outweigh disadvantages



174 © Cadence Design Systems, Inc. All rights reserved.

We discussed simple sequence and property arguments, which can be considered as text macros, in the Property Reuse module.

You can also define direction and types for arguments, and arguments can be used with local variables.

Property MYPROP is declared with a local typed and directional argument MY_LVAR to capture the serial data. It behaves exactly the same as the previous example.

For properties, an argument must be an input. If we omit keyword input, then the argument is implicitly input because, for a property, it cannot be anything else. Arguments can be optionally be typed except in Language Reference Manual 2005.

For sequences, the direction can be either input, output or inout. The type is optional.

So, you could have nested sequences, where one sequence passes out a value to its parent sequence, up and down a hierarchy of nested sequences, but that is really complicated to debug, so is generally avoided.

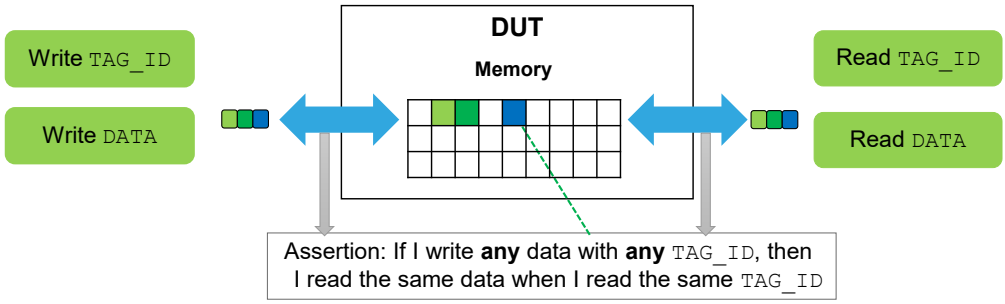
Typed and directional arguments have issues. There are different rules in different Language Reference Manual versions. They make assertions hard to debug or confirm they are correctly implemented. There are different rules for properties and sequences. Scheduling issues are a nightmare. You need to consider overlapping assertions. Input local variables are not visible outside of the sequence or property, so properties cannot pass out values. You can't use sequence methods like triggered. There are issues with sequence composition operators. So, there is a long list of restrictions and the advantages rarely outweigh the disadvantages.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog

Tracking Data Values Using Local Variables

- Data stored in memory upon write
- Data will not be overwritten if it has not been read yet
- Memory location freed upon a read, permitting future writes
- User responsible for only reading locations with valid data



Assumptions are required so this assert does not produce false negatives

- Only write when TAG_ID location is empty
- Only read when TAG_ID location has valid data, i.e., there has been a valid write previously
- Must eventually read all locations

175 © Cadence Design Systems, Inc. All rights reserved.

We have seen one good use of local variables - deserialization. The another good use is to track data values.

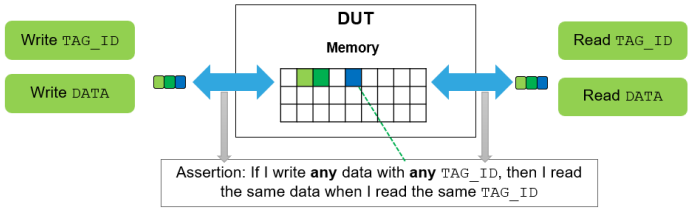
This example design has an internal memory. We write data to the DUT using an identifier tag and store the data in memory. Data cannot be overwritten until it is read, so there must be a free memory location for the write. We read by specifying a tag, which outputs the relevant data. Once a data item has been read, it's memory location is marked as empty and is available to write. The user must only read locations which actually contain data.

To check this behaviour, a reasonable assertion might be that a write of any data with any TAG_ID, eventually results in a read of the same TAG_ID and same data. This is a good formal assertion because you don't have to check all the individual data items. However, you will need some assumptions in formal to avoid false negatives. Our simulation testbench would also have to follow the same rules for the interface. For example, you should only write when the TAG_ID location is empty. Only read when the TAG_ID location has had a valid write. Eventually you must read all locations. All data written must be read. You cannot omit a specific location and claim that as a failure.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog

Example: Tracking Data Values Using Local Variables



Assumptions are required so this assert does not produce false negatives

- Only write when TAG_ID location is empty
- Only read when TAG_ID location has valid data, i.e., there has been a valid write previously
- Must eventually read all locations

```
ASM_ONLY_WRITE_LOCATION_WHEN_EMPTY: assume property (wr |-> wr_location_empty);
ASM_ONLY_READ_LOCATION_WHEN_VALID:  assume property (rd |-> rd_location_valid);
ASM_EVENTUALLY_MUST_RD_VALID_TAG:   assume property (s_eventually empty );
property WR_EVENTUALLY_READ_LVAR;
  logic [TAG_WIDTH-1:0] TAG;  //local vars declaration
  logic [DWIDTH-1:0]    DATA; //local vars declaration
  (wr, TAG = wr_tag_id, DATA = wr_data) |=>
            s_eventually (rd && rd_tag_id == TAG ##1 rd_data == DATA);
endproperty
AST_WR_EVENTUALLY_READ_LVAR: assert property( WR_EVENTUALLY_READ_LVAR);
```

176 © Cadence Design Systems, Inc. All rights reserved.

So, our assumptions for formal (which must also be respected for the simulation testbench) are:

A write must be to an empty location. So, we need auxiliary code to generate the signal wr_location_empty, to indicate the location for a specific TAG_ID is empty.

A read must be to a valid location. Again, we need auxiliary code to generate the signal rd_location_valid, to indicate the location for a specific TAG_ID has been written.

We must read all written locations, so eventually the memory is empty, indicated by another signal.

The property has two local variables. One for TAG_ID and one for the Data. For every write, we store the TAG_ID into the local variable TAG, and the data into the local variable DATA. A write enables the assertion, and at some point in the future, s_eventually , we must read a TAG_ID matching TAG and then on the next cycle, the read data must equal DATA. So, these assertions and assumptions would work in both simulation and formal.

Every write will enable a separate copy of the assertion, storing the specific values of TAG_ID and data. Each copy must eventually match those TAG_ID and DATA values for a read.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog

Multiclocked Properties



Multiclocked properties can use either the property implication operator

- Next cycle implication:

```
@(negedge CLK1) SEQ1 |=> @(posedge CLK2) SEQ2
```

  - Synchronizes last cycle of 1st clock to closest next cycle of 2nd clock

- Same cycle implication:

```
@(negedge CLK1) SEQ1 |-> @(negedge CLK1) SEQ2 ##1 @(posedge CLK3) SEQ3
```

  - Synchronizes last cycle of 1st clock to same delta cycle (if using simulation) of 2nd clock
  - Otherwise synchronizes last cycle of 1st clock to closest next cycle of 2nd clock
  - `SEQ3` uses next `posedge CLK3`



Caution

- Formal tools use a cycle-based model hence there is no concept of delta cycles!
- Be careful with multi-clocked properties – there are many issues to understand!
- Recommend using specific Clock Domain Crossing (CDC) tools rather than trying to code clock domain crossing checks in SVA

177 © Cadence Design Systems, Inc. All rights reserved.

cadence

We can use as many clocks as we like in a property. The first example uses negedge CLK1 for SEQ1 in the enabling condition, then a next cycle implication operator and then posedge CLK2 for SEQ2 fulfilling condition. The next cycle implication means once SEQ1 completes, we jump to the next future posedge of CLK2 before we start evaluating SEQ2. If we have a posedge of CLK2 in the same delta cycle as negedge CLK1, that edge is not used to evaluate SEQ2 because of the next cycle implication. We're using negedge CLK1 until we observe another clock and then we evaluate using the new clock. We keep going to the right until we see another clock. This is called clock flow.

It becomes more complicated with same cycle implication. What does same cycle mean if you have different clocks on either side of the operator?

Again, we evaluate the SEQ1 enabling condition using negedge CLK1. The same cycle implication means once SEQ1 completes, we look in the same delta cycle for a clocking event for the fulfilling sequence SEQ2. If there is no event in that delta cycle, then we jump to the next future clocking event to start evaluating SEQ2. The example uses negedge CLK1 on both sides of the implication operator, but if the clock for SEQ2 was negedge CLK2, we would complete SEQ1 on negedge CLK1, and check in the same delta cycle for a negedge CLK2.

This becomes very complicated to understand. Also, formal tools are cycle based and have no understanding of delta cycles. So, you have to be really careful with multi-clock properties. There is no standard way of approaching multi-clock properties. There are specialized Clock Domain Crossing (CDC) tools to handle cross domain checks. It is better to use these tools rather than try and write these checks in SVA.

© Cadence Design Systems, Inc. All rights reserved.

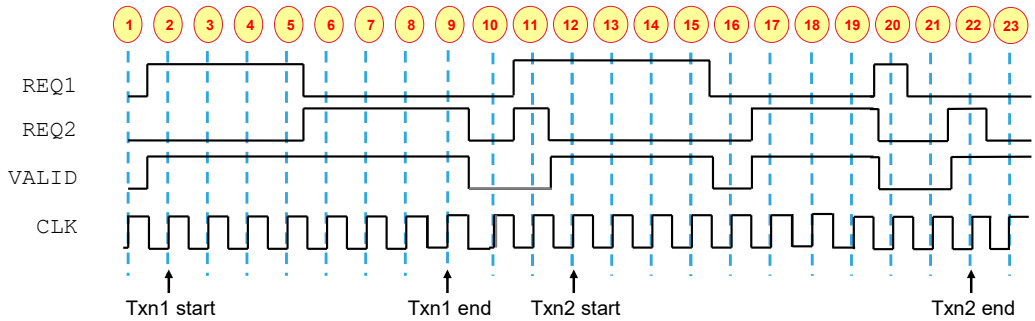
This document is for the sole use of Rajat G of Analog

Complex Properties

Requirements is for “simple” interface: REQ1 and REQ2 must not overlap. REQ1 and REQ2 must both last exactly 4 cycles. REQ2 must begin on cycle after REQ. If VALID = 0, then it doesn’t matter what REQ1 and REQ2 are doing; VALID can go to 0 at any time, for any number of cycles.

Easier solution for REQ1 and REQ2 must not overlap:

```
property P1;
  (@(posedge CLK)
   not (REQ1 && REQ2 && VALID));
endproperty
```



178 © Cadence Design Systems, Inc. All rights reserved.

The specification for this interface seems simple.

- REQ1 and REQ2 must not occur on the same cycle.
- REQ1 and REQ2 must be high for exactly 4 cycles.
- REQ2 must begin on the cycle after REQ1.

This seems simple. However, what complicates these requirements is the VALID signal. The requirements only apply when VALID is high. If VALID is low REQ1 and REQ2 can do anything. VALID can go low at any time for any number of cycles.

In the waveform, VALID stays high for cycles 2 to 9. We have REQ1 high for four consecutive cycles (2 to 5). Immediately after REQ1 finishes, we have 4 cycles of REQ2 (6 to 9). REQ1 and REQ2 do not occur in the same cycle. So, the transaction starts at cycle 2 and successfully completes at cycle 9.

At cycle 11, both REQ1 and REQ2 are high, but VALID is low, so this is not an issue. Our second transaction starts at cycle 12 when VALID is high and REQ1 is high. REQ1 is high for 4 cycles until 15. On cycle 16 we expect REQ1 to be low and REQ2 to be high, but VLIAD is low, so we don't care what happens on cycle 16. REQ2 and VALID are high on cycle 17, so we start counting the 4 expected cycles of REQ2. We count 3 cycles of REQ2 until cycle 19, but on cycles 20 and 21 VALID is low, So, again, we ignore what REQ1 and REQ2 are doing. VALID goes high on cycle 22, when REQ2 is also high, and that counts as our 4th cycle of REQ2. So, the second transaction starts cycle 12 and ends cycle 22.

It is very difficult to write assertions which cater for all possible variations, given that VALID can go low at any cycle for any number of cycles.

<<clocking, complex>>

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog

Solution: Easy with Complex Clock Expression

Simplify properties with more complex clock expression

Define a qualified sampling clock

- Use `iff` clause

```
property P2;
  ( @(posedge CLK iff VALID)
     not(REQ1 && REQ2));
endproperty
```

REQ1 and REQ2 must not overlap

```
property CHK_REQ_LEN (REQ, CLK, VALID, NUM_CYC);
  ( @(posedge CLK iff VALID)
    ( !REQ ##1 REQ |=> REQ[*NUM_CYC-1] ##1 !REQ );
endproperty

CHK_REQ1_LEN: assert property(CHK_REQ_LEN(REQ1, CLK, VALID, NUM_CYC));
CHK_REQ2_LEN: assert property(CHK_REQ_LEN(REQ2, CLK, VALID, NUM_CYC));
```

REQ1 and REQ2 must both last exactly 4 cycles (NUM_CYC = 4)

```
sequence REQ_N_CYC(REQ, N)
  REQ[*N];
endsequence

property R2_FOLLOWS_R1
  ( @(posedge CLK iff VALID)
     REQ_N_CYC(REQ1, 4) |=> REQ_N_CYC(REQ2, 4) );
endproperty
```

REQ2 must begin on cycle after REQ1 and then complete

It is typical to not specify a clock for the named sequences

179 © Cadence Design Systems, Inc. All rights reserved.

cadence

However, writing the properties is trivial if we use an iff clause in the property clock.

So now we only evaluate the property on posedge CLK if VALID is high. For cycles where VALID is not high, we do not evaluate the property.

We will also use arguments in our properties. We need to check both REQ1 and REQ2 last for four cycles. We can write a single property with arguments and instantiate the property for both requests. We can make our property even more flexible by passing request, CLK, VALID and number of cycles as arguments.

The property CHK_REQ_LEN is evaluated at the posedge of CLK if VALID is true. If we have not REQ followed by REQ, this implies that we get NUM_CYC-1 consecutive repetitions of REQ (because we use next cycle implication) followed by not REQ. We assert that property twice, passing REQ1 for one and REQ2 for the other.

To check we have 4 cycles of REQ1 followed immediately by 4 cycles of REQ2, we declare a sequence REQ_N_CYC with two arguments, the signal name and the number of repetitions. In the property R2_FOLLOWS_R1, the enabling condition is the sequence with arguments REQ1 and four cycles. The fulfilling condition is the sequence with arguments REQ2 and four cycles. Again, these sequences are only evaluated at posedge CLK if VALID is true.

So, our properties are greatly simplified by choosing an appropriate clocking expression.

Remember the clocking iff is not the same as disable iff. This is not an abort. This is a conditional event expression. disable iff or sync abort or sync accept or sync reject will cancel evaluation of the property. Clocking iff just suspends it.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog

Property Clocking Recommendations



Choose the simplest and correct clocking edge to evaluate properties, using the least-frequent and non-periodic checks needed, because proper property clocking is fundamental for accurate verification.

If DUT is fully synchronous to clock rising edge

- Use clock rising edge to evaluate properties
  - Enables Formal tools to perform clock optimizations
- Same result but nothing gained by using clock falling edge
  - Gate level simulations probably fail unless you have > ½ cycle slack
- Same result from Dynamic and Formal Verification

Use the most infrequent property clock possible for checking behavior

- Don't necessarily need to check every property on each active clock

Property clock does not need to be periodic

- For example, you may only want to check a property when a write address changes

Always take time to understand the issues

- Property clocking is absolutely fundamental
- If you get this part wrong, then everything else will be wrong

 © Cadence Design Systems, Inc. All rights reserved.

cadence

Property clocking recommendations.

Always use the same edge of the clock for properties as your design clock. If a DUT only changes state at a rising edge of clock, then it's essential that your properties only use the rising edge of a clock because this makes things more efficient in formal. There is no need to consider the negedge of clock. We can also reuse the assertions in gate level simulations,.

If possible, use the most infrequent property clock for checking behaviour. You do not need to check every property on every active edge of a clock. You can qualify with iff, for example.

The property clock does not need to be periodic. You can create custom signals purely for clocking properties if, for example, you only want to check a property when a write address changes.

Always take time to understand the issue. If you get the clocking wrong, everything else is wrong. You have to really understand what you're doing.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog

Quiz



1.
```
sequence ONE;   (A ##1 B[*3] ##1 C);   endsequence
sequence TWO;   (K ##1 L ##1 M ##1 N); endsequence
sequence THREE; (X ##2 Y[*2]);         endsequence
sequence WHATSEQ;
 (ONE.triggered ##1 TWO.triggered ##1 THREE.triggered);
endsequence
```

Expand WHATSEQ?

Solution:

```
sequence WHATSEQ_EXPANDED;
  A ##1 B ##1 (B && K) ##1 (B && L && X) ##1 (C && M) ##1 (N && Y) ##1 Y;
endsequence;
```

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog

Quiz (continued)



2. In the property below, `end` must go high within 5 and 15 cycles of `start`. For a response of 10 cycles or less, `short` is asserted with `end`, For greater than 10 cycles, `long` is asserted with `end`. Modify the property to check right number of cycles for each alternate response:

```
property count_cycles;
  @(posedge clk)
  $rose(start) |=> !end[*4:14] ##1 ((end && short) or (end && long));
endproperty
```

Solution:

```
property count_cycles_modified;
int cyc;
  @(posedge clk)
  ($rose(start), cyc=0) |=> (!end, cyc=cyc+1)[*4:14] ##1
     ((end && short && cyc<= 10) or (end && long && cyc > 10));
endproperty
```

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog

Summary

You now have some powerful constructs for describing complex assertion sequences

- Endpoints
  - Convert the completion of a sequence into a boolean
  - Allow multiple concurrent sequences to be described
- Action blocks
  - Allow procedural code to be executed on the pass or failure of an assertion
- Assertion local variables
  - Easier definition of complex assertions by accessing local dynamic variables on the completion of a sequence or property
- Defining conditional and multi-clocked properties

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog



Module 11

Constructs Which Form Properties

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog

Module Objectives

After this module, you will be able to

- Describe the constructs that form properties in SVA, constructs that returns a property
- Identify which constructs are of practical use and understand why they matter
- Explain the constructs with pragmatic use: `until` family constructs, `always` construct, `followed-by` operators, `implies` operator
- Evaluate why certain SVA constructs and operators are not of pragmatic use: `if-else` Properties, Conjunction and Disjunction operators, `iff` operator, `nexttime` family of operators

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog

Overview of Property Forming Constructs



IEEE 1800-2009 introduced many new operators that return a property expression

- Some of these are of pragmatic use and some are not

As standards are decided by committee, it's the harsh reality that bad ideas don't always get rejected

Some of these operators were introduced because:

- They have a practical use
- Others just because PSL has a similarly named operator

Many of the expressions allow you to form a property expression, which can be used where the syntax requires a property expression

- This couldn't have been done previously in some cases

This section is divided into 2 sections

1. Expressions of practical use
2. Expressions little or not of practical use

We show descriptions of each operator and some practical examples

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog



Submodule 11.1

Property Forming Constructs: Constructs with Pragmatic Value

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog

Submodule Objectives

After this submodule, you will be able to

- Explain the `until` constructs to model temporal conditions
- Apply the `always` construct to enforce conditions that must hold continuously
- Use `followed-by` operators `(#-#, #=#)` to require property satisfaction after sequence completion
- Differentiate the `implies` operator by its unidirectional, concurrent evaluation semantics

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog

`until` Property Operator

```
<property expr 1> until <property expr 2>
```

If `B` occurs, then from the next cycle, `C` is required to be true every cycle until `D` is true

There are other forms of the `until` operator, combinations of weak/strong and [non-]overlapping forms

We discuss these later

`P_UNTIL_NOVRLP_W` is the **weak non-overlapping** form, which means:

- `C` is not required to be true in the cycle where `D` occurs – non-overlapping
- `D` is never required occur, as long as `C` stays true – a weak property

Weak non-overlapping form of `until`

```
P_UNTIL_NOVRLP_W:
  assert property ( B |=> (C until D) );
```

Equivalent property described with sequences

```
P_UNTIL_NOVRLP_W_SEQ:
  assert property ( B |=> (C[*] ##1 D) );
```

 © Cadence Design Systems, Inc. All rights reserved.

cadence

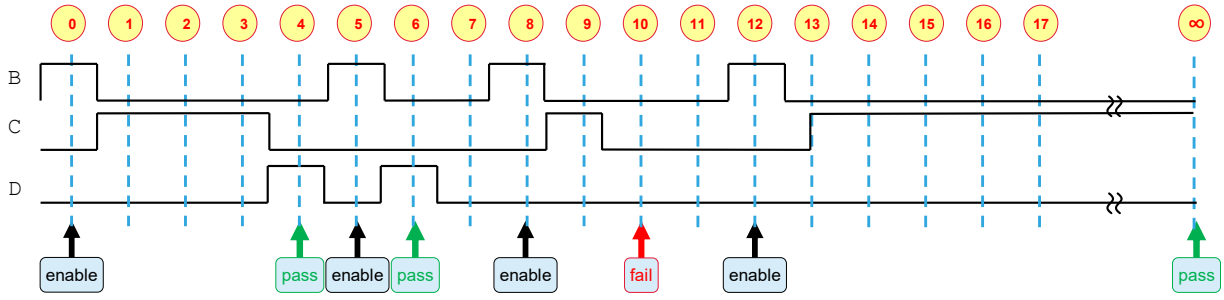
This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog

`until` Property Waveform

```
P_UNTIL_NOVRLP_W:
  assert property ( B |=> (C until D) );
```



190 © Cadence Design Systems, Inc. All rights reserved.

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog

`until_with` Property Operator

```
<property expr 1> until_with <property expr 2 >
```

If `B` occurs then from the next cycle, `C` is required to be true every cycle until `D` is true, *including* the cycle `D` becomes true

`P_UNTIL_OVRLP_W` is the **weak overlapping** form, which means:

- `D` is never required occur, as long as `C` stays true – a weak property
- `C` is required to be true in the cycle where `D` occurs – overlapping

Weak overlapping form of `until`

```
P_UNTIL_OVRLP_W:
  assert property ( B |=> (C until_with D) );
```

Equivalent property described with sequences

```
P_UNTIL_OVRLP_W_SEQ:
  assert property ( B |=> (C[+] ##0 D) );
```

 © Cadence Design Systems, Inc. All rights reserved.

cadence

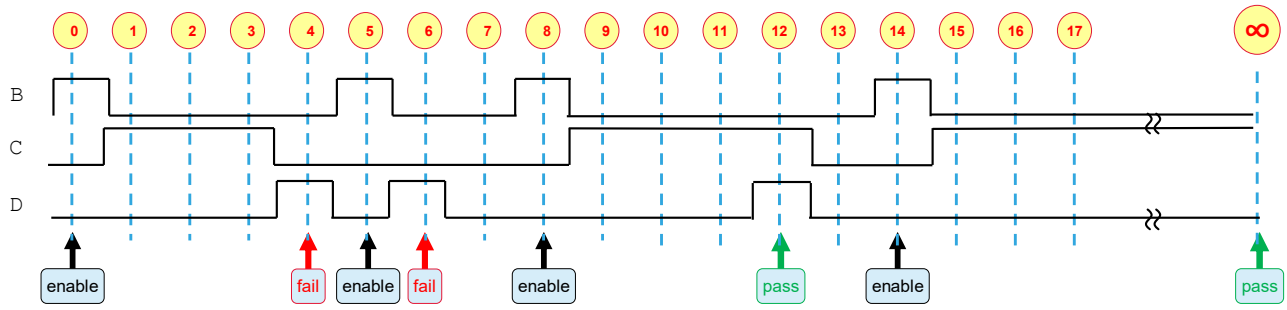
If you need C to be high in the cycle where D occurs, you use until_with. This is the weak overlapping form of until. It's overlapping because we require C to be true in the same cycle as D is true. Again, we can write a sequence equivalent, defining 1 to an infinite number of repetitions of C and using fusion, ##0, to check C is true when D occurs.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog

`until_with` Property Waveform

```
P_UNTIL_OVRLP_W:
  assert property ( B |=> (C until_with D) );
```



192 © Cadence Design Systems, Inc. All rights reserved.

We have a similar waveforms as before. B occurs on cycle 0 and C stays high until cycle 3, but in cycle 4, where D occurs, C is not high, so the property fails.

We have B on cycle 5, followed by D on cycle 6, but again C is not high in the same cycle as D, cycle 6, so the property fails.

We have B at cycle 8, and C stays high from cycle 9 to cycle 12, where we find D. So, C and D are true in the same cycle (12) and the property passes.

B occurs again at cycle 14 and C is high from cycle 15 until infinity, but D never occurs. The property is a pass because it is a weak form.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog

s_until Property Operator

```
<property expr 1> s_until <property expr 2 >
```

If B occurs then from the next cycle, C is required to be true every cycle until D is true

P_UNTIL_NOVRLP_S is the **strong non-overlapping** form, which means:

- D is required occur – a strong property
- C is not required to be true in the cycle where D occurs – non-overlapping

Strong non-overlapping form of until

```
P_UNTIL_NOVRLP_S:
  assert property ( B |=> (C s_until D) );
```

Equivalent property described with sequences

```
P_UNTIL_NOVRLP_S_SEQ:
  assert property ( B |=> strong(C[*] ##1 D) );
```

 © Cadence Design Systems, Inc. All rights reserved.

cadence

If second operand must occur, then we need a strong form of until with s__in the front. Remember when we talked about liveness properties, s_ means strong.

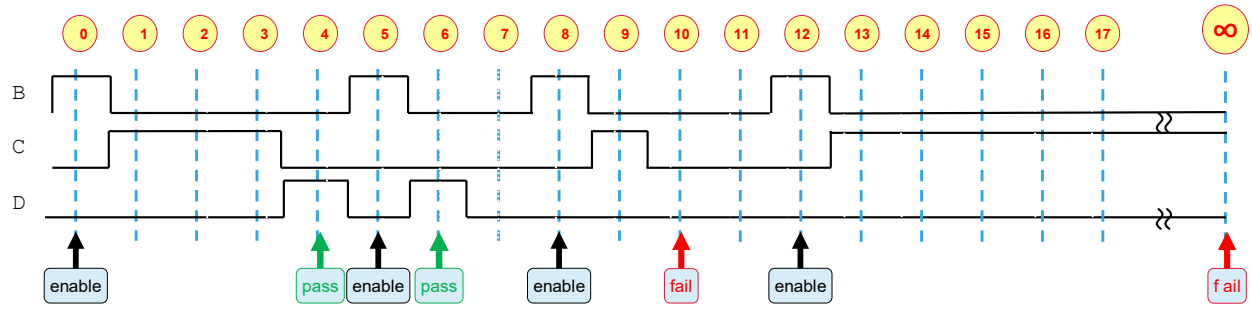
s_until is the strong, non-overlapping form of until. The equivalent sequence uses the keyword strong and surrounds the rest of the expression with parentheses. So, D has to occur.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog

`s_until` Property Waveform

```
P_UNTIL_NOVRLP_S:
  assert property ( B |=> (C s_until D) );
```



194 © Cadence Design Systems, Inc. All rights reserved.

cadence

In the waveform, the behavior is exactly the same as the until example, except for the final stimulus.

B occurs on cycle 0 and C stays high until cycle 3. The property passes in cycle 4, where D occurs. The property is non-overlapping, so we don't check C in cycle 4.

B occurs on cycle 5, followed immediately by D on cycle 6, where the property passes. Again, we don't check C in the cycle as D occurs.

B at cycle 8 enables the property. C occurs at cycle 9, but is low at cycle 10, and D is also low, therefore the property fails.

The property is again enabled at cycle 12. C stays high from cycle 13 to infinity, and the property fails because it's the strong form. D must occur.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog

s_until_with Property Operator

```
<property expr 1> s_until_with <property expr 2 >
```

If B occurs then from the next cycle, C is required to be true every cycle until D is true, *including* the cycle D becomes true

P_UNTIL_OVRLP_S is the **strong overlapping** form which means:

- D is required occur – a strong property
- C is required to be true in the cycle where D occurs – overlapping

Strong overlapping form of `until`

```
P_UNTIL_OVRLP_S:
  assert property ( B |=> (C s_until_with D) );
```

Equivalent property described with sequences

```
P_UNTIL_OVRLP_S_SEQ:
  assert property ( B |=> strong(C[+] ##0 D) );
```

 © Cadence Design Systems, Inc. All rights reserved.

cadence

s_until_with is the strong form of until_with. So, this is the strong, overlapping form of until. We require C to be high in the same cycle as D, and we require D to eventually occur.

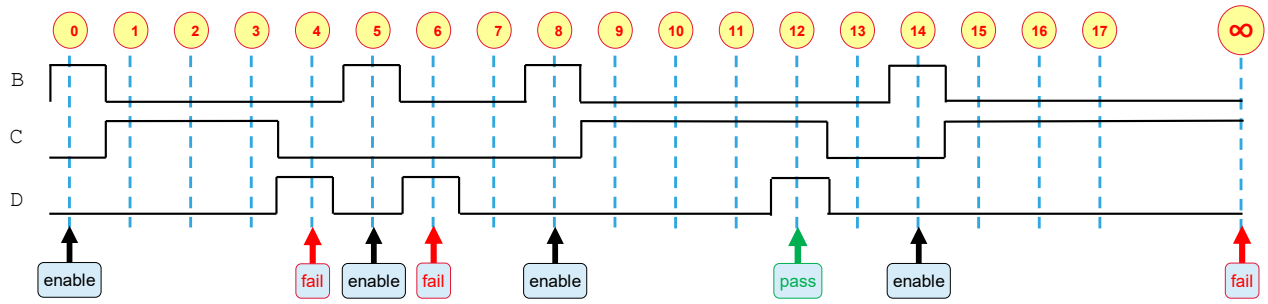
So, the sequence equivalent has the keyword strong, and , as for until_with, C occurring one or more times. And when D occurs, we check C is still high using ##0.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog

`s_until_with` Property Waveform

```
P_UNTIL_OVRLP_S:
  assert property ( B |=> (C s_until_with D) );
```



 © Cadence Design Systems, Inc. All rights reserved.

We have similar waveforms as before.

B occurs on cycle 0 and C stays high until cycle 3, but in cycle 4, where D occurs, C is not high, so the property fails.

B on cycle 5, followed by D on cycle 6, but again C is not high with D in cycle 6, so the property fails.

B at cycle 8, and C stays high from cycle 9 to cycle 12, where we find D. So, C and D are true in the same cycle (12) and the property passes.

B occurs again at cycle 14 and C is high from cycle 15 until infinity, but D never occurs. The property fails because it is a strong form.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog

Why Sequence-Based Properties Are Preferable to `until` Forms?



`until` form disadvantages:

- You need to remember how they behave, including all different forms
  - This is not convenient, easy to make mistakes
- With sequences there is never any ambiguity or doubt
  - You just need to know how to define a sequence

`until` form advantage:

- You could reasonably argue that they are more readable than the sequence form
  - A weak argument in my view

```
P_UNTIL_NOVRLP_W:
  assert property ( B |=> (C until D) );
```

=

```
P_UNTIL_NOVRLP_W_SEQ:
  assert property ( B |=> (C[*] ##1 D) );
```

197 © Cadence Design Systems, Inc. All rights reserved.

cadence

The until form disadvantages are you have to remember what each form does. It invites mistakes for no particular benefit. With sequences, there's never any ambiguity or doubt. All we have to know is how to define a sequence.

The only argument you could make for the until forms is they are more readable than the sequence forms but that is arguable.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog

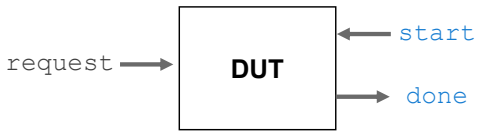
Why Liveness Assertions Can Fail in Formal Verification?

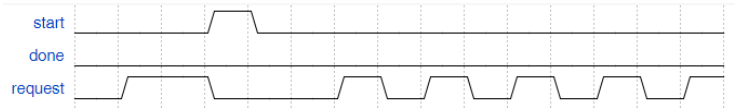
A liveness assertion can fail in formal verification because the environment is allowed to keep inputs active forever, preventing forward progress, termed as Input Livelocks.

New packets continue to enter causing resource never to be available.

For example:

```
assert property (start |-> s_eventually done);
```





Not sufficient to say that inputs will eventually be idle

- e.g., `assume (s_eventually !request)` does not solve the case above

Need to forbid new input requests/events/transactions during the loop

- See next slide

False Failure
`done` never asserted
because requests keep coming

198 © Cadence Design Systems, Inc. All rights reserved.

cadence

Input live locks are conditions where liveness assertions might generate false failures because the inputs keep the design busy.

For example, an assertion saying if a packet is input, eventually all packets are processed is easy for a formal tool to fail by sending packets in continually.

In the example, once the DUT receives start, it can start processing requests and raises done when it has finished.

The assertion states that once we have input start to the DUT then eventually the DUT must respond with done. This is a perfectly reasonable property to write. The problem is that the tool can fail the assertion by continually making requests. If we keep making requests, we can never be done.

This is not a design error, it is a verification problem because formal can fail the assertion under normal design behavior. This just shows you how good the tool is at finding problems. We could try a strong assumption that eventually, at some future cycle, there will be no requests, but that doesn't solve the problem. request going low for a cycle would satisfy the assumption but does not stop repeated future requests. request can just bounce up and down forever. What we need to do is say that from some point, there are no further requests.

© Cadence Design Systems, Inc. All rights reserved.

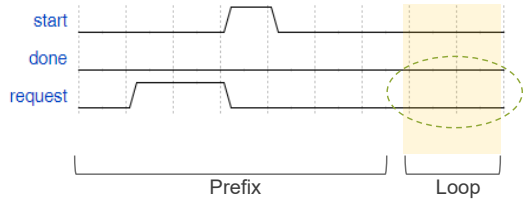
This document is for the sole use of Rajat G of Analog

Preventing Input Livelocks Using `always`

```
always <property expression>
```

Defines a property that requires a property expression to be true in the current cycle and all future cycles

Use `s_eventually always` to force events to be always true in loop




```
assume property (
  s_eventually always !request );
```

 © Cadence Design Systems, Inc. All rights reserved.

cadence

We can solve input livelock problems using the keyword always.

always takes a property expression operand and the whole always returns a property expression.

Instead of an assumption (s_eventually !request), which we said would not prevent future requests, we write (s_eventually always !request).

This means that at some future cycle, request must be always low for infinity.

There is no other way of achieving a solution to this input livelock problem in SVA other than using the always keyword. So always can be really useful.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog

Followed-By Operator

Overlapping: **sequence #-# property**

Non-overlapping: **sequence #=# property**

If a sequence completes, then a property must be true

- The sequence must complete

For example, property `FB_NC`

```
property FB_NC(
    @(posedge clk) (B ##1 C) #=# (D |=> E) );
endproperty
```

- Sequence `( B ##1 C )` **must** occur
- In the cycle after (non-overlapping) which `( B ##1 C)` occurs, then either:
  - `D` occurs followed next cycle by `E`, **or**
  - `D` does not occur (vacuous)

LRM defines Followed-By operator equivalents:

`#=#` == `not(<sequence_expr> |=> not <property_expr>)`

`#-#` == `not(<sequence_expr> |-> not <property_expr>)`

200 © Cadence Design Systems, Inc. All rights reserved.

cadence

The followed by operator can be difficult to understand.

The operator is actually three characters. #-# is same cycle overlapping. #=# is next cycle nonoverlapping. Similar to the implication operators.

The left-hand operand is a sequence and the right-hand side is a property. The followed-by operator states that firstly, the left-hand sequence must complete and secondly, when that sequence completes, the right-hand property is true. For #-#, the property must be true in the same cycle as the sequence completes. For #=#, the next cycle.

Here's an example, which looks perfectly reasonable. We must see the sequence B followed by C, and when we complete that sequence, then on the next cycle, the property must be true. So, either we have D followed on the next cycle by E, or D is false (a vacuous pass for the implication operator).

Now the LRM defines these equivalents for the followed by operators, which just adds to the confusion.

Sequence implies not (property expression), and then we invert the truth of that. For both next cycle and same cycle. It is hard to understand what that means. So, let's try and simplify this.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog

What Does It Mean If We `assert` a Followed-By Operator?

```
<sequence_expr> #-# <property_expr>
```

```
not(<sequence_expr> |-> not <property_expr>)
```

Let us consider that `<sequence_expr> S` and `<property_expr> P` are both Booleans

S	P	not P	S |-> not P	not (S |-> not P)
0	0	1	True	False
1	0	1	True	False
0	1	0	True	False
1	1	0	False	True

```
FC_SC_ALL_BOOLS: assert property(S #-# P);
```

The only cycles on which this does not fail are when `S` is proceeding and `P` is passing

```
property FB_NC(
    @(posedge clk) (B ##1 C) #=# (D |=> E) );
endproperty
```

This property holds if D is false or if E follows D in the next cycle.

So how can Followed-By be useful?

201 © Cadence Design Systems, Inc. All rights reserved.

cadence

The LRM defines property expressions. A property expression can be a sequence expression. A sequence expression can be a Boolean, a sequence of length one cycle.

So, in the simplest case, we can define both operands of a followed-by operator as Booleans.

We can construct a table, where S is the sequence and P is the property, and we have columns for each term of the LRM specified equivalent for the followed-by operator, for example not P is the inverse of the truth of the property.

Once we fill out the table, we can draw some conclusions. If the sequence is false, or the property is false, then the followed-by operator fails. In fact, the only condition under which the operator does not fail is when the sequence is proceeding (currently true) and the property is passing. So, remember, an implication property passes under two circumstances, either when the enabling condition completes and is followed by a correct fulfilling condition, or if the enabling condition is not true (a vacuous pass).

If we look again at the example:

```
(B ##1 C) #=# (D |=> E)
```

The property succeeds if we have D followed next cycle by E or if D is false.

As an assertion, this property is virtually useless. So how can followed-by be useful?

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog

Followed-By Example: Liveness Covers

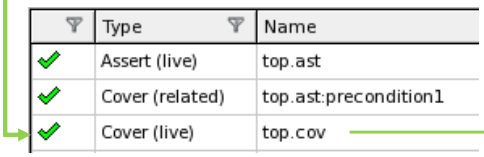
Possible to create liveness covers using the `always` keyword

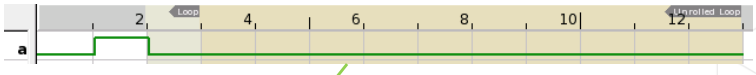
Recommended to have one liveness cover for each assertion

- Make sure precondition can be hit and extended to infinity

```
assert property (a |-> s_eventually b);
cover property (a #-# always 1);
```

Followed-by operator
`(X #-# Y)` is equivalent to `(not (X |-> not Y))`





Cover trace shows `a` followed by infinity

202 © Cadence Design Systems, Inc. All rights reserved.

Followed-by operators are used with always to create liveness covers.

The assert says that if we have A then s_eventually (strong form) we see B. However, it may be possible, especially in formal, for A to occur and the verification to finish before B occurs. The assertion would not fail, and you would never know that something is preventing an infinite length trace.

The followed-by cover says firstly A occurs and from the same cycle, the cover is always true.

In formal, this proves that A can occur and from A, formal can create an infinite length trace, meaning there isn't any over constraining in your environment.

So, this is a combination of always and the followed-by operator to create a cover to infinity in formal. Jasper shows the infinite length trace as a loop – the shaded portion of the waveform, bounded by loop labels, which can be repeated forever.

Requires `analyze -sv09 (or above) and elaborate -lrm_cover_property`

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog

`implies` Operator



```
prop1 implies prop2;
```

For the expression to pass, `prop2` must be true if `prop1` is true

This is a unidirectional relationship

- If `prop1` is not true, we don't care about `prop2`

`prop1` and `prop2` are evaluated concurrently

Also useful for sequences

- Remember that a property expression can be a sequence expression

 © Cadence Design Systems, Inc. All rights reserved.

cadence

The implies operator is a unidirectional relationship. This is not the same as an implication operator.

If the left-hand property operand prop1 is true, then the right property operand prop2 has to be true as well. If prop1 is not true, we don't care what prop2, which is why this is unidirectional. prop1 and prop2 are evaluated concurrently.

The implies will succeed if prop1 and prop2 are vacuously passing, i.e. their enabling conditions are false, so implies is of limited use.

We can use implies with sequences as well.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog

Don't Confuse `implies` with `|=>`

For `IMPLIES_GOOD` to pass takes 2 cycles:

- `B` and `D` need to be true on the same cycle
- `C` and `E` need to be true on the cycle after that

```
property IMPLIES_GOOD
    ( (B ##1 C) implies (D |=> E) );
endproperty
```

For `IMP_BAD_NC` and `IMP_BAD_M_IMP` to pass takes 4 cycles:

- `B` needs to be true…
- …then `C` true next cycle…
- …then `D` true next cycle…
- …then `E` true the cycle after that

```
property IMP_BAD_NC
    ( (B ##1 C) |=> (D |=> E) );
endproperty
```

≡

```
property IMP_BAD_M_IMP
    ( B |=> C |=> D |=> E );
endproperty
```

204 © Cadence Design Systems, Inc. All rights reserved.

cadence

Do not confuse implies, which is a property operator, with the implication operator.

Property IMPLIES_GOOD is B followed by C, implies D next cycle E. Remember the implies operands are evaluated concurrently. So, for a non vacuous pass, B and D must be true on the same cycle and then C and E have to be true on the following cycle. If B or D are not true at a specific cycle, the property passes vacuously.

If we used next cycle implication instead of implies (property IMP_BAD_NC), this takes 4 cycles to pass – B followed by C followed by D followed by E. This looks strange, with multiple implication operators, but this is equivalent to IMP_BAD_M_IMP.

When you have an expression with multiple implication operators, every term to the left-hand side of the right most implication operator is part of the enabling condition. So, it's not until we reach the final, right hand implication operator, that we actually have a requirement to check something. The only time IMP_BAD_M_IMP will fail is if you see B followed by C followed by D and then you don't get E the next cycle.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog



Submodule 11.2

Property-Forming Constructs: Constructs with Little Pragmatic Value

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog

Submodule Objectives

After this module, you will be able to

- Explain the `if-else` construct and its limited practical use
- Analyze Conjunction and Disjunction properties and their vacuous behavior
- Differentiate the `iff` operator as a bidirectional property relationship
- Relate the `nexttime` operator to equivalent sequence-based timing behavior

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog

`if-else` Property Operator

```
if (<Boolean>)
  <property expr1>
else
  <property expr2 >
```

These are what I describe as WWYEWTDT properties:

- Why Would You Ever Want To Do That?

```
property if_else_NC;
  if (A)
    B |=> C
  else
    D |=> E;
endproperty
```

```
property if_else_NC_seq;
  if (A)
    B |=> C ##1 J
  else
    D |=> E ##1 K;
endproperty
```

Works with sequences

```
property if_else_complicated;
  if (Y)
    R |=> S ##1 T
  else (if (F)
          N |=> P ##1 Q
        else
          M |=> V ) ;
endproperty
```

Don't over-complicate matters

 © Cadence Design Systems, Inc. All rights reserved.

cadence

if else is just a conditional statement. If some Boolean is true, then the first property expression has to hold, otherwise, i.e. if the Boolean is false, then the second property expression has to hold.

For property if_else_NC, if A is true, then either B is true on the same cycle, followed next cycle by C or B is false (for a vacuous pass). Otherwise, if A is false, either D is true on the same cycle, followed next cycle by E or D is false. If else also works with sequences.

The whole if expression, from keyword if to the semicolon of property expression 2, itself returns a property expression. So, we can nest if else expressions to any degree we wish. Although it becomes hard to understand what's happening for debug.

However, this raises the question "Why would you ever want to do that?"

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog

Why `if-else` Properties Are Not Pragmatic?

Disadvantages:

- Complicates coverage analysis
  - Did I execute all branches (might be multiple if-else branches)?
- If the property is asserted and fails, we need to spend time analysing which branches(s) fail
- In formal, if the property is asserted and passes, we don't know all branches will always pass without more effort and risk
- In formal, we normally decompose the problem in order to run proofs in parallel to reduce runtime
  - i.e., have more than one property to describe the equivalent behaviour of the single property
- Does not do anything we couldn't easily describe in a less complicated way

```
P1_if_else:
  assert property(
    if (A) B |=> C
    else   D |=> E);
```

≡

```
PROP_A_TRUE_IMP :
  assert property( A |-> B |=> C);
PROP_A_FALSE_IMP:
  assert property(!A |-> D |=> E);
```

≡

```
PROP_A_TRUE_BOOL :
  assert property( A && B |=> C);
PROP_A_FALSE_BOOL:
  assert property(!A && D |=> E);
```

Advantages:

- I have never seen one since `if-else` was first introduced in the 2005 LRM
- I do not think having a single property is a practical advantage

208 © Cadence Design Systems, Inc. All rights reserved.

cadence

if else complicates coverage analysis. If a property has multiple if else branches, how do we know we covered them all?

If we assert the property and it fails, how do we know which branch failed? Or do multiple branches fail? We would have to eliminate each possibility in turn. This is needless work which doesn't teach us anything.

If we assert the property and it passes, we don't know whether all branches will always find a pass without further effort and coverage.

The standard practice in formal is the exact opposite of if else. We decompose problems into separate multiple properties, rather than aggregating them. Aggregated properties take longer to prove and require more debugging if they fail. In formal, describing behaviour in multiple properties gives two key benefits. Firstly, it is less complex to debug and secondly multiple properties can be run in parallel to save time.

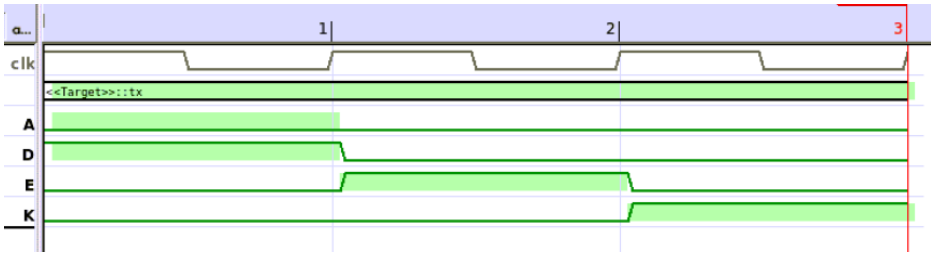
So, for simplicity and efficiency, we could just write two properties instead of one, and if the property terms are Boolean expressions, we could optimize further.

There are no advantages of using if-else, but plenty of disadvantages.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog

Example `if-else` Properties: RHS Sequence Case




```
P2: assert property ( if (A) B |=> C ##1 J
                      else   D |=> E ##1 K );
```

The waveform shows an example (cover) of the property passing (called a witness)

- Green shading highlights what the signals values need to be for `P2` to pass

If `D` were not true at cycle 1 then this does not mean `P2` fails as the LHS would be incomplete (vacuous pass)

If `E` were not true at cycle 2 or `K` were not true at cycle 3 then this means `P2` fails

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This waveform is an example of the if-else passing, as shown by a Jasper witness. The green shading shows the required signal values for the assertion to pass. Unshaded cycles for a signal can be any value.

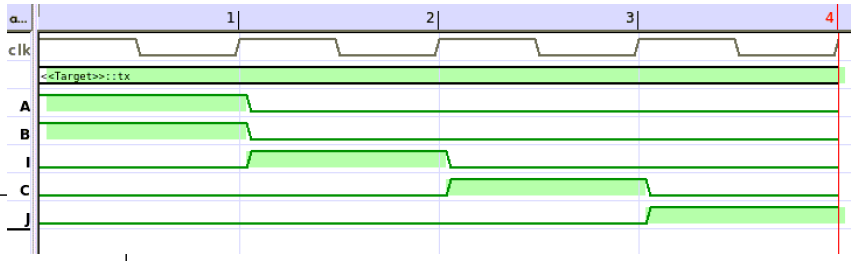
This waveform executes the else branch here. A is false in cycle 1, in the same cycle as D, which implies E at cycle 2 followed by K at cycle 3.

If D was false at cycle 1, P2 would not fail because D is the enabling condition for the implication operator in the else branch. So, the else branch property would vacuously pass. If D was true at cycle 1, and E was not true at cycle 2, or K is not true at cycle 3, the property would fail.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog

Example `if-else` Properties: LHS Sequence Case




```
property if_else_NC_seq_LHS;
  if (A) B ##1 I |=> C ##1 J
  else   D ##1 L |=> E ##1 K;
endproperty
P1: assert property (if_else_NC_seq);
```

The waveform shows an example (cover) of the property passing (called a witness)

- Green shading highlights what the signal values need to be for `P1` to pass

If `I` were not true at cycle 2 then this does not mean `P1` fails, as the LHS would be incomplete (vacuous pass)

If `C` were not true at cycle 3 or `J` were not true at cycle 4 then this means `P1` fails

210 © Cadence Design Systems, Inc. All rights reserved.

If we have a sequence on the left-hand side of either branch of an if-else, then the if Boolean condition is only checked on the first cycle of the sequence to determine which branch to take.

So again, we use a Jasper witness to show execution of the if branch. The green shading shows the required signal values for the assertion to pass. Unshaded cycles for a signal can be any value.

The left-hand side of both property branches is now a sequence. The if branch is B followed by I implies next cycle C followed by J. A is true on cycle 1 to select the if branch but need not be true on cycle 2 when we check the second term, I, of the enabling sequence.

This can be very confusing if you have overlapping versions of the property

So again, B must be true in the same cycle as A is true, cycle 1. If I were not true at cycle 2, P1 would vacuously pass, because I is part of the enabling sequence for the implication operator. If C were not true at cycle 3 or J not true at cycle 4, P1 would fail because they are the fulfilling sequence of the property.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog

Conjunction Property Operator

```
<property expr 1> and <property expr 2>
```

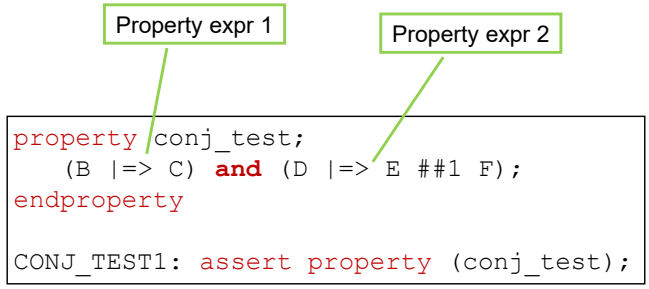
CONJ_TEST1 is a conjunction of 2 properties

CONJ_TEST1 succeeds where both of those 2 properties hold

- Either vacuously or non-vacuously

They are what I describe as WWYEWTDT properties

- Why would you ever want to do that?



211 © Cadence Design Systems, Inc. All rights reserved.

Property conjunction is an and where both operands are a property expression.

For property conj_test, at every cycle, properties (B |=> C) and (D |=> E ##1 F) must be true. This includes vacuously passing, where B and D are both false.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog

Why Conjunction Properties Are Not Pragmatic?

Disadvantages

- Complicates coverage analysis
  - Did both properties complete non-vacuously at the same time?
- If the property is asserted and fails, I need to spend time analysing which property expression(s) failed
- In formal, if the property is asserted and passes, I don't know both property expression(s) will always pass non-vacuously without more effort and risk
- In formal, you normally decompose the problem in order to run proofs in parallel to reduce runtime
  - i.e., have more than one property to describe the equivalent behaviour of the single property
- It does not do anything you couldn't easily describe in a less complicated way

```
property conj_test;
   (B |=> C) and (D |=> E ##1 F);
endproperty
CONJ_TEST1: assert property (conj_test);
```

≡

```
I_B_IMP_C:  assert property (B |=> C);
I_D_IMP_EF: assert property (D |=> E ##1 F);
```

Advantages

- I have never seen one since property `and` was first introduced in the 2005 LRM
- I do not think having a single property is a practical advantage

 © Cadence Design Systems, Inc. All rights reserved.

cadence

Property conjunction complicates coverage analysis.

If we assert the property and it fails, how do we know which property failed? Or if both properties failed? We would have to eliminate each possibility and spend more time debugging whether both fail or only one fails.

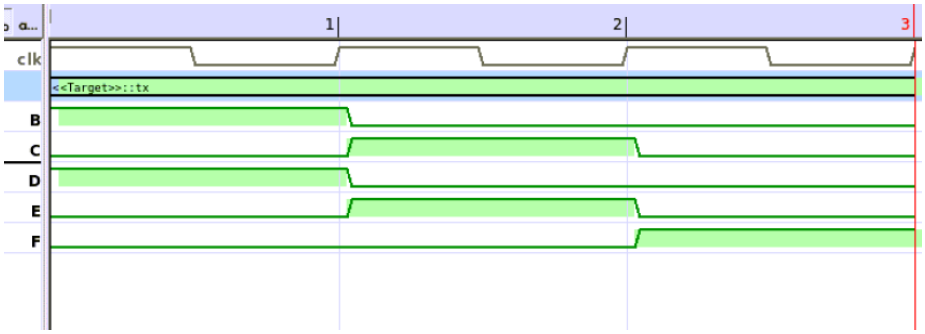
If we assert the property and it passes, we still need to evaluate whether these were vacuous passes or non-vacuous.

Again, as we have said before, in formal we decompose problems into separate multiple properties, rather than aggregating them. Aggregated properties take longer to prove and require more debugging if they fail. In formal, describing behaviour in multiple properties gives two key benefits. Firstly, it is less complex to debug and secondly multiple properties can be run in parallel to save time.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog

Example: Conjunction Property




```
P2: assert property ((B |=> C) and (D |=> E ##1 F);
```

The waveform shows an example (cover) of the property passing (called a witness)

- Green shading highlights what the signals values need to be for `P2` to pass

If `D` were not true at cycle 1 then this does not mean `P2` fails as the LHS would be incomplete (vacuous pass)

If `E` were not true at cycle 2 or `F` were not true at cycle 3 then this means `P2` fails

213 © Cadence Design Systems, Inc. All rights reserved.

cadence

So again, we use a Jasper witness to show the property passing. The green shading shows the required signal values for the assertion to pass. Unshaded cycles for a signal can be any value.

Note that a witness cover in Jasper is always going to show a non-vacuous pass because a vacuous pass is of no use.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog

Disjunction Property Operator

```
<property expr 1> or <property expr 2 >
```

DISJ_TEST1 is a disjunction of 2 properties

DISJ_TEST1 succeeds where either (or both) of those 2 properties hold

- Either vacuously or non-vacuously

They are what I describe as WWYEWTDT properties

- Why would you ever want to do that?

Property expr 1

Property expr 2

```
property disj_test;
   (B |=> C) or (D |=> E ##1 F);
endproperty

DISJ_TEST1: assert property (disj_test);
```

214 © Cadence Design Systems, Inc. All rights reserved.

Property disjunction is an or where both operands are a property expression.

Again, either property can succeed vacuously or non-vacuously. We only need one of the property operands to either vacuously or non-vacuously pass in order for the whole property to pass.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog

Why Disjunction Properties Are Not Pragmatic?

Disadvantages

- Complicates coverage analysis
  - Which property(s) complete non-vacuously?
- If the property is asserted and fails, I need to spend time analysing which property expression(s) failed
- In formal, if the property is asserted and passes, I don't know both property expression(s) will always pass non-vacuously without more effort and risk
- In formal, you normally decomposes the problem in order to run proofs in parallel to reduce runtime
  - i.e., have more than one property to describe the equivalent behaviour of the single property
- If it doesn't matter if one property expression fails, then why bother writing an assertion in the first place?
  - Non-trivial to write a set of properties equivalent to a *property* `or`
- It is normally better (and the actual user intent) to define a *single* property with the *sequence* `or` operator (which is different from the *property* `or`)

```
SEQ_OR_EG: assert property ( (B || D) |=> C or (E ##1 F) );
```

Advantages

- I have never seen one since property `or` was first introduced in the 2005 LRM
- I do not think having a single property is a practical advantage

 © Cadence Design Systems, Inc. All rights reserved.

cadence

Again, property disjunction complicates coverage analysis.

If we assert the property and it passes, how do we know which property passes? Is it possible that one property never passes? We still need to evaluate whether these were vacuous or non-vacuous passes. So, we have to spend time analysing the result.

Again, as we have said before, in formal we decompose problems into separate multiple properties, rather than aggregating them. Aggregated properties take longer to prove and require more debugging if they fail. In formal, describing behaviour in multiple properties gives two key benefits. Firstly, it is less complex to debug and secondly multiple properties can be run in parallel to save time

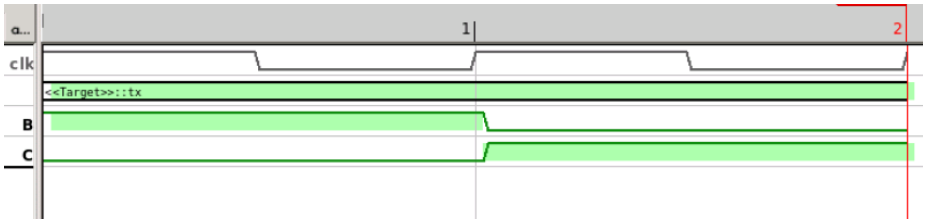
For a property or, either operand is a property, for example because it has an implication operator.

What we find is when people use property or, they really intended to use a sequence or but didn't code it correctly.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog

Example: Disjunction Property




```
P2: assert property ((B |=> C) or (D |=> E ##1 F);
```

- The waveform shows an example (cover) of the property passing (called a witness)
- Green shading highlights what the signal values need to be for `P2` to pass
- ***Note*** that signals `D`, `E` and `F` not shown as they are irrelevant for `P2` passing given the values of `B` and `C`

 © Cadence Design Systems, Inc. All rights reserved.

cadence

So again, we use a Jasper witness to show the property passing. The green shading shows the required signal values for the assertion to pass. Unshaded cycles for a signal can be any value.

The witness cover in Jasper shows just one operand passing, (B |=> C), because only one is required. Jasper will always show the shortest option, with the fewest cycles.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog

`iff` Operator

```
<property expr 1> iff <property expr 2>
```

Unlike implication (`|=>`, `|->`), `iff` is bi-directional

- Either **both** are true or **neither** of them are true

LRM defines these as being equivalent

```
prop1 iff prop2
```

≡

```
(prop1 implies prop2) and (prop2 implies prop1)
```

 © Cadence Design Systems, Inc. All rights reserved.

cadence

The iff property operator is different from the iff clocking expression and the disable iff to abort properties.

Each operand is a property. iff means either both of them are true or neither of them are true. In Boolean logic, that is like an xnor gate.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog

`nexttime` Property Operator

```
nexttime[index] <property expr 1>
```

The index is optional and must be a constant expression

`nexttime` and `nexttime[1]` are equivalent

There are multiple forms of the `nexttime` operator, combinations of weak/strong and [non-]indexed forms

- We discuss these later

They are what I describe as WWYEWTDT properties

- Why would you ever want to do that?

```
AST_D_NEXTT_E_2_F: assert property  (
      D |-> nexttime[1] E ##1 F );
```

Indexed

Both properties are exactly equivalent

```
AST_D_NEXTT_E_2_F: assert property  (
      D |-> nexttime E ##1 F );
```

Non-indexed

218 © Cadence Design Systems, Inc. All rights reserved.

cadence

PSL had nexttime operators or next operators, so presumably these were added to SVA for consistency.

nexttime can be indexed with a constant expression, but the index is optional and defaults to 1. So, both these properties are the same, as an unindexed nexttime defaults to 1.

Effectively nexttime[1], or nextime, is equivalent to ##1.

nexttime returns a property expression, which allows you to return a property expression where you otherwise might not be able to, but this is of questionable practical use.

There are multiple forms of the nexttime operator, as we will see.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog

nexttime Property Example: Weak Indexed Form

If D occurs then after 2 cycles, E is required to be true and F must be true in the cycle after that

D_NEXTT_E_2_F_W is the **weak indexed** form which means:

- If D occurs, and verification finishes the next cycle then E or F are not obliged to occur

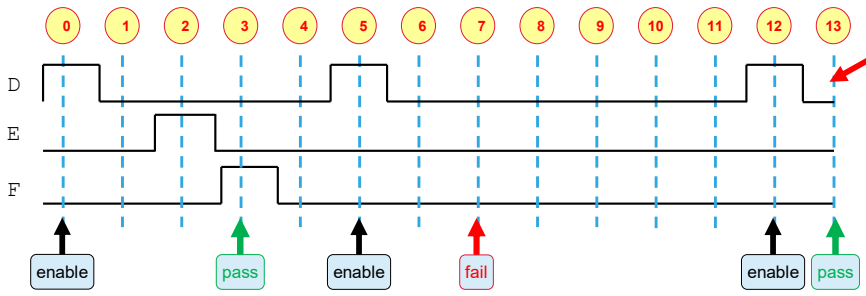
```
nexttime[index] <property expr 1>
```

Weak indexed form of nexttime

```
D_NEXTT_E_2_F_W: assert property
  ( D |-> nexttime[2] E ##1 F );
```

Equivalent property described with sequences

```
D_NEXTT_E_2_F_W_SEQ: assert property
  ( D |-> ##2 E ##1 F );
```



Verification ends at cycle 13

Verification ended means:

1. **Simulation** – We stopped the simulation
2. **Formal** – [over]constraints mean that there are no further cycles to explore

219 © Cadence Design Systems, Inc. All rights reserved.

cadence

The example is a weak, indexed form of nexttime. Weak means if the property is enabled, but the verification ends before the fulfilling condition completes, this is not a failure. So, E or F are not required to occur.

We can write a sequence equivalent for property D_NEXTT_E_2_F_W which is the easiest way of understanding nexttime. nexttime[2] gives the effect of ##2.

The property is enabled by D at cycle 0. We have E 2 cycles later at cycle 2 and the property passes on cycle 3 with F.

D at cycle 5 enables the property, which fails at cycle 7 because there is no E.

D at cycle 12 enables the property, and the verification ends at cycle 13 where the property passes, because it is weak.

How can verification end? For simulation, we simply stop the simulation, through, for example, the UVM objection mechanism terminating the run phase. In formal, constraints mean there are no further cycles to explore.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog

`s_nexttime` Property Example: Strong Indexed Form

If `D` occurs then after 2 cycles, `E` is required to be true, and `F` must be true in the cycle after that

`D_NEXTT_E_2_F_S` is the strong indexed form which means:

- If `D` occurs, then `E` and `F` are obliged to occur

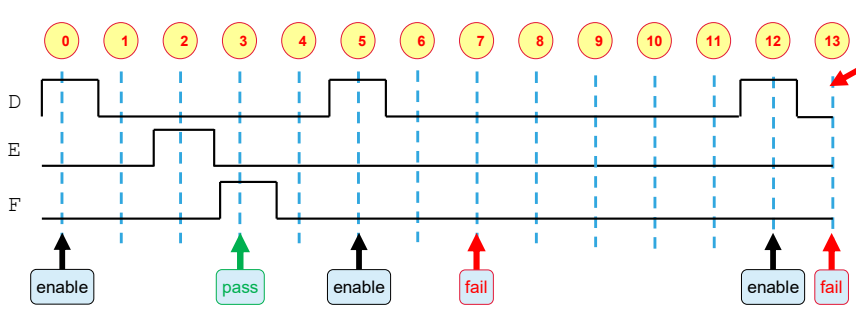
```
s_nexttime[index] <property expr 1>
```

Strong indexed form of `nexttime`

```
D_NEXTT_E_2_F_S: assert property
  ( D |-> s_nexttime[2] E ##1 F );
```

Equivalent property described with sequences

```
D_NEXTT_E_2_F_S_SEQ: assert property
  ( D |-> strong(##2 E ##1 F) );
```



Verification ends at cycle 13

Verification ended means:

1. **Simulation** – We stopped the simulation
2. **Formal** – [over]constraints mean that there are no further cycles to explore

220 © Cadence Design Systems, Inc. All rights reserved.

The strong form is s_nexttime and the sequence equivalent has strong surrounding rest of the expression. Strong means the sequence has to complete. Therefore, if we have D, F must occur.

The waveform has the same results as the previous slide, except in cycle 13, when the verification ends, we had D, without the fulfilling condition completing, so the property fails.

The property is enabled by D at cycle 0. We have E 2 cycles later at cycle 2 and the property passes on cycle 3 with F.

D at cycle 5 enables the property, which fails at cycle 7 because there is no E.

D at cycle 12 enables the property, and the verification ends at cycle 13 where the property fails, because it is strong.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog

Why `nexttime` Operators Are Not Pragmatic?

Disadvantages

- You need to remember how they behave, including all different forms
  - This is not convenient, easy to make mistakes and invites human error
- They don't do anything that you could describe in a simpler and more intuitive way

```
D_NEXTT_E_2_F_W: assert property
  ( D |-> nexttime[2] E ##1 F );
```

=

```
D_NEXTT_E_2_F_W_SEQ: assert property
  ( D  |-> ##2 E ##1 F );
```

Advantages

- I have never seen one since `nexttime` was first introduced in the 2009 LRM
  - I do not think one could even argue that it's more readable

 © Cadence Design Systems, Inc. All rights reserved.

cadence

A disadvantage of nexttime is you have to remember how all the various forms behave. Index, non-index, strong or not strong. nexttime does not do anything you couldn't do with a sequence, in a simpler and more intuitive way.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog

Summary

- Not all property forming constructs or operators are useful.
- With some exceptions, the operators can often be described in a more straightforward and standard way using sequences.
- `until` operators are powerful but confusing, and sequence forms are usually easier to understand.
- `always` is extremely useful for preventing input livelock in formal verification.
- `Followed-by` operators are best used for liveness covers, not for assertions.
- Many SVA constructs exist in the LRM but add little value, so simpler alternatives are preferred.

 © Cadence Design Systems, Inc. All rights reserved.

cadence

Just because a language construct is defined in the LRM it does not the construct is useful, or that there is not an easier alternative.

There are some constructs which are useful in specific cases:

always for input livelocks.

Followed-by for covers.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog

Lab



Lab 5 Property Forming Constructs-Based Lab

- In this lab, you will define properties using constructs that return a property expression rather than a sequence expression.

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog



Module 12

The Verification Completeness Problem

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog

Module Objectives

In this module, you

- Assess verification methodologies
- Analyze a case study that highlights the verification completeness problem
- Identify techniques for minimizing the risk of verification holes
- Identify the merits of each verification technique

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog

Scale of the Functional Verification Problem



Complex processor IC designs can, for instance, contain around 500,000 Flip-Flops (FFs).

Those FFs could be in any of 2^{500000} different states at any particular time

The actual number of states will be much less because

- By design, not all states will be possible
- There will be many states we don't particularly care about

However, the number of states is still absolutely massive

It is essential that you accept this truth

You can not possibly test everything

226 © Cadence Design Systems, Inc. All rights reserved.

cadence

The scale of the problem is really at the root cause of why you cannot possibly test everything.

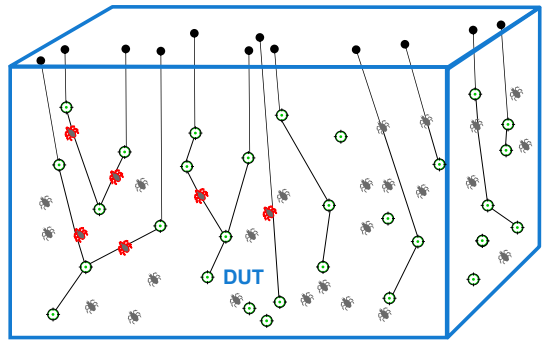
The most complex ICS of today may contain 500,000 flip flops. 500,000 flip flops gives you two to the 500,000 maximum possible states at any particular time. That theoretical maximum will never be achieved because, by design, many of these states will not be possible and there are also many states we do not particularly care about because they don't represent real life operation for the design. Even so, we've still got a huge state space to explore. The first thing to accept is, as a simulation user, you can't possibly test everything. The problem then becomes one of minimising the risk of bugs escaping verification.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog

Directed Testing: The Traditional Approach to Verification

Defines DUT states to be tested based on specified behavior and corner cases. Writes directed test to visit and verify each item in the test plan.



Redo if design changes

- Automation — Significant manual effort to write all the tests
- Automation — Work required to verify each goal was reached
- Completeness — Poor coverage of non-goal scenarios … especially the cases that you didn't "think of"

 © Cadence Design Systems, Inc. All rights reserved.

cadence

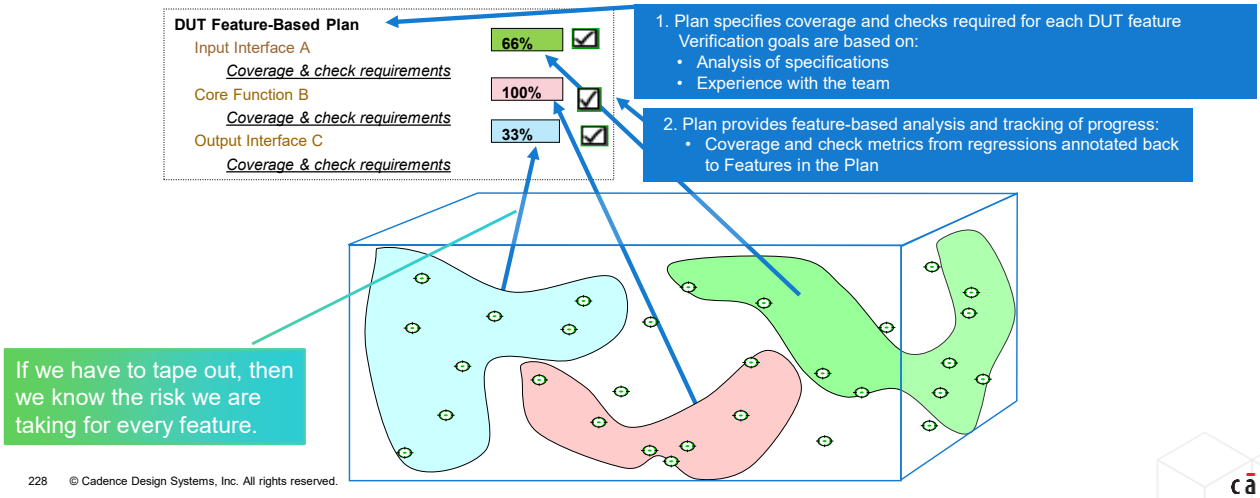
The history of verification began with specific directed testing, which involved conceptualizing and applying stimuli to a design, then observing the outputs. This method was labor-intensive, requiring manual test creation and execution, and starting over with any design changes. Additionally, it had poor coverage of non-goal scenarios, making unforeseen issues the biggest challenges.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog

What Is Metric Driven Verification (MDV)?

MDV emphasizes coverage structured within a hierarchical verification plan. This method outlines a feature-based approach comprising a detailed list of checks to verify that the design's functionality meets expectations.



228 © Cadence Design Systems, Inc. All rights reserved.
cadence

The verification methodology has evolved to prioritize coverage and is structured within a hierarchical verification plan. This approach outlines a feature-based strategy that includes a comprehensive list of checks necessary to ensure the design's functionality meets expectations. The plans are developed based on the team's experience and an analysis of the functional specifications, guaranteeing that all required features are thoroughly evaluated. The core goal of any verification process, whether formal or through simulation, is to achieve the highest possible coverage, ideally reaching 100%, and ensuring that no checks fail. We can view these metrics as indicators of the level of risk we take if we proceed with tapeout today. Ultimately, verification is about managing risk.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog

Concise and Simplistic Summary of MDV

Define a hierarchical verification plan that:

- States the features the design is supposed to have
- Specifies the checks to be done in order to test those features
- Specifies the coverage required in order to be confident that we tested every important scenario the design is expected to work under

We know we have finished verification when we reach 100% coverage with no checks failing.

229 © Cadence Design Systems, Inc. All rights reserved.

cadence

To effectively convey what MDV is and its significance in the simulation world, it's acknowledged as the most efficient methodology. A hierarchical verification plan is developed, outlining the expected features of the design derived from the functional specification.

This plan specifies the necessary checks to test these features and the required coverage to ensure confidence in the results.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog

The Problem of Verification Completeness



A verification plan, even when fully implemented and supported by tools, does not guarantee correct or complete verification. Human error can leave functional gaps undetected.

The plan itself may be flawed, misinterpreted, or insufficient, leading to gaps in stimulus, checks, or coverage.

To show how this can happen for even the simplest of designs, we will follow a case study:

- The schematic we will show defines the functional specification.
- The intent is to verify every feature of the design by defining SVA properties.
- We will verify these properties using Jasper® Formal Verification.

 © Cadence Design Systems, Inc. All rights reserved.

cadence

However, just because we have a plan doesn't mean everything is going to be OK. The plan might be wrong. The plan is susceptible to the human errors like everything else. The plan might be correct but not implemented correctly. There may be a problem with the stimulus, the checks and/or the coverage measures. Everything could still go wrong.

We will illustrate this in a case study. We will show a simple schematic which defines the functionality required. We will write assertions to verify every feature of the design has and use the Jasper formal verification tool to verify the properties.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog

Coverage Does Not Mean Completeness

- Properties are typically derived from the verification plan
- This is true for both formal verification and coverage-driven simulation: did we specify enough coverage?
- Passing all properties can still give a **false sense of completeness.**
- Tools may report that:
  - all I/Os and flip-flops are covered
  - no dead code exists
- This does not mean that we have written properties to cover all functionality
  - In the same sense that code coverage does not tell us we have complete functional coverage

 © Cadence Design Systems, Inc. All rights reserved.

cadence

We will aim to write sufficient properties, derived from the verification plan, to check the behaviour is correct.

The reasoning we will go through with this design does not necessarily only apply to formal. You could make similar arguments for simulation. If we have a constrained random stimulus testbench, and our goal is to maximise coverage, how do we know our coverage measures are correct? How can we be certain they cover all the required functionality?

Formal can give us some help. A formal tool could report that every I/O and every flip flop is covered in the fan-in cone of some assertion. However, this does not mean we have written enough properties to cover all functionality. Even if the formal metrics tells us everything has been covered. Just in the same sense that code coverage does not tell us we have complete functional verification.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog

False Sense of Verification Completeness

Shown is a schematic of the functional requirements that are implemented in RTL:

- We write blackbox properties, which we think completely checks all desired functionality
- Human error or oversight might mean that we don't specify all desired behavior

Tool might tell us

- All I/Os and all FFs are included in some assertion:
- There is no deadcode
- All checks pass
- All cover items pass

Should we conclude that verification is complete?

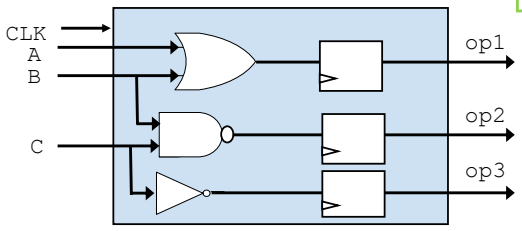
- No way of knowing that we specified enough properties to check all functionality
- Human error or oversight might escape detection in the verification process

At first glance these properties seem sensible, however ….

```
P1: assert property ( (A || B) |=> op1 );
P2: assert property ( (!A && !B) |=> op2 );
P3: assert property ( !C |=> op3 );
```

Human error, `A` has no effect upon `op2` but property still holds

a	b	c	op1`	op2`	op3`
0	0	0	0	1	1
0	0	1	0	1	0
0	1	0	1	1	1
0	1	1	1	0	0
1	0	0	1	1	1
1	0	1	1	1	0
1	1	0	1	1	1
1	1	1	1	0	0



232 © Cadence Design Systems, Inc. All rights reserved.

cadence

This schematic defines the design requirements. We write a property set that we hope will check all the design behaviour.

At first inspection these look like a reasonable set of properties. P1 says if we have A or B, then on the next cycle op1 should be true. That would appear to check the or gate and the op1 flip flop. Likewise for the other outputs. However, human error or oversight might mean we don't capture the correct behaviour.

P2 says if not A and not B, then on the next cycle op2 is true. But if we check the schematic, A does not appear in the fan-in cone for op2. However, it just so happens, that P2 will always be true. You can check the Truth Table to confirm this. So even though the tool might report all I/O and flip flops are included in some assertion, and that there is no branches of code which cannot be reached, and that all checks pass, and all cover items pass, (i.e. 100% coverage in the context of no checks failing), which we defined as the end of verification, human error means we still have not verified the design correctly.

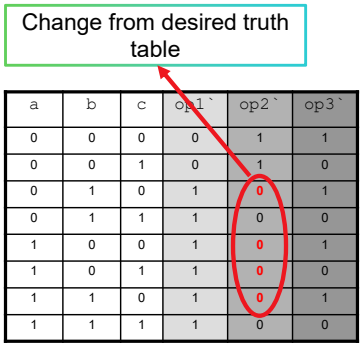
© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog

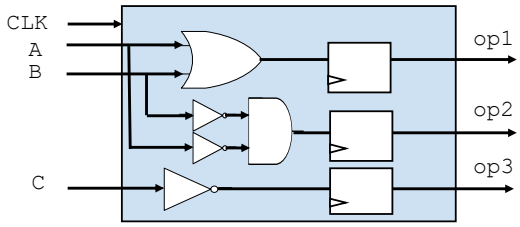
Stronger Failure: Design Mutations That Still Pass

Shown is a schematic whose behavior satisfies our properties. This is a mutant of the desired design; our verification environment fails to detect this mutation of our desired design.

Hence, we did not capture a property set that completely and uniquely defines desired behavior.




```
P1: assert property ( (A || B) |=> op1 );
P2: assert property ( (!A && !B) |=> op2 );
P3: assert property ( !C |=> op3 );
```



233 © Cadence Design Systems, Inc. All rights reserved.

cadence

If we make a change to the design, we might expect the properties to fail, if they completely define the required behaviour. This is called design mutation. We have changed the schematic and it just so happens that the change will not be detected by any of these assertions. If the properties all passed for the previous schematic, they will all pass for the mutated design. So, we have managed to change the design without the properties detecting this.

This raises an interesting philosophical question. Do we really want a property set that completely and uniquely defines the desired behaviour? If our properties have captured all the desired design behaviour, do we really care if the design changes and none of the properties fail? We have captured all of the design behaviour that is important. This is our decision because the tool cannot read our minds.

We could detect the mutant design if the properties are sufficient.

Another issue is that in practice, the mutation might not propagate an observable difference from the expected behavior to somewhere it could be detected, i.e. the outputs included in our assertions.

In simulation, this could also indicate a problem with stimulus. We did not provide the correct stimulus to observe the difference in behavior caused by the mutation.

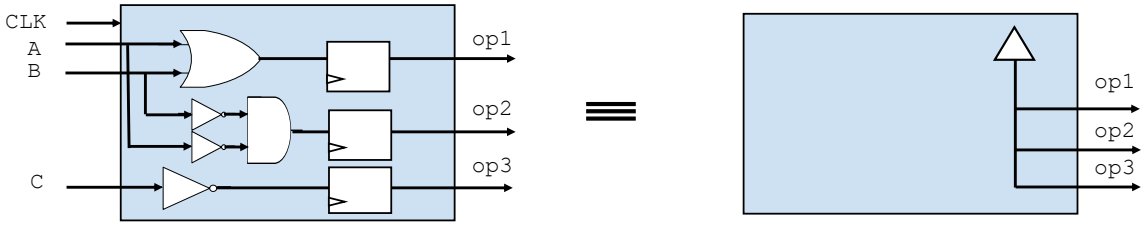
© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog

Solution: Fixing the Fundamentals
Stronger Assertions

All these properties pass with the outputs tied to the power rail

```
P1: assert property ( (A || B) |=> op1 );   ☑
P2: assert property ( (!A && !B) |=> op2 ); ☑
P3: assert property ( !C |=> op3 );         ☑
```



The correct assertion to check `OP1` is not particularly intuitive

```
P1: assert property( op1 == $past(A || B)); ☑
```

 © Cadence Design Systems, Inc. All rights reserved.

cadence

In fact our problem is much more serious than failure to detect a design mutation. We do not even have the fundamentals correct. All our properties still pass if all the outputs are tied to the power rail. Instead of the required left-hand schematic, a designer could present the right-hand version and all the properties would pass.

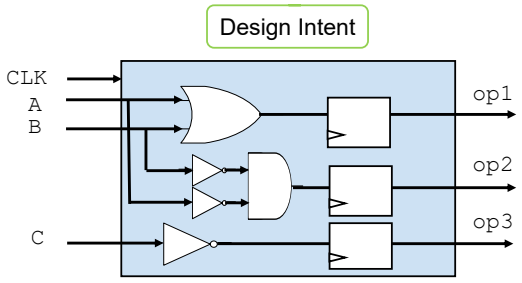
The solution is to use a Boolean expression without implication, which is not very intuitive. For example, solution property P1 states op1 must always equal the past value of the expression (A || B). The expression captures the logic for the output and $past, one cycle ago, allows for the flip flop.

© Cadence Design Systems, Inc. All rights reserved.

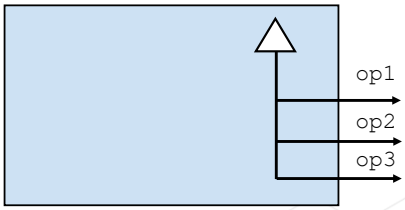
This document is for the sole use of Rajat G of Analog

Solution: Fixing the Fundamentals
Fixing What Assertions Miss

- Our assertions didn't check what we intended.
- We likely also missed writing meaningful user-defined covers for this design.
- So, the goal becomes risk reduction: lower the chance that such bugs escape verification.
- Which Jasper features can help detect this?
  - Superlint Auto Formal Toggle check
  - Jasper Coverage App
    - None of the designs would be in the proof core



Buggy design implementation upon which our assertions still pass




```
P1: assert property ( (A || B) |=> op1 );  ☑
P2: assert property( (!A && !B) |=> op2 ); ☑
P3: assert property( !C |=> op3 );         ☑
```

235 © Cadence Design Systems, Inc. All rights reserved.

How can we catch the problem that our assertions didn't check the correct behaviour.

We need to think in terms of risk management. Lowering the probability that this, or any bug escapes verification.

How could we detect this problem?

Toggle checks on the three outputs will show that these signals can never change.

The Jasper coverage app will tells us which parts of the design were actually used in proving an assertion or achieving a cover statement. This will show that we tested nothing inside the buggy design implementation.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog

Can We Achieve Completeness?



Is it impossible for a tool to read our minds? Probably, but we can get closer using tool features. No current tools can tell you when you have a complete property set.

There is no substitute for:

- Understanding the requirements
- Intelligent planning
- Attention to detail
- Hard work

No EDA tool can tell you that your verification is complete.

All the EDA tools can tell us is if we didn't test all the things that we said we would, namely if we are incomplete.

 © Cadence Design Systems, Inc. All rights reserved.

cadence

No EDA tool can tell us that we have got complete verification.

There is no substitute for doing all the things only humans can do, which are listed on the slide.

However, what an EDA a tool can tell us is we didn't test all of the things that we said we would, which at least gets us part of the way towards the understanding of whether we're complete or not.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog

Solution: Assessing Completeness

Defining your verification objectives upfront is the best way to assess when you are done

Beyond this, there are other ways to assess incompleteness:

- Do all the outputs of the block have assertions describing their behavior?
- Jasper Coverage App can help assess which portion of the design your assertions are exercising:
  - Cannot tell you when you are done, but when you are not done
- There are no tools that can tell you when you have done enough verification at present

 © Cadence Design Systems, Inc. All rights reserved.

cadence

We need to define, in advance, what we consider as a complete set of tests. Then we can measure against this plan in assessing completeness. EDA tools can then tell you whether you are incomplete in your verification.

Tools can help further in assessing completeness. They can tell you if all the outputs of a block have assertions describing their behaviour. Jasper coverage app can tell us what parts of the design were and were not exercised during all proofs and all covers.

Essentially the tools can tell us we are not finished, but never that we are. There will always be more to test. You cannot test everything, as we discussed at the beginning of this module.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog

Verification Execution Completeness

This is something that we can know

The goal is the same in both dynamic (simulation) and formal

All check items pass/proved in context of coverage items *equals* verification complete

Execution Method	Execution Item	Completeness
Dynamic	Check	All checks are executed with no violation (checks pass)
	Coverage	All coverage items are reached (pass); **driver** + DUV dependent

Execution Method	Execution Item	Completeness
Formal	Check	All checks are proven with no violation (checks proved)
	Coverage	All coverage items are reachable; **constraints** + DUV dependent

 © Cadence Design Systems, Inc. All rights reserved.

cadence

All we know is how much we have executed.

In dynamic verification, simulation and emulation, our requirements in are typically to define checks and coverage, which are all the expected scenarios under which the design is expected to operate. Our goal is 100% coverage in the context of all checks passing.

In formal, that's exactly the same, although in formal coverage means a slightly different thing. The coverage in formal tells you whether that scenario is ever possible. But it is the same situation. In formal our checks without coverage are useless. We need 100% coverage in the context of no checks failing.

We can know for sure that all checks we have defined have passed and we have reached all cover items.

The holy grail of verification is knowing whether those checks and cover items defined all behaviours which we want to verify.

This might be possible in the future but not at the present time.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog

What Can Be Automated?



It is probably more important to ask the question in reverse, what can't be automated:

- Writing the functional specification
- Writing the Verification Plan
- Interpreting the verification plan to implement the checks and coverage
- Interpreting the results from verification runs
- Understanding the limitations of the verification techniques we use

EDA is about automating all that can be, for example:

- Managing huge amounts of data
- Compilation/recompilation
- Running and rerunning verification tools after bug fixes
- Analysing which tests are the most valuable in terms of coverage achieved
- Tracking bug rates/coverage accumulation rates

 © Cadence Design Systems, Inc. All rights reserved.

cadence

A more important question, and a more realistic one, is what cannot be automated.

Writing the functional spec in the first place and the verification plan based on the spec.

Interpretation of that verification plan into the individual checks and coverage.

Interpreting in the context under which the results were received during the different verification runs

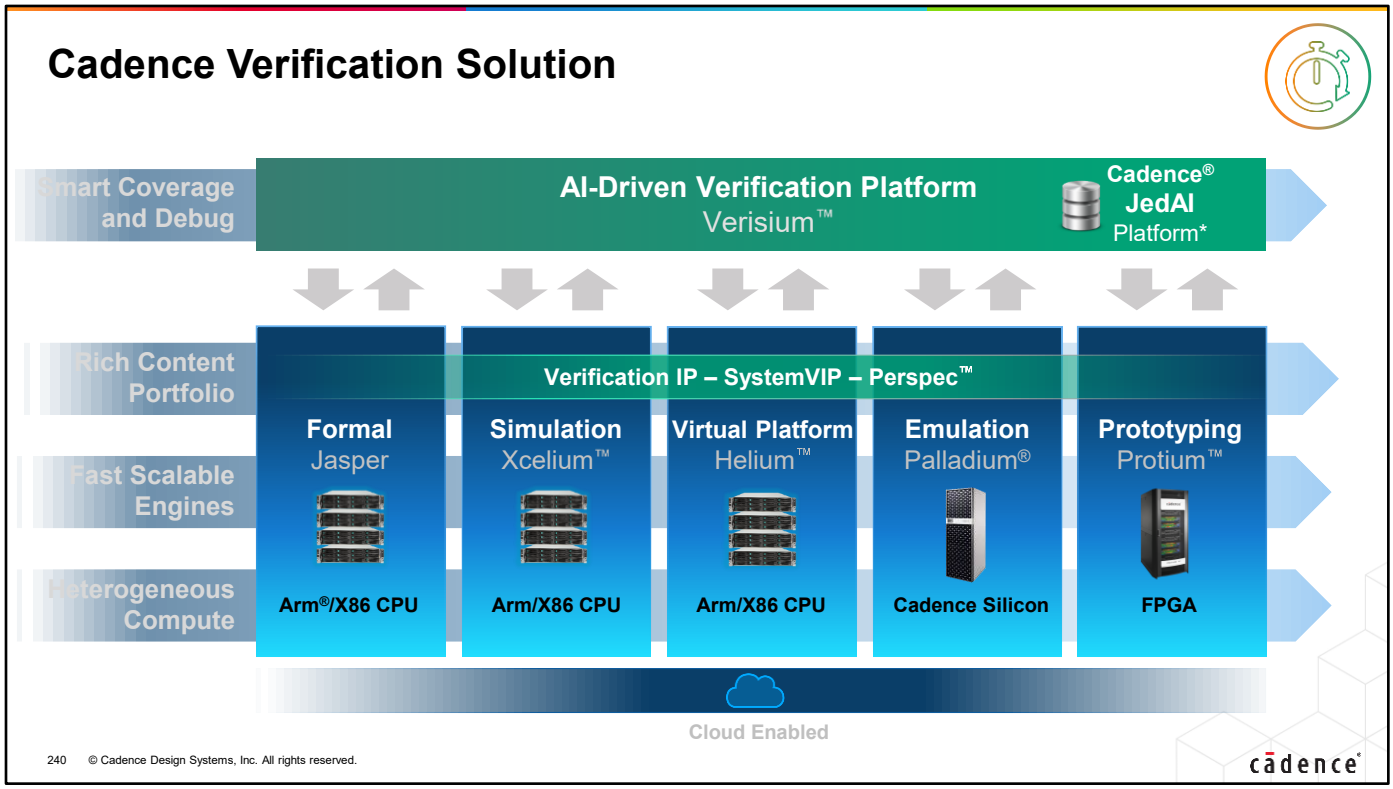
Understanding the limitations of any technology we're using.

The final point is why different verification technologies exist like formal, simulation, hardware acceleration, emulation, FPGA prototyping and so on. We have all these different methods of verification because not one methodology or technique does everything. There is no single solution.

EDA is about automating all that we can. Huge designs create huge amounts of data to be managed. Efficient compilation and recompilation is essential. Executing verification runs and if they fail, re-executing them after bug fixes. Analysing which tests give us the most information in the shortest time. Tracking bug rates and our coverage accumulation. Those are the kinds of things at which EDA can excel.

© Cadence Design Systems, Inc. All rights reserved.

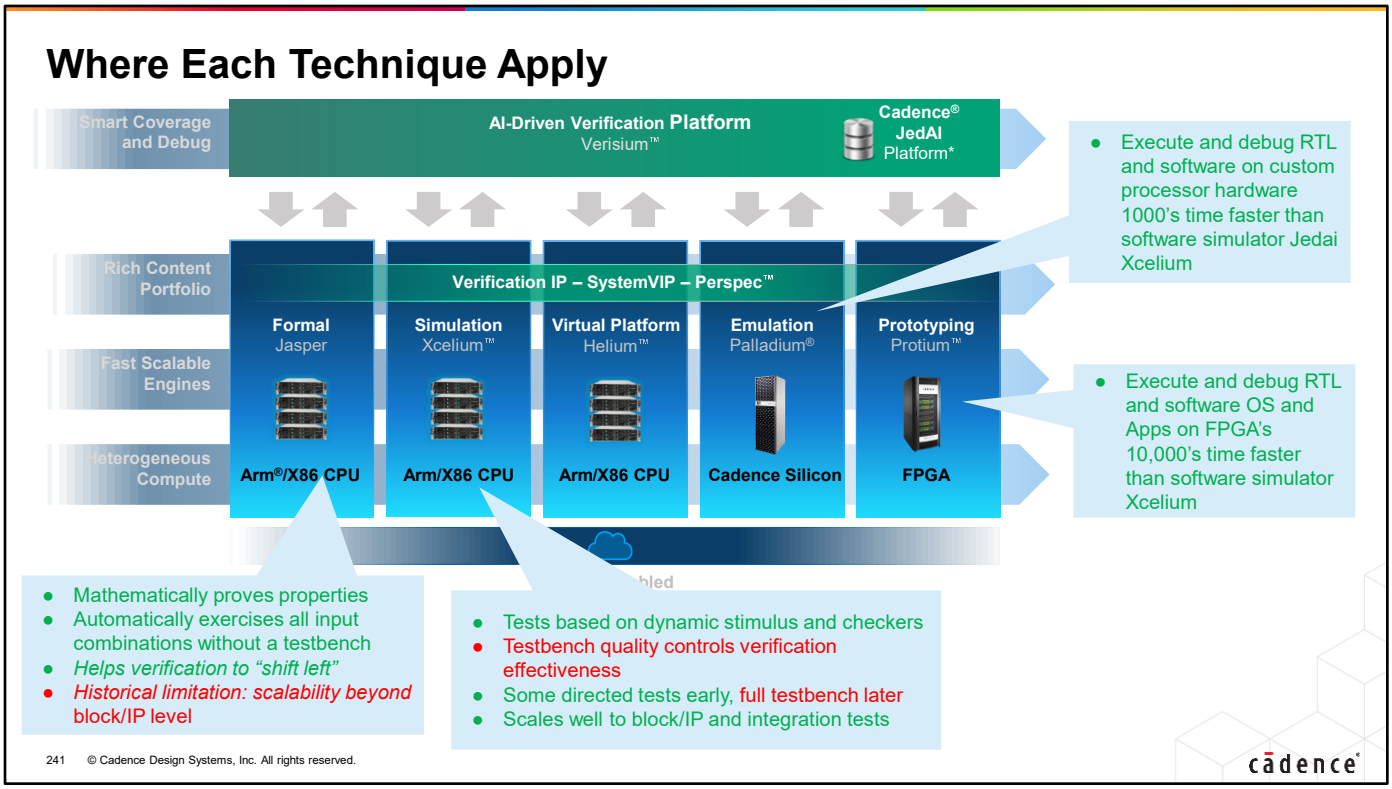
This document is for the sole use of Rajat G of Analog



Cadence has a verification suite for a significant reason. Various techniques are appropriate at different levels of integration. This includes formal and static verification, mixed signal, low power, and functional safety. All these elements must be addressed, and it's essential to understand how each technique fits into the overall process.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog



Formal verification encompasses mathematical proofs and algebra proofs, exercising all input combinations without needing a test bench. However, a historical limitation of formal verification is its scalability beyond the block and Intellectual Property (IP) level.

If exhaustive proofs are the goal, simulation stands as the oldest form of verification. Initially, verification predominantly relied on directed tests, but the current trend has shifted toward a metric-driven verification environment.

Emulation offers a solution by allowing processes to run thousands of times faster than traditional software simulators like Xcelium.

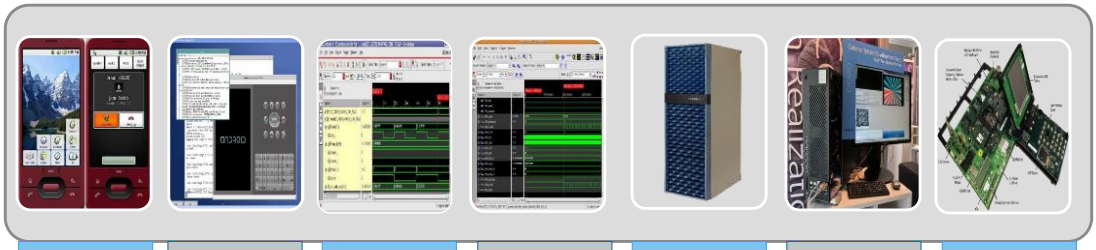
Prototyping is employed to debug the operating system (OS) and software, offering speeds tens of thousands of times faster than software and simulation.

Prototyping, by utilizing an FPGA array, effectively allows for the entire chip to be prototyped, facilitating software running in real-time.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog

There Is No "One Size Fits All"
Verification and Software Platforms Need to Interoperate



SDK OS Simulation	Virtual Platform	Formal Analysis	HDL Simulation	Acceleration Emulation	FPGA Prototype	Prototyping Board
●Highest speed ●Earliest in the flow ●Ignore hardware	●Almost at speed ●Less accurate (or slower) ●Before RTL ●Great to debug (but less detail) ●Easy replication	●Less-scalable ●Exhaustive ●Early RTL ●Great for IP ●No SW execution	●KHz range ●Accurate ●Excellent HW debug ●Broadly available ● Mixed-abstractions ●Limited SW execution	●MHz Range ●RTL accurate ●After RTL is available ●Good to debug with full detail ●Expensive to replicate	●10's of MHz ●RTL accurate ●After stable RTL is available ●OK to debug ●More expensive than software to replicate	●Real time speed ●Fully accurate ●Post Silicon ●Difficult to debug ●Sometimes hard to replicate

242 © Cadence Design Systems, Inc. All rights reserved.

cadence

There is no magic silver bullet or one-size-fits-all solution.

Initially, the process involves software simulations using less accurate models.

HDL simulation operates as if it has a clock frequency in the kilohertz range, making the simulation of even a second's worth of real time quite lengthy.

Acceleration and emulation techniques become valuable at this juncture because they can achieve speeds in the megahertz range, simulating a 1 megahertz clock.

FPGA prototyping further advances the process, achieving speeds in the tens or hundreds of megahertz.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog

Summary

- Functional Verification focuses on risk management, not on verifying everything
- The goal is to minimize the chances of critical bugs escaping the verification process
- Multiple tools and methodologies support this goal
- Verification responsibility cannot be delegated entirely to automation
- Automation assists with data management, repetitive tasks, and computation
- Functional Verification remains a human-driven process, and is therefore prone to human error
- Consequently, no single verification methodology is optimal for all situations, all the time

 © Cadence Design Systems, Inc. All rights reserved.

cadence

It is important to understand that functional verification is not about completeness. It is about risk management. We cannot possibly test everything. We just concentrate on the features which are important to us and try to minimise the chances of undetected bugs.

To help us, we have can many different tools and methodologies. Automation can help with repetitive processes and data management and analysis. There is no single verification tool which is best for every circumstance. Each has their own advantages and disadvantages. There is no alternative to large numbers of experienced humans.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog



Module 13

Course Conclusions

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog

What Have We Learned ?

Motivation and methodology of using ABV

How to define Sequential and Boolean properties

Language features and methodologies for property reuse

- Including from formal to simulation and vice versa

Motivation and methodology of defining coverage with SVA

How to correctly define Liveness and Safety properties

Motivation and methodology of using Auxiliary (HDL helper) code

 © Cadence Design Systems, Inc. All rights reserved.

cadence

To review what we learned in Part 1:

We learned about the motivation and the methodology of using Assertion Based Verification (ABV).

How to define properties and sequences.

How we can use SVA language features and the ABV methodology to reuse properties and create Verification Components (VCs) .

Checks and coverage are both needed regardless of whether we're using formal or simulation.

How to define liveness properties and the difference between liveness and safety, and the correct way of defining them to avoid backward compatibility problems.

We frequently need auxiliary code for situations where SVA itself is not sufficient.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog

SVA Issues

SVA syntax looks easy but the application is not.

- The syntax is “sugared” and meant to look easy
- The concepts are hard

Not all common behaviors can be checked with SVA alone

Aux code often required to:

- Make properties simpler to express
- Do things that assertion languages cannot express
- Reduce “complexity” for Formal

Special Techniques can be applied when using Formal

- These are not compatible with Simulation

 © Cadence Design Systems, Inc. All rights reserved.

cadence

SVA is deliberately defined to look easy, but the concepts are hard. For example, dealing with overlapping and all the different ways an assertion can pass and fail.

A surprising number of seemingly simple and common behaviours can be checked with SVA alone.

So we need auxiliary code, which can also simplify formal analysis and simplify property writing.We should avoid writing complicated properties which are hard to maintain, hard to understand and hard to debug. We should decompose problems into multiple properties and use simple auxiliary code and, simple properties to solve problems.

We also have specific considerations for formal, requiring special techniques which are not compatible with simulation. Formal has unique concepts for which there are no equivalents in simulation.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog

SVA Part 1 (Formal and Simulation Users) Summary

- We introduced Assertion-Based Verification (ABV) and the role of SVA in describing design behavior.
- We understood the roles of assertions, assumptions, and covers.
- We learned how SVA properties are independent of simulation and formal tools.
- We learned the core SVA constructs: boolean properties, sequences, implications, clocking, and disable conditions, to model temporal behavior correctly.
- We distinguished safety and liveness properties, and where each applies.
- We recognized that SVA alone is not always sufficient, and why auxiliary code is required to handle overlapping, efficient counting.

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog

SVA Part 2 (Simulation Users Only) Summary

We showed how to describe complex sequence relationships using the sequence operators.

However, being able to describe complex sequences is a double-edged sword.

- They are hard to define correctly and debug
- For formal, they create proof complexity issues

We saw some advanced SVA features to describe complex behavior, such as local variables and sequence endpoints.

SVA has many operators and constructs that return a property expression.

- Not all of them are of pragmatic use
- They often overcomplicate things that can already be achieved using sequences in a simpler way

We saw the challenge and know you have achieved a complete property set for verification.

 © Cadence Design Systems, Inc. All rights reserved.

cadence

To review what we learned in Part 2:

We learned how to describe complex sequence relationships with the many different sequence operators. However, these, and other features like nested sequences, are, in practise, really hard to define correctly, to debug and to know your intent was implemented correctly. For formal these create also proof complexity issues.

We saw several advanced SVA features such as local variables and endpoints, and we showed the two practical uses of local variables – deserialization and data checking.

We discussed the many operators which return a property expression. Some of them useful and some which are not.

We saw the problem of completeness in verification. This is universal problem, not just for SVA, formal or simulation. Verification in general is hard. If it was easy and could be automated, then we would not need designers or verification engineers because some tool could just do it all. That is clearly not the case and never will be in the foreseeable future.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog

Reference

SystemVerilog Language Reference Manual

- www.ieee.org
- IEEE 1800-2023

Refer to the link below for Jasper University for all the Jasper App and Formal Verification RAK

- Jasper University
- ASK

Textbooks

- *"Formal Verification: An Essential Toolkit for Modern VLSI Design"* by Erik Seligman
- *"SystemVerilog Assertions Handbook"* by Cohen *et al*

 © Cadence Design Systems, Inc. All rights reserved.

cadence

The latest IEEE-1800 standard for SVA is 2017, which seems a long time ago, but SVA is a fairly stable language now.

The excellent, recommended textbook is Formal Verification by Eric Seligman. This is a really practical guide to using SVA and verification in general.

Another popular book is SystemVerilog Assertions by Ben Cohen. This is more of a language reference guide, rather than an overall methodology. It is not as practical as the Eric Seligman book. It describes all the language features, but does not really tell you what is useful and what is not.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog



Module 14

Next Steps

cadence

This page does not contain notes.

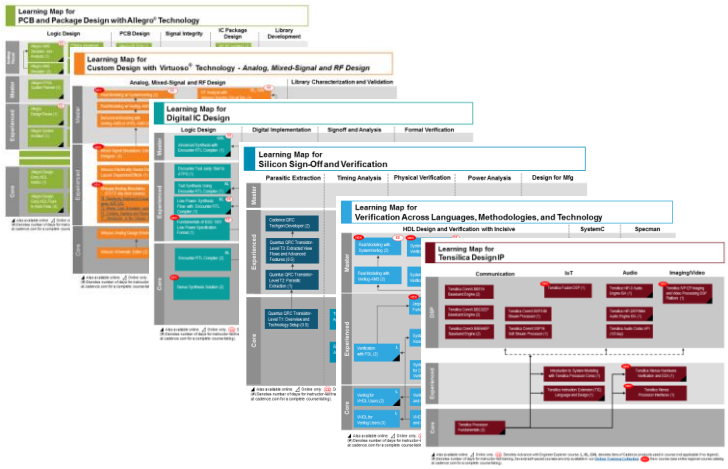
© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog

Learning Maps

Cadence® Training Services learning maps provide a comprehensive visual overview of the learning opportunities for Cadence customers.

Click here to see all our courses in each technology area and the recommended order in which to take them.



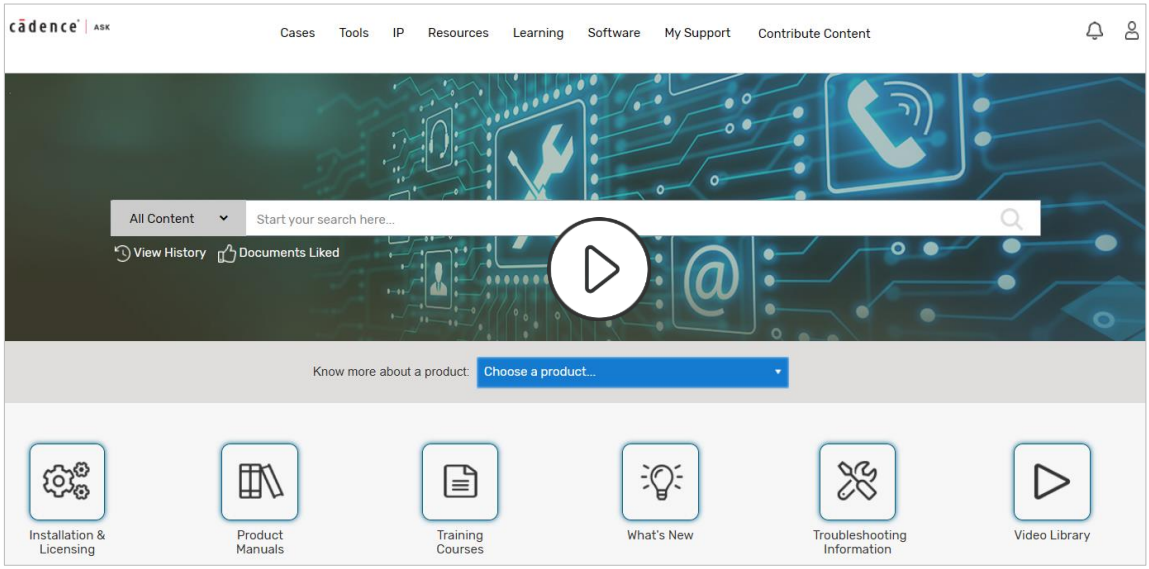
 © Cadence Design Systems, Inc. All rights reserved.

Go here to view the learning maps:
http://www.cadence.com/Training/Pages/learning_maps.aspx

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog

Cadence Learning and Support – ASK Portal



Cadence Ask now includes over 2000 product/language/methodology videos ("Training Bytes")!

252 © Cadence Design Systems, Inc. All rights reserved.

Click here to view the demo of Cadence Learning and Support – ASK Portal

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog

Wrap Up

- Complete Post Assessment, if provided
- Complete the Course Evaluation
- Get a Certificate of Course Completion
- Complete the Course's Digital Badge exam to earn your Badge

Thank you!

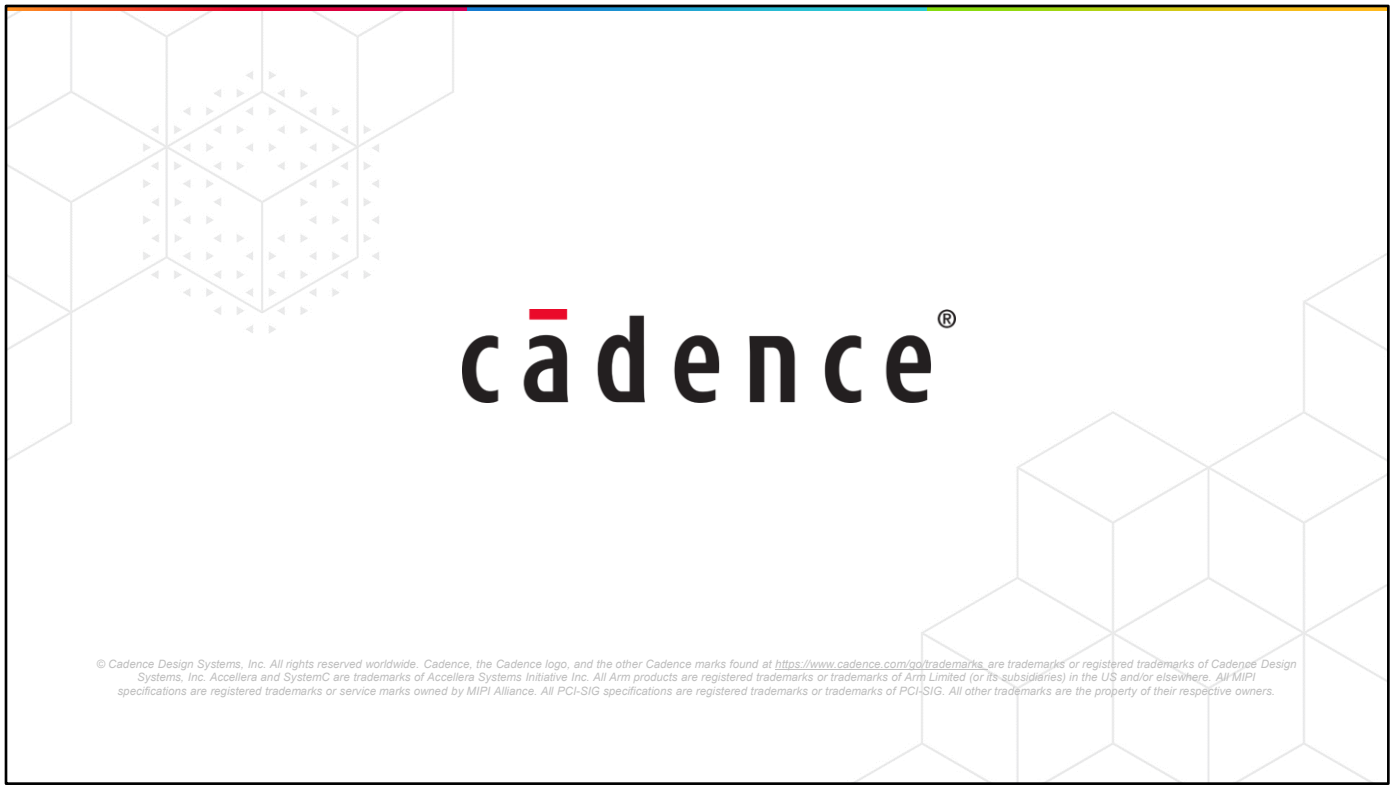
 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Rajat G of Analog



This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.